



Master's thesis

Carlo Rosso

Neural Network Simplification through Kolmogorov Complexity Bounding

Date: August 2026

Advisor: Raghavendra Selvan
Co-advisor: Pedram Bakhtiarifard

“Non sunt multiplicanda entia sine necessitate.”

(Entiteter bør ikke mangfoldiggøres uden nødvendighed)

(Entities must not be multiplied beyond necessity)

— **Vilhelm af Ockham**

Acknowledgements

First and foremost, I would like to express my deepest gratitude to my supervisors, **Raghavendra Selvan** and **Pedram Bakhtiarifard**, for welcoming me into this fascinating project. Throughout this journey, they listened to my ideas with immense patience, skillfully helping me filter out noisy intuitions while teaching me to navigate the trade-offs between an idea's complexity, ease of implementation, and the tangible insights we could gain from it. Their guidance helped me find my own footing as an independent thinker, fostered my scientific curiosity, and showed me both the unpredictability and the genuine joy of doing research.

I am profoundly grateful to my girlfriend for her constant love, patience, and care, especially during the demanding final stretch of writing this thesis. Her unwavering support and understanding kept me grounded when I needed it most.

My heartfelt thanks also go to my dear friend **Gaetano**, who is now pursuing his own dreams elsewhere, for his enduring friendship and shared conversations along the way. I also owe a special debt of gratitude to my **Danish family**, who first opened the door to the opportunity that brought me to Copenhagen; without their kindness and encouragement, completing my master's degree here would not have been possible.

Finally, this thesis is dedicated to my family. To my brother, for always being on the other end of the phone to listen, debate ideas, and offer thoughtful advice. And above all, to my parents, who have given me every sacrifice, opportunity, and support they could possibly offer. This milestone belongs to them as much as it does to me.

Department Department of Computer Science
 Author: Carlo Rosso
 Title : Neural Network Simplification through Kolmogorov Complexity Bounding
 Description: Modern deep neural networks achieve remarkable empirical success through over-parameterization, but this comes at the expense of high computational, memory, and energy costs. However, trained deep neural networks (DNNs) exhibit substantial structural redundancy, which we investigate through the lens of Algorithmic Information Theory (AIT) and Kolmogorov complexity. Building upon the Mosaic-of-Motifs (MoMos) framework (Bakhtiarifard et al., 2026a), we provide a comprehensive characterization of algorithmic compression in deep learning. First, we employ the Quantized Block Decomposition Method ($\hat{\mathcal{K}}_{\text{QuBD}}$) to evaluate the algorithmic complexity of DNNs, demonstrating that $\hat{\mathcal{K}}_{\text{QuBD}}$ tracks optimization dynamics to which previous binarized estimators are blind. Second, we formulate geometry-aware 2D block mappings aligned with internal network structures, showing that predictive accuracy is governed primarily by block size and establishing the spatial foundation for multi-scale architectures. Third, to address the description-length bottleneck of standard MoMos where mosaic storage dominates, we propose the first recursive step of MoMos, yielding competitive Pareto trade-offs on classification benchmarks. Finally, we bridge algorithmic information theory with statistical learning theory by proving the conditions under which MoMos preserves full linear Vapnik-Chervonenkis (VC) dimension across canonical standard-basis configurations. Together, these results provide theoretical and empirical foundations for designing neural networks with strictly bounded algorithmic complexity.
 Advisor: Raghavendra Selvan
 Co-Advisor: Pedram Bakhtiarifard
 Date: 16 08 2026

Table of content

Acknowledgements	3
Notation	7
1 Introduction	8
1.1 Thesis Contributions	8
1.2 Document Structure	9
2 Kolmogorov Complexity	11
2.1 Universal Turing Machine	11
2.1.1 The Halting Problem	13
2.2 Measuring the Amount of Information	14
2.2.1 Algorithmic Complexity	15
2.2.2 Algorithmic Prefix Complexity	16
2.3 Approximating Kolmogorov Complexity	18
2.3.1 Coding Theorem Method (CTM)	18
2.3.2 Block Decomposition Method (BDM)	19
2.3.3 Quantized Block Decomposition Method (QuBD)	21
3 Neural Networks with Weight-Sharing Constraints	24
3.1 Soft-Weight Sharing	25
3.1.1 Probabilistic Formulation and Objective	25
3.2 Deep Compression	27
3.2.1 Pruning, Trained Quantization, and Entropy Coding	27
3.3 Hyper-Compression	28
3.3.1 Ergodic Regeneration	29
4 Mosaic-of-Motifs Characterization	31
4.1 Mosaic-of-Motifs Characterization	32
4.1.1 Complexity and Hypothesis Space Cardinality	32
4.1.2 Algorithmic Implementation	34

4.2	2D Mosaic-of-Motifs	34
4.2.1	Complexity of 2D MoMos	35
4.3	Empirical Results	36
4.3.1	Capacity Metric and Theoretical Critique	36
4.3.2	Compression Ratio	37
4.3.3	Training Dynamics and Algorithmic Complexity	38
4.3.4	Long-Run Convergence and Soft Regularization Dynamics	39
4.4	Discussion and Future Directions	40
5	The V-Fold MoMos Parameterization	43
5.1	Theoretical Motivation and Recursive Limitations	44
5.2	The V-Fold MoMos Framework	44
5.2.1	Formal Definition and Hypothesis Class	45
5.2.2	Hypothesis Space Cardinality	45
5.2.3	Description-Length Upper Bound	48
5.3	Algorithmic Implementation	48
5.3.1	Worked Reconstruction Example	50
5.4	Capacity Scaling and Structural Regularization	51
5.5	Empirical Evaluation	53
5.5.1	Pareto Analysis and Cross-Validation Verification	53
5.5.2	Storage Allocation and Codebook Breakdown	55
5.5.3	Training Dynamics and Algorithmic Complexity Trajectories	56
5.5.4	Long-Run Convergence and Motifs Clustering	58
5.6	Chapter Summary	58
6	Toward Generalization Guarantees for MoMos	62
6.1	Generalization and Algorithmic Complexity Bounds	63
6.2	Vapnik–Chervonenkis Dimension and Linear Models	65
6.2.1	VC Dimension and Hypothesis Space Cardinality	65
6.2.2	VC Dimension of Linear Models	66
6.3	MoMos-Parameterized Linear Models	67
6.4	Chapter Summary	70
7	Conclusion and Future Directions	72
7.1	Summary of Contributions	72
7.2	Key Insights	73
7.3	Practical Implications	74
7.4	Limitations	75
7.5	Future Research Directions	76

Notation

The following notation and abbreviations are used throughout this thesis.

Notation	Meaning
$[k]$	The index set $\{1, 2, \dots, k\}$ of k indices.
$\mathbf{A}[\mathbf{i}]$	Indexing (gather) operator: selects the entries (or columns) of $\mathbf{A}$ listed by the index vector $\mathbf{i}$; e.g., $\mathcal{Z}[\mathcal{M}]$ gathers the motif columns of $\mathcal{Z}$ named by the mosaic $\mathcal{M}$.
x_2	Binary representation of an object or integer x : a binary string in $\{0, 1\}^*$ denoted with a base-2 subscript $_2$ at the end (e.g., 1010_2).
$\mathbf{1}(P)$	Indicator function: evaluates to 1 if predicate P is true, and 0 otherwise.
$f(x) \leq \mathcal{O}(g(x))$	Asymptotic upper bound: $f(x) \leq c \cdot g(x)$ as $x \rightarrow \infty$, for a fixed constant $c \in \mathbb{N}$.
$\hat{\mathcal{K}}_\phi$	Computable approximation or upper bound of the Kolmogorov complexity $\mathcal{K}(\cdot)$ evaluated under method or parameterization ϕ (e.g., $\hat{\mathcal{K}}_{\text{CTM}}$, $\hat{\mathcal{K}}_{\text{BDM}}$, $\hat{\mathcal{K}}_{\text{QuBD}}$).
r_ϕ	Description-length compression ratio $nq/\hat{\mathcal{K}}_\phi(\mathbf{w})$, where nq is the uncompressed full-precision baseline (n parameters at q bits) and $\hat{\mathcal{K}}_\phi(\mathbf{w})$ is the description length under method ϕ ; higher values indicate stronger compression.
MLP	Multi-layer perceptron.
CNN	Convolutional neural network.

Chapter 1

Introduction

Modern deep learning achieves state-of-the-art performance across computer vision, natural language processing, and scientific modeling by scaling neural networks to billions of parameters. However, this scale incurs substantial computational latency, memory footprints, and energy costs during training and inference (Bakhtiarifard et al., 2025). Despite this high dimensionality, trained networks exhibit substantial structural redundancy: large proportions of learned weights can be pruned, quantized, or tied together with minimal degradation in task accuracy (Han et al., 2016; Ullrich et al., 2017; Frankle and Carbin, 2019).

Traditional capacity measures rely on scalar parameter counts or statistical Shannon entropy. These metrics evaluate unstructured marginal distributions, failing to capture deterministic regularities, spatial repetitions, or modular reuse across weight tensors (Sakabe et al., 2025). Algorithmic Information Theory (AIT) provides a mathematically principled framework to quantify and exploit such redundancy (Sakabe et al., 2025; Bakhtiarifard et al., 2026a,b). Through the lens of Kolmogorov complexity, a model is simple not when it has few scalar parameters, but when it can be generated by a short program.

1.1 Thesis Contributions

This thesis investigates the theoretical foundations, architectural designs, and empirical dynamics of neural network simplification under algorithmic complexity constraints. Building upon the Mosaic-of-Motifs (MoMos) parameterization (Bakhtiarifard et al., 2026a), we analyze how structural weight sharing bounds Kolmogorov complexity during gradient optimization, develop hierarchical parameterizations, and establish generalization guarantees.

The key contributions of this work are summarized as follows:

1. **Algorithmic Complexity Tracking via $\hat{\mathcal{K}}_{\text{QuBD}}$:** Prior evaluations of neural network algorithmic complexity (Sakabe et al., 2025) relied on binarized $\hat{\mathcal{K}}_{\text{BDM}}$, which discards continuous weight precision via sign thresholding. In this thesis, we adopt the Quantized Block

Decomposition Method ($\hat{\mathcal{K}}_{\text{QuBD}}$) (Bakhtiarifard et al., 2026b) to evaluate multi-bit weight configurations. We demonstrate that $\hat{\mathcal{K}}_{\text{QuBD}}$ preserves magnitude distributions across bit-planes and faithfully tracks optimization dynamics and loss progression throughout training.

2. **Empirical Analysis of Geometry-Aware 2D Block Mappings:** Standard MoMos relies on a 1D flattening operator. We design and implement two-dimensional slicing operators that partition weight tensors along architectural dimensions, such as input features in Multi-Layer Perceptrons (MLPs) and receptive field channels in Convolutional Neural Networks (CNNs). Our empirical analysis reveals that classification accuracy clusters primarily by block size s , irrespective of block geometry.
3. **Development of the V-Fold MoMos Parameterization:** In standard MoMos, the mosaic index map $\mathcal{M}$ dominates description length ($m \lceil \log_2 k \rceil$ bits). We introduce V-Fold MoMos, which partitions parameter assignments into v localized folds, restricting each block to a local sub-codebook of size $k' \leq k$. This reduces mosaic storage to $\lceil \log_2 k' \rceil$ bits per entry while maintaining a shared global motif codebook. V-Fold MoMos improves structural flexibility and achieves competitive Pareto trade-offs on MLPs.
4. **Generalization Guarantees and VC-Dimension Bounds:** We establish a theoretical bridge between algorithmic parameter compression and statistical learning theory. First, we apply countable-hypothesis PAC bounds (Wilson, 2025) to prove that reducing Kolmogorov complexity strictly tightens the generalization gap. Second, we analyze the combinatorial expressivity of motif parameterizations via the Vapnik-Chervonenkis (VC) dimension, proving the conditions under which MoMos preserves full linear VC dimension ($d_{\text{VC}} = n$) on standard-basis configurations.

1.2 Document Structure

The remainder of this thesis is organized as follows:

- **Chapter 2 (Kolmogorov Complexity):** Establishes the foundations of Algorithmic Information Theory, defining the Universal Turing Machine, prefix Kolmogorov complexity, and computable approximations ($\hat{\mathcal{K}}_{\text{CTM}}$, $\hat{\mathcal{K}}_{\text{BDM}}$, and $\hat{\mathcal{K}}_{\text{QuBD}}$).
- **Chapter 3 (Neural Networks with Weight-Sharing Constraints):** Surveys post-training weight-sharing paradigms, focusing on Soft Weight-Sharing (Ullrich et al., 2017), Deep Compression (Han et al., 2016), and Hyper-Compression (Fan et al., 2026).
- **Chapter 4 (Mosaic-of-Motifs Characterization):** Formalizes the Mosaic-of-Motifs (MoMos) parameterization (Bakhtiarifard et al., 2026a), derives description-length upper bounds, and analyzes 2D geometry-aware mappings on CIFAR-10.

- **Chapter 5 (The V-Fold MoMos Parameterization):** Introduces the V-Fold MoMos framework, formulating hierarchical sub-codebook partitioning to compress mosaic indexing overhead while retaining competitive classification performance.
- **Chapter 6 (Toward Generalization Guarantees for MoMos):** Connects algorithmic complexity to statistical learning theory via countable-hypothesis PAC bounds, VC dimension, and Cover’s function-counting theorem.
- **Chapter 7 (Conclusion and Future Directions):** Synthesizes the primary theoretical and empirical findings of this thesis, evaluates methodological limitations, and outlines directions for future research.

Chapter 2

Kolmogorov Complexity

In this work, we bridge algorithmic information theory (AIT) with machine learning, specifically for deep neural networks scaling to trillions of parameters. As shown in recent work, the complexity of neural networks, measured via algorithmic complexity, strongly correlates with training dynamics and loss progression throughout optimization (Bakhtiarifard et al., 2026b; Sakabe et al., 2025; Bakhtiarifard et al., 2026a).

In Section 2.1, we define the Universal Turing Machine. In Section 2.2, we define algorithmic complexity as the length of the shortest program that, when executed on a Universal Turing Machine (UTM), outputs a given object. We then demonstrate why Kolmogorov complexity is uncomputable through a reformulation of Gödel’s incompleteness result (Li and Vitányi, 2008). In Section 2.3, we introduce computable approximations of the algorithmic complexity, such as the Coding Theorem Method (CTM), the Block Decomposition Method (BDM), and the Quantized Block Decomposition Method (QuBD), which upper-bound Kolmogorov complexity and overcome its uncomputability.

2.1 Universal Turing Machine

The description length of an object is an intrinsic property corresponding to the number of bits required for its representation. Here, we assume that the target object is representable as a finite binary string and thus possesses a computable description, which may be the string itself or a program that generates it.

Consequently, description length depends on the underlying computational model executing the program. As shown below, this dependence is minor because a description can be translated to a different computational model with an additive constant overhead.

To formalize this concept, we must define what constitutes a computer and computability itself. First proposed by Alan Turing, the theoretical model of computation is given by the Turing Machine (TM), which formalizes the limits of computation and models any physical computing

device. However, certain problems remain fundamentally unsolvable by any computing device, with exact Kolmogorov complexity being an example.

We adopt the formalization of Turing machines from Sipser (2013) to establish the operational foundation of computation:

Definition 2.1 (Turing Machine (TM)). A Turing machine is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{acp}, q_{rej})$, where Q , Σ , and Γ are finite sets subject to:

1. Q is the set of states;
2. Σ is the input alphabet, which does not contain the blank symbol $\sqcup$;
3. Γ is the tape alphabet, where $\Sigma \subseteq \Gamma$ and $\sqcup \in \Gamma$;
4. $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the transition function;
5. $q_0 \in Q$ is the initial state;
6. $q_{acp} \in Q$ is the accept state;
7. $q_{rej} \in Q$ is the reject state, with $q_{rej} \neq q_{acp}$.

Given an input string w of length n , the TM $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{acp}, q_{rej})$ operates according to Algorithm 1.

Algorithm 1 Execution of a TM M on input w

```

1:  $tape \leftarrow$  array of blank symbols  $\sqcup$ 
2:  $tape[1:n] \leftarrow w$ 
3:  $position \leftarrow 1$ 
4:  $state \leftarrow q_0$ 
5: while  $state \notin \{q_{acp}, q_{rej}\}$  do
6:    $(state, b, action) \leftarrow \delta(state, tape[position])$ 
7:    $tape[position] \leftarrow b$ 
8:    $position \leftarrow position - 1$  if  $action = L$  else  $position + 1$ 
9:   if  $position = 0$  then
10:     $position \leftarrow 1$ 
11:   end if
12: end while
13: return  $state$ 

```

Unless stated otherwise, we assume without loss of generality that $\Sigma = \{0, 1\}$. This constraint does not limit the capabilities of the Turing machine, because any finite alphabet of cardinality k can be mapped to binary sequences of length $\lceil \log_2 k \rceil$.

To track the execution state of a TM at any step during computation, we define a configuration:

Definition 2.2 (Configuration). A configuration of a TM is a triple (q, x, y) , where $q \in Q$ is the current state, $x \in \Gamma^*$ is the string on the tape preceding the current position, and $y \in \Gamma^*$ is the non-empty string starting at the current position.

The initial configuration on input w is (q_0, ϵ, w) , where ϵ is the empty string.

A configuration is an *accepting configuration* if its state is q_{acc} ; analogously, it is a *rejecting configuration* if its state is q_{rej} . Accepting and rejecting configurations terminate the computation of the TM and are called *halting configurations*.

A TM is a *decider* if it halts on every input.

Given a TM M , the set of strings w for which M halts in the accept state q_{acc} is called the *language recognized by M* .

To formalize computations that halt deterministically on all inputs, we define decidable languages:

Definition 2.3 (Decidable Language). A language is decidable if there exists a decider that recognizes it.

A TM defined thus far can be interpreted as a program: upon receiving an input, it performs a predetermined sequence of steps and computes an output. Specifically, a TM may loop indefinitely, or it may be stuck computing an infinite sequence, such as the digits of a real number. Otherwise, the TM terminates in a halting state, either accepting or rejecting.

A general-purpose computer, on the other hand, can execute any program provided that the program is implemented on its alphabet and state space. Similarly, we define the Universal Turing Machine (UTM) as a TM capable of simulating any other TM. To reconcile differences in state sets and alphabets, we introduce a canonical string encoding: let $\langle o \rangle \in \{0, 1\}^*$ be a binary encoding of any object o that admits a binary representation.

To model programmable computing devices capable of executing arbitrary programs encoded as binary strings, we introduce the Universal Turing Machine:

Definition 2.4 (Universal Turing Machine (UTM)). A Turing machine U is *universal* if, for any Turing machine M and any input string $w \in \Sigma^*$, $U(\langle M, w \rangle) = M(w)$.

This equivalence implies that if $M(w)$ halts, $U(\langle M, w \rangle)$ halts with identical output, and if $M(w)$ loops indefinitely, $U(\langle M, w \rangle)$ loops indefinitely.

2.1.1 The Halting Problem

Before proving that Kolmogorov complexity is uncomputable, we formalize the undecidability of the acceptance problem A_{TM} , as the uncomputability proof for Kolmogorov complexity relies on an analogous diagonalization argument.

Theorem 1 (Undecidability of A_{TM}). Let A_{TM} be the language defined as

$$A_{\text{TM}} := \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w \}. \quad (2.1)$$

There is no decider that recognizes A_{TM} ; hence, A_{TM} is undecidable.

Proof ([Sipser, 2013](#)). Assume by contradiction that A_{TM} is decidable, and let H be a decider for A_{TM} . On input $\langle M, w \rangle$, where M is a TM and w is a string, H halts and accepts if M accepts w , and halts and rejects if M fails to accept w . That is, H is a TM satisfying

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ accepts } w, \\ \text{reject} & \text{if } M \text{ does not accept } w. \end{cases} \quad (2.2)$$

We construct a new Turing machine D that uses H as a subroutine. Machine D calls H to determine the behavior of M when given its own description $\langle M \rangle$ as input. Once D obtains this outcome, it outputs the opposite result: it rejects if M accepts $\langle M \rangle$, and accepts if M does not accept $\langle M \rangle$.

Executing a machine on its own description is mathematically well-defined, analogous to passing a program's source code to itself as input (e.g., a Haskell compiler written in Haskell compiling its own source code). In summary,

$$D(\langle M \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ does not accept } \langle M \rangle, \\ \text{reject} & \text{if } M \text{ accepts } \langle M \rangle. \end{cases} \quad (2.3)$$

Evaluating D on its own description $\langle D \rangle$ yields

$$D(\langle D \rangle) = \begin{cases} \text{accept} & \text{if } D \text{ does not accept } \langle D \rangle, \\ \text{reject} & \text{if } D \text{ accepts } \langle D \rangle. \end{cases} \quad (2.4)$$

This implies that D accepts $\langle D \rangle$ if and only if D does not accept $\langle D \rangle$, which is a contradiction. Consequently, neither D nor H can exist, proving that A_{TM} is undecidable. $\square$

In the next section, we introduce the formal measure of information and explain its mathematical motivation. To do so, we rely on computability theory and UTMs to establish a rigorous baseline for what is computable.

2.2 Measuring the Amount of Information

In this section, we present a framework for measuring the information content of an object that admits an exact binary representation, allowing the original object to be reconstructed completely

from its description.

Consider the following two binary strings:

$$A := 0101010101, \quad B := 0110101110. \quad (2.5)$$

Both strings consist of 10 bits, so $l(A) = l(B) = 10$. Intuitively, string A admits a shorter representation because it consists of the pattern 01 repeated five times; conversely, string B exhibits no obvious pattern, suggesting no trivial shorter representation.

We define the amount of information contained in an object as the length of its shortest representation or description. Thus, string A contains less information than string B because it possesses a shorter description.

When measuring information content, we restrict our attention to the shortest description. It is always possible to describe an object using the same object, but such a trivial description is never shorter than the object and yields no insight into its intrinsic information content.

2.2.1 Algorithmic Complexity

One way to use algorithms to describe binary strings is to fix a UTM that, given an input program, computes the output and halts in an accepting state. The resulting output string on the tape is the object described by the input program.

To establish program execution lengths on a fixed UTM as an objective complexity measure, we define algorithmic complexity:

Definition 2.5 (Algorithmic Complexity). Let x be a binary string and fix a UTM U . The algorithmic complexity of x relative to U is the length of the shortest program w that, when executed on U , outputs x :

$$\mathcal{C}_U(x) = \min \{l(w) \mid U(w) = x\}. \quad (2.6)$$

Below, we show that the algorithmic complexity of arbitrary objects is uncomputable, which implies that alternative tools are needed to reason about the amount of information contained in an object.

Theorem 2 (Uncomputability of Algorithmic Complexity). Let x be an object with a binary representation. Its algorithmic complexity $\mathcal{C}(x)$ is uncomputable.

Proof (*Li and Vitányi, 2008*). Assume by contradiction that $\mathcal{C}(x)$ is computable. Then there exists a Turing machine M that, on input n (encoded using $\lceil \log_2 n \rceil$ bits), operates as follows:

1. Sequentially evaluates $\mathcal{C}(x)$ for all binary strings x of length n in lexicographical order.
2. Outputs the first string x satisfying $\mathcal{C}(x) \geq n$.

By construction of M , the output string x has length n and satisfies $\mathcal{C}(x) \geq n$. On the other hand, M combined with input n constitutes an effective description of x . The machine M itself has a fixed description length of c bits, while the input n requires $\lceil \log_2 n \rceil$ bits.

Consequently, $M(n)$ yields a program that outputs x , establishing an upper bound on its algorithmic complexity:

$$\mathcal{C}(x) \leq \lceil \log_2 n \rceil + c. \quad (2.7)$$

Combining both inequalities yields:

$$n \leq \mathcal{C}(x) \leq \lceil \log_2 n \rceil + c. \quad (2.8)$$

As $n \rightarrow \infty$, the logarithmic term $\lceil \log_2 n \rceil + c$ grows strictly slower than n , leading to a contradiction for sufficiently large n . Therefore, $\mathcal{C}(x)$ cannot be computable. $\square$

The definition of plain algorithmic complexity lacks subadditivity: for binary strings x and y , we have

$$\mathcal{C}_U(\langle x, y \rangle) \leq \mathcal{C}_U(x) + \mathcal{C}_U(y) + 2 \log_2(\min(\mathcal{C}_U(x), \mathcal{C}_U(y))) + O(1), \quad (2.9)$$

because a UTM requires the shortest programs for both x and y , as well as the length of the shorter description, to delimit their separation (Li and Vitányi, 2008). In particular, any object x carries two distinct types of information: it expresses the bit patterns of which it is composed and preserves the string length. When two strings are concatenated, the information regarding string length is lost. In the next section, we introduce algorithmic prefix complexity, which satisfies subadditivity and consequently establishes cleaner algebraic properties.

2.2.2 Algorithmic Prefix Complexity

Algorithmic complexity and algorithmic prefix complexity are commonly referred to as plain Kolmogorov complexity (or plain Kolmogorov–Chaitin complexity) and prefix-free Kolmogorov complexity (or prefix-free Kolmogorov–Chaitin complexity), respectively. Throughout the remainder of this work, we refer to prefix-free Kolmogorov complexity simply as Kolmogorov complexity.

To simplify the algebraic formulations of object complexity, we adopt algorithmic prefix complexity. This requires defining a new class of UTMs: a prefix-free universal Turing machine recognizes only a prefix-free language, where no string is a proper prefix of any other string in the same language. Consequently, a TM recognizing such a language can always determine the termination of any input string. In the remainder of this work, we use UTM to denote a prefix-free universal Turing machine and TM to denote a prefix-free Turing machine.

To establish subadditivity and enforce self-delimiting program boundaries, we restrict computation to prefix-free UTMs:

Definition 2.6 (Algorithmic Prefix Complexity). Let x be a binary string and fix a prefix-free universal Turing machine U (where the domain $\text{Dom}(U)$ of valid programs is a prefix-free code). The algorithmic prefix complexity of x relative to U is the length of the shortest program $w \in \text{Dom}(U)$ that, when executed on U , outputs x :

$$\mathcal{K}_U(x) = \min \{l(w) \mid U(w) = x\}. \quad (2.10)$$

Note that we defined the information content of an object based on the shortest description length relative to a fixed UTM U . Because a UTM can itself be described as a binary string, one might consider the joint complexity of the UTM and the shortest program executing on it. The following theorem demonstrates that, for any fixed UTM U , the Kolmogorov complexity on U is optimal up to an additive constant because any UTM U can simulate another UTM U' , and the simulator program required for this conversion has fixed length independent of the target object x .

Theorem 3 (Invariance Theorem). For any two UTMs U and U' , there exists a fixed prefix program p such that $U'(w) = U(\langle p, w \rangle)$ for all strings w , where $l(p) \leq c_{U,U'}$ for some constant $c_{U,U'} \in \mathbb{N}$ depending only on U and U' . Choosing w as the shortest program such that $U'(w) = x$, we obtain:

$$\mathcal{K}_U(x) \leq l(w) + l(p) = \mathcal{K}_{U'}(x) + c_{U,U'}. \quad (2.11)$$

This theorem allows us to consider the shortest algorithmic description of an object independently of the chosen UTM. In practice, this means that the shortest description on a fixed UTM is asymptotically close to the shortest description on any UTM. Hence, we can reason about the shortest algorithmic description independently of the physical computing device and the chosen programming language.

In practical settings, we study the Kolmogorov complexity of large objects, such as deep neural networks containing millions or billions of parameters. Conversely, minimalist UTMs can be specified in a few hundred bits (e.g., 425 bits [Li and Vitányi \(2008\)](#)). Consequently, fixed additive constants are negligible compared to the scale of the objects under consideration.

In the remainder of this work, we refer to Kolmogorov complexity as $\mathcal{K}(\cdot)$, independently of the chosen UTM.

This completes the theoretical foundation of Kolmogorov complexity. In the remainder of this chapter, we review computable approximation methods from the literature, focusing on approaches designed for large objects whose description lengths far exceed the complexity of the underlying UTM. In particular, we concentrate on estimation methods tailored to deep neural networks.

2.3 Approximating Kolmogorov Complexity

Because Kolmogorov complexity is uncomputable ([Theorem 2](#)), practical applications necessitate accurate complexity estimates. As shown by [Valle-Pérez et al. \(2019\)](#), deep neural networks generalize well in practice because the parameter-to-function map is intrinsically biased toward solutions with low algorithmic complexity.

To evaluate these theoretical connections empirically, several frameworks have been developed to approximate Kolmogorov complexity. We begin by introducing the Coding Theorem Method (CTM), which provides exact algorithmic complexity estimates for small discrete objects.

2.3.1 Coding Theorem Method (CTM)

CTM defines an empirical probability distribution over Turing machines by running small TMs to compute the frequency of the objects they generate. Given the algorithmic probability of an object estimated via CTM, its prefix Kolmogorov complexity can be calculated up to an additive constant. For small objects, this additive constant is relatively large compared to the object size. Additionally, CTM can simulate only small TMs that are guaranteed to halt.

Formally, consider a prefix-free UTM U whose input tape is fed by a sequence of independent fair coin flips, where each bit is 0 or 1 with probability $1/2$. The probability that a random input sequence begins with a prefix program p of length $l(p)$ is $2^{-l(p)}$.

To quantify the likelihood that a randomly generated program produces a specific output on a prefix-free UTM, we define universal algorithmic probability:

Definition 2.7 (Universal Algorithmic Probability). The universal algorithmic probability $m(x)$ of a binary string $x \in \{0, 1\}^*$ relative to a prefix-free UTM U is the probability that U outputs x when provided with a random input stream:

$$m(x) := \sum_{p: U(p)=x} 2^{-l(p)}. \quad (2.12)$$

By Kraft's inequality ($\sum_{p \in \text{Dom}(U)} 2^{-l(p)} \leq 1$), the total probability mass $\sum_x m(x) \leq 1$ can be strictly smaller than 1 because U may not halt on all input streams. Consequently, $m(x)$ forms a universal semi-measure over binary strings rather than a full probability distribution. Intuitively, $m(x)$ assigns higher probability to strings that can be produced by many short programs, reflecting an intrinsic simplicity bias in random computation.

Levin's Coding Theorem relates universal algorithmic probability $m(x)$ directly to prefix Kolmogorov complexity $\mathcal{K}(x)$.

Theorem 4 (Levin's Coding Theorem ([Li and Vitányi, 2008](#))). For any binary string $x \in \{0, 1\}^*$ and fixed UTM U ,

$$-\log_2 m(x) = \mathcal{K}(x) + O(1). \quad (2.13)$$

Theorem 4 shows that Kolmogorov complexity is directly determined by the output distribution of random programs, and vice versa. This relationship requires a valid probability distribution over programs ($\sum_x m(x) \leq 1$), which is guaranteed by forcing the underlying UTM to be prefix-free.

Empirical Estimation via TM Enumeration

While $m(x)$ is uncomputable (as computing it would allow computing $\mathcal{K}(x)$, violating **Theorem 2**), CTM approximates $m(x)$ empirically for small objects by enumerating small deterministic Turing machines up to a fixed number of states k and alphabet symbols m , executed for up to T steps ([Sakabe et al., 2025](#)).

Let $\mathcal{M}_{k,m}$ denote the space of Turing machines with k states and m symbols. By executing every $M \in \mathcal{M}_{k,m}$ and recording the halting machines within T steps, CTM computes the frequency distribution:

$$D(k, m)(x) := \frac{|\{M \in \mathcal{M}_{k,m} \mid M \text{ halts within } T \text{ steps and outputs } x\}|}{|\{M \in \mathcal{M}_{k,m} \mid M \text{ halts within } T \text{ steps}\}|}. \quad (2.14)$$

By replacing uncomputable universal probability with empirical frequencies from enumerated small Turing machines, we obtain the CTM estimator:

Definition 2.8 (Coding Theorem Method). Using $D(k, m)(x)$ as an empirical surrogate for algorithmic probability $m(x)$, the Coding Theorem Method (CTM) estimate of the Kolmogorov complexity of x is defined as:

$$\hat{\mathcal{K}}_{\text{CTM}}(x) := -\log_2 D(k, m)(x). \quad (2.15)$$

In practice, standard CTM implementations utilize dynamic programming to accelerate this computation.

CTM suffers from a severe operational bottleneck: the state space of Turing machines grows super-exponentially as $O((2km)^{km})$. As established by [Sakabe et al. \(2025\)](#), exact TM enumeration is computationally feasible only for 1D strings up to length 11 bits ($n \leq 11$) and 2D binary grids up to size 4×4 . This limitation necessitates a multi-scale decomposition strategy, leading to the Block Decomposition Method, which is explained in the next section.

2.3.2 Block Decomposition Method (BDM)

A natural starting point for measuring the complexity of a neural network is the Shannon entropy: it quantifies the expected information content of a system purely from the probability distribution over its possible states. Applied to the weight distribution of a neural network, Shannon entropy offers a tractable and theoretically grounded indicator of the amount of redundancies in the neural network.

To establish a statistical baseline for information content based on symbol frequencies alone, we state the definition of Shannon entropy:

Definition 2.9 (Shannon Entropy). For a discrete random variable X taking values in an alphabet $\mathcal{A}$ with probability distribution $P(x)$, Shannon entropy is defined as:

$$H(X) := - \sum_{x \in \mathcal{A}} P(x) \log_2 P(x). \quad (2.16)$$

Shannon entropy is simple to compute, but it fails to account for the algorithmic complexity of patterns within the object. For example, deterministic sequences generated by simple algorithms, such as the binary expansion of π , exhibit high statistical complexity despite possessing low Kolmogorov complexity $\mathcal{K}(x)$. BDM addresses Shannon entropy's inability to detect algorithmic structure by using [Definition 2.8](#) to evaluate the complexity of local patterns. Simultaneously, BDM overcomes CTM's computational intractability on large objects by partitioning a binary tensor or string into sub-blocks and approximating global algorithmic complexity through a logarithmic cost on block frequencies.

To scale algorithmic complexity estimation to large objects while mitigating TM enumeration constraints, BDM partitions strings into sub-blocks evaluated via CTM:

Definition 2.10 (Block Decomposition Method). Let X be a binary tensor or string partitioned into a multiset of smaller sub-blocks $\{x_i\}$ of fixed block size l . Let $\{x_1, x_2, \dots, x_k\}$ denote the set of distinct sub-blocks in the partition, where each unique block x_i occurs with multiplicity $n_i \geq 1$. The BDM complexity estimate of X is defined as:

$$\hat{\mathcal{K}}_{\text{BDM}}(X) := \sum_{i=1}^k \hat{\mathcal{K}}_{\text{CTM}}(x_i) + \log_2(n_i). \quad (2.17)$$

BDM evaluates the exact algorithmic complexity of each unique sub-block once via [Definition 2.8](#), while penalizing block repetitions with a logarithmic cost $\log_2(n_i)$. This logarithmic correction directly reflects the subadditivity property of Kolmogorov complexity ($\mathcal{K}(x^m) \leq \mathcal{K}(x) + \log_2 m + O(1)$).

The theoretical behavior of BDM depends directly on block size l relative to object size $|X|$. If object size $|X|$ is fixed while block size increases toward $l \rightarrow |X|$, the partition collapses to a single block ($k = 1, n_1 = 1$), and BDM converges to the CTM estimate and true Kolmogorov complexity:

$$\lim_{l \rightarrow |X|} \hat{\mathcal{K}}_{\text{BDM}}(X) = \hat{\mathcal{K}}_{\text{CTM}}(X) \approx \mathcal{K}(X). \quad (2.18)$$

Conversely, when block size l remains fixed while object length $|X| \rightarrow \infty$, global block frequency dominates the local CTM values. The behavior of BDM asymptotically aligns with the behavior of Kolmogorov complexity ([Li and Vitányi, 2008](#)) (Example 2.8.1).

In machine learning, [Sakabe et al. \(2025\)](#) provided the first empirical application of BDM to neural network parameters. By tracking BDM complexity metrics during Binary Neural Network (BNN) optimization, they showed that gradient-based optimization drives network weights toward lower algorithmic complexity $\mathcal{K}(x)$. This finding offers strong support for the “learning-as-compression” hypothesis, confirming that SGD acts as an algorithmic compressor searching for short parameter descriptions.

2.3.3 Quantized Block Decomposition Method (QuBD)

BDM ([Definition 2.10](#)) and CTM ([Definition 2.8](#)) enable local algorithmic complexity estimation for binary strings and small discrete grids. However, when BDM is applied to large deep neural networks (DNNs), it presents two fundamental bottlenecks: single-bit binarization discards fine-grained numerical precision, while full bit-string concatenation causes the global multiplicity penalty to dominate, asymptotically converging toward Shannon entropy ([Zenil et al., 2022](#)).

To resolve these limitations, [Bakhtiarifard et al. \(2026b\)](#) introduced the Quantized Block Decomposition Method (QuBD), which extends algorithmic complexity estimation to continuous, multi-bit parameters by decomposing quantized weight tensors into hierarchical bit-planes.

Uniform Quantization

To evaluate the algorithmic complexity of continuous parameters using discrete block decomposition methods, continuous weight values must first be mapped into a discrete, finite representation. For this purpose, QuBD applies uniform scalar quantization ([Han et al., 2016](#); [Ullrich et al., 2017](#)) to project continuous weight vectors into integer codes of fixed bit width.

Definition 2.11 (Uniform Scalar Quantization). Let $\mathbf{w} = (w_1, w_2, \dots, w_n) \in [\mathbf{w}_{\min}, \mathbf{w}_{\max}]^n \subset \mathbb{R}^n$ be a continuous weight tensor. Given a target precision of q bits, the uniform scalar quantization function $Q_q : [\mathbf{w}_{\min}, \mathbf{w}_{\max}] \rightarrow \{1, \dots, 2^q\}$ maps each component weight $w_i \in \mathbb{R}$ to a discrete integer code q_i :

$$q_i = Q_q(w_i) := \text{round} \left(\frac{w_i - \mathbf{w}_{\min}}{\Delta} \right), \quad \text{where } \Delta = \frac{\mathbf{w}_{\max} - \mathbf{w}_{\min}}{2^q}. \quad (2.19)$$

Following quantization via [Definition 2.11](#), each symbol $q_i \in \{1, \dots, 2^q\}$ is uniquely expanded into a binary sequence of length q :

$$q_i = \sum_{\ell=1}^q b_{i,\ell} 2^{\ell-1}, \quad b_{i,\ell} \in \{0, 1\}, \quad (2.20)$$

where $b_{i,q}$ denotes the Most Significant Bit (MSB) and $b_{i,1}$ denotes the Least Significant Bit (LSB).

Then, QuBD organizes the parameter vector into q orthogonal binary *bit-planes* $\beta_1, \dots, \beta_q \in \{0, 1\}^n$. The ℓ -th bit-plane β_ℓ gathers the ℓ -th significance bit across all n

parameters:

$$\beta_\ell := (b_{1,\ell}, b_{2,\ell}, \dots, b_{n,\ell})^T. \quad (2.21)$$

To illustrate this bit-plane grouping, consider four quantized parameters with integer values $q_1 = 1$, $q_2 = 8$, $q_3 = 12$, and $q_4 = 15$ ($n = 4, q = 4$). Their 4-bit binary expansions are:

$$q_1 = 0001_2, \quad q_2 = 1000_2, \quad q_3 = 1100_2, \quad q_4 = 1111_2. \quad (2.22)$$

QuBD collects bits across parameters by significance level into four distinct bit-planes:

$$\beta_4 = (0, 1, 1, 1)^T \implies 0111_2, \quad (2.23)$$

$$\beta_3 = (0, 0, 1, 1)^T \implies 0011_2, \quad (2.24)$$

$$\beta_2 = (0, 0, 0, 1)^T \implies 0001_2, \quad (2.25)$$

$$\beta_1 = (1, 0, 0, 1)^T \implies 1001_2. \quad (2.26)$$

Formal Definition and Theoretical Guarantees

To construct a multi-scale algorithmic complexity metric for continuous parameter spaces, QuBD evaluates the BDM complexity of each binary bit-plane independently and aggregates the resulting estimates:

Definition 2.12 (Quantized Block Decomposition Method (QuBD)). Let $\mathbf{w} \in \mathbb{R}^n$ be a parameter vector quantized to q -bit integer codes via [Definition 2.11](#) and decomposed into bit-planes $\beta_1, \beta_2, \dots, \beta_q \in \{0, 1\}^n$. The QuBD complexity estimate of $\mathbf{w}$ is defined as:

$$\hat{\mathcal{K}}_{\text{QuBD}}(\mathbf{w}) := \sum_{\ell=1}^q \hat{\mathcal{K}}_{\text{BDM}}(\beta_\ell). \quad (2.27)$$

[Bakhtiarifard et al. \(2026b\)](#) established formal error bounds demonstrating that QuBD yields a strictly tighter estimate of true Kolmogorov complexity $\mathcal{K}(\mathbf{w})$ than binarization-based BDM. Approximating the target weight vector from its q most significant bit-planes y_q incurs a residual complexity error $R_q = \mathcal{C}(\mathbf{w} \mid y_q)$ bounded by:

$$0 \leq R_q \leq (q^* - q)n + O(1), \quad (2.28)$$

where q^* is the full-precision bit width. The residual loss R_q decreases monotonically as additional bit-planes are included ($R_q \geq R_{q+1} - O(\log(q^*n))$) and vanishes to $O(1)$ at full precision, confirming that QuBD avoids the severe information loss of single-bit thresholding.

Hierarchical Complexity and Empirical Dynamics

Empirical evaluations across convolutional networks, Vision Transformers, and modular arithmetic models reveal an intrinsic information hierarchy in deep neural network parameters (Bakhtiarifard et al., 2026b). Algorithmic information concentrates predominantly in the most significant bit-planes (β_q, β_{q-1}) , whereas least significant bit-planes (β_1, β_2) exhibit high algorithmic complexity near maximum entropy ($\Delta\mathcal{K} \approx 100\%$), remaining indistinguishable from random noise throughout optimization.

Tracking $\hat{\mathcal{K}}_{\text{QuBD}}(\mathbf{w})$ during model training provides strong empirical evidence for several phenomena:

1. *Optimization Dynamics:* $\hat{\mathcal{K}}_{\text{QuBD}}(\mathbf{w})$ decreases monotonically as networks train, demonstrating that gradient-based optimization actively compresses parameter representations into algorithmically simpler states.
2. *Overfitting:* When models overfit noisy training data, $\hat{\mathcal{K}}_{\text{QuBD}}(\mathbf{w})$ increases, reflecting the memorization of uncompressible sample-specific patterns.
3. *Grokking:* During delayed generalization, $\hat{\mathcal{K}}_{\text{QuBD}}(\mathbf{w})$ remains plateaued throughout the initial memorization phase and drops rapidly when validation accuracy reaches 100%, continuing to decrease as representations simplify further post-generalization.

Structural Applicability and Limitations

The applicability of QuBD relies on two core structural prerequisites:

1. *Decomposability:* The target object must be separable into discrete elementary components, such as individual neural network weight parameters or kernel matrices.
2. *Hierarchical Representation:* The components must possess an intrinsic significance hierarchy, enabling a natural ordering from highly informative MSBs to noisy LSBs.

QuBD exhibits two primary structural limitations:

1. *Cross-plane Independence Assumption:* QuBD computes the sum of per-plane complexities $\sum_{\ell=1}^q \hat{\mathcal{K}}_{\text{BDM}}(\beta_\ell)$, assuming zero cross-plane mutual algorithmic information.
2. *Sub-block Estimation Bottleneck:* Local complexity within each bit-plane $\hat{\mathcal{K}}_{\text{BDM}}(\beta_\ell)$ continues to rely on CTM precomputed lookup tables over small sub-blocks (Definition 2.8), inheriting CTM’s computational constraint.

In Chapter 3 and Chapter 4, we employ $\hat{\mathcal{K}}_{\text{QuBD}}(\mathbf{w})$ as our primary empirical diagnostic to evaluate how Mosaic-of-Motifs (MoMos) and V-Fold MoMos constrain and simplify parameter representations during deep neural network optimization.

Chapter 3

Neural Networks with Weight-Sharing Constraints

In [Chapter 2](#), we introduced the principles of algorithmic information theory, which highlight that structural regularities enable data compression. Statistical measures such as Shannon entropy ([Definition 2.9](#)) exploit symbol probabilities to reduce representation size, whereas algorithmic complexity accounts for broader deterministic structures.

Modern deep neural networks are typically trained in heavily over-parameterized regimes, resulting in parameter vectors containing millions or billions of high-precision scalar weights. Despite their high empirical dimensionality, trained neural networks exhibit significant structural redundancy. To exploit this redundancy, weight-sharing methods enforce parameter tying. Much like building complex structures from a small set of repeated Lego bricks, these methods constrain network components to reuse a small set of canonical parameter blocks or values.

Weight-sharing techniques analyzed in this chapter operate within a post-training framework. An unconstrained neural network is first trained to convergence on a target task using standard gradient-based optimization. Following convergence, post-training compression methods induce structured regularities across learned parameters, such as discrete value clustering, codebook quantization, or functional generation, thereby reducing the network’s effective description length while preserving performance.

This chapter provides a theoretical and algorithmic overview of three foundational post-training weight-sharing paradigms in the literature:

1. **Soft Weight-Sharing** ([Ullrich et al., 2017](#)): A Bayesian approach grounded in the Minimum Description Length principle that models scalar weights ($s = 1$) using a Gaussian Mixture Model prior with k components. During fine-tuning, weights are simultaneously optimized for task performance and assigned to index map $\mathcal{M}$ pointing to mixture centroids.
2. **Deep Compression** ([Han et al., 2016](#)): A classic three-stage compression pipeline combin-

ing magnitude-based network pruning, k -means trained quantization (scalar weight-sharing with block size $s = 1$) with a learned codebook $\mathcal{Z}$, and Huffman coding (Huffman, 1952) to compress quantization index map $\mathcal{M}$.

3. **Hyper-Compression** (Fan et al., 2026): A recent functional compression paradigm that parameterizes weight blocks of size s via a generator function $\mathcal{Z}$ over search budget k . Instead of storing explicit weight matrices, a deterministic trajectory function computes parameter blocks from index map $\mathcal{M}$, establishing strong conceptual parallels to motif-based weight sharing.

3.1 Soft-Weight Sharing

Soft Weight-Sharing (Ullrich et al., 2017) unifies network pruning and quantization into a single retraining procedure grounded in the Minimum Description Length principle (Rissanen, 1978). Instead of enforcing hard discrete parameter constraints during optimization, the method places a continuous probabilistic prior over the network weights and jointly optimizes empirical task loss alongside a description complexity penalty.

Empirically, Soft Weight-Sharing achieves high compression factors with minimal degradation in predictive performance. On the fully connected LeNet-300-100 architecture, the method delivers a $64\times$ compression ratio while matching the uncompressed test error rate of 1.89% (0.00% accuracy drop). On the convolutional LeNet-5 network, it reaches a $162\times$ compression ratio, increasing test error slightly from 0.80% to 0.96% (a minor 0.16% drop in accuracy).

3.1.1 Probabilistic Formulation and Objective

Let $\mathbf{w} \in \mathbb{R}^n$ denote the flattened weight vector of a pre-trained neural network, partitioned into scalar blocks of size $s = 1$. Soft weight-sharing models the prior distribution of individual scalar weights w_i using a finite Gaussian Mixture Model with k components:

$$p(\mathbf{w}) = \prod_{i=1}^n \sum_{j=0}^{k-1} \pi_j \mathcal{N}(w_i \mid \mu_j, \sigma_j^2), \quad (3.1)$$

where $\pi_j \geq 0$ with $\sum_{j=0}^{k-1} \pi_j = 1$ represent mixture component probabilities, $\mu_j \in \mathbb{R}$ represent component means, and $\sigma_j^2 > 0$ represent component variances.

To achieve simultaneous pruning and quantization, the mixture density contains two distinct functional components:

1. **Pruning via Zero Spike**: Component $j = 0$ is designated as a fixed zero spike centered at $\mu_0 = 0$ with narrow variance σ_0^2 and a large initial mixture mass $\pi_0 = 0.999$. Weights assigned to this component during post-processing are pruned.

2. **Quantization via Non-Zero Centroids:** Components $j \in \{1, \dots, k-1\}$ represent flexible learned clusters with trainable means μ_j , variances σ_j^2 , and mixture weights π_j , serving as an adaptive scalar quantization codebook $\mathcal{Z} = (\mu_1, \dots, \mu_{k-1}) \in \mathbb{R}^{1 \times (k-1)}$.

The pre-trained network is retrained by minimizing a joint objective combining the negative log-likelihood of the target data with the negative log-prior of the weight distribution:

$$\mathcal{L}(\mathbf{w}, \{\mu_j, \sigma_j, \pi_j\}_{j=0}^{k-1}) = \mathcal{L}^E(\mathbf{w}) + \tau \mathcal{L}^C(\mathbf{w}), \quad (3.2)$$

where $\mathcal{L}^E(\mathbf{w}) = -\log p(\mathbf{T} \mid \mathbf{X}, \mathbf{w})$ measures empirical task error on input data $(\mathbf{X}, \mathbf{T})$, $\mathcal{L}^C(\mathbf{w}) = -\sum_{i=1}^n \log p(w_i \mid \{\mu_j, \sigma_j, \pi_j\}_{j=0}^{k-1})$ regularizes description length, and $\tau > 0$ balances task accuracy against model compression.

During gradient descent on Equation 3.2, the complexity term $\mathcal{L}^C$ exerts an attractive force that pulls scalar weights toward the nearest Gaussian centroid μ_j , while empirical loss $\mathcal{L}^E$ maintains task accuracy. Once retraining converges, hard post-training quantization maps each scalar weight w_i to an index map entry $\mathcal{M}_i \in [k]$ via maximum responsibility:

$$\mathcal{M}_i = \arg \max_{j \in [k]} \pi_j \mathcal{N}(w_i \mid \mu_j, \sigma_j^2), \quad (3.3)$$

yielding index map $\mathcal{M} = (\mathcal{M}_1, \dots, \mathcal{M}_n) \in [k]^n$, followed by setting $w_i \leftarrow \mu_{\mathcal{M}_i}$. Weights assigned to $\mathcal{M}_i = 0$ are pruned to zero ($w_i = 0$), yielding a sparse weight vector, while all remaining weights collapse onto a discrete scalar codebook $\mathcal{Z}$ of at most $k - 1$ non-zero values.

Unconstrained optimization of mixture parameters can introduce numerical instabilities, such as catastrophic variance collapse where component variances σ_j^2 shrink unboundedly or mode proliferation where clusters drift. To stabilize optimization, Inverse-Gamma hyperpriors are placed on component variances σ_j^2 , establishing a lower bound on cluster widths, while Beta hyperpriors stabilize mixture probabilities π_j .

Soft Weight-Sharing shares fundamental goals with modern model compression, yet it differs in structural granularity and optimization dynamics when compared to block-level paradigms such as Mosaic-of-Motifs (Chapter 4):

1. *Scalar block size:* Soft Weight-Sharing operates strictly on scalar weights ($w_i \in \mathbb{R}$), corresponding to block size $s = 1$, clustering values onto scalar codebook entries $\mathcal{Z}_j \in \mathbb{R}$.
2. *Continuous Probabilistic Attraction:* Soft Weight-Sharing uses a continuous mixture density during training, allowing weights to migrate smoothly between centroids before constructing the discrete index map $\mathcal{M}$.
3. *Adaptive Codebook Allocation:* By optimizing mixture masses π_j and means μ_j under gradient descent, Soft Weight-Sharing automatically discovers pruning ratios and codebook $\mathcal{Z}$ locations.

3.2 Deep Compression

Deep Compression (Han et al., 2016) is a foundational post-training pipeline that reduces a trained network by composing three sequential stages: magnitude pruning, trained quantization with scalar weight sharing, and Huffman coding (Huffman, 1952). Each stage extends the savings produced by the prior stage while preserving accuracy. On AlexNet the three stages compose to a $35\times$ reduction (240 MB to 6.9 MB) and on VGG-16 to $49\times$ (552 MB to 11.3 MB), in both cases with no loss of accuracy.

3.2.1 Pruning, Trained Quantization, and Entropy Coding

The first stage prunes every connection whose trained magnitude falls below a threshold, then retrain survivors to recover accuracy. The resulting sparse weight vector is stored in compressed sparse row or column format using relative indexing: only the gap between consecutive nonzero positions is encoded, in 8 bits for convolutional layers and 5 bits for fully connected layers. A gap exceeding the width of its field cannot be represented directly, so a filler zero is inserted at the bound and the count restarts, keeping the index stream at fixed width. Pruning precedes quantization, so far fewer weights must be covered by a codebook of the same size, lowering within-cluster error.

The second stage replaces explicit scalar values using a small shared codebook. For each layer independently, surviving weights are partitioned by one-dimensional k -means clustering into k clusters, and every weight in a cluster is replaced by the centroid of that cluster. Storing only codebook $\mathcal{Z}$ of k centroids and index map $\mathcal{M}$ suffices for reconstruction. After quantization, training continues and loss gradients accumulate on codebook centroids, allowing codebook $\mathcal{Z}$ to keep learning while index map $\mathcal{M}$ remains fixed:

$$\frac{\partial \mathcal{L}}{\partial \mathcal{Z}_j} = \sum_{i: \mathcal{M}_i=j} \frac{\partial \mathcal{L}}{\partial w_i}, \quad (3.4)$$

where $\mathcal{M}_i \in [k]$ is the cluster assignment of weight w_i and $\mathcal{Z}_j$ is centroid j .

To quantify the compression ratio achieved by post-training scalar quantization, we construct a discrete codebook and index map representation. Accounting for the storage overhead of both the centroid dictionary and the index map yields the formal description length of the quantized weight vector.

Definition 3.1 (Trained-quantization description length). Let $\mathbf{w} \in \mathbb{R}^n$ be the pruned weight vector partitioned into n blocks of size $s = 1$, and fix cluster count k . The trained-quantization stage represents $\mathbf{w}$ by codebook $\mathcal{Z} = (\mathcal{Z}_1, \dots, \mathcal{Z}_k) \in \mathbb{R}^k$ and index map $\mathcal{M} = (\mathcal{M}_1, \dots, \mathcal{M}_n) \in [k]^n$, with reconstruction $w_i = \mathcal{Z}_{\mathcal{M}_i}$. Storing each centroid at b bits and each index in $\mathcal{M}$ with $\lceil \log_2 k \rceil$

bits yields description length

$$\pi_{\text{dc}} = kb + n \lceil \log_2 k \rceil, \quad (3.5)$$

and stage compression ratio

$$r_{\text{dc}} = \frac{nq}{kb + n \lceil \log_2 k \rceil}, \quad (3.6)$$

where q is the uncompressed bit precision.

The product nq represents the total bits required to store uncompressed weights at their original precision.

The third stage applies Huffman coding (Huffman, 1952) as a lossless, offline pass over the two non-uniform streams produced by earlier stages: index map $\mathcal{M}$ and sparse index deltas. Because symbol distributions are non-uniform, a prefix-free code assigns shorter codewords to frequent symbols and recovers an additional 20% to 30% of storage at zero accuracy cost. Entropy coding replaces the fixed $\lceil \log_2 k \rceil$ bits per index in Equation 3.5 by the empirical entropy of index map $\mathcal{M}$, which is smaller when the cluster distribution is non-uniform. The same description-length accounting bounds the MoMos representation in Chapter 4.

Deep Compression pursues the same post-training compression aim as the block-level Mosaic-of-Motifs parameterization (Chapter 4), but the two diverge in three key aspects:

1. *Scalar block size*: The method operates strictly on scalar weights $w_i \in \mathbb{R}$ (block size $s = 1$), clustering values onto scalar centroids. In contrast, MoMos (Definition 4.1) partitions $\mathbf{w}$ into blocks of size $s > 1$ and rebuilds it from reusable motif columns $\mathbf{z}_j$ gathered by index map $\mathcal{M}$ as $\mathcal{Z}[\mathcal{M}]$.
2. *Gradient-trainable centroids*: Codebook $\mathcal{Z}$ centroids are refined by backpropagation Equation 3.4 before the final Huffman pass, so the pipeline is genuinely trainable in its intermediate form.
3. *Exact lossless code over quantized values*: Description length π_{dc} from Definition 3.1 yields an exact, lossless code over quantized values, so bound $\mathcal{K}(\hat{\mathbf{w}}) \leq \pi_{\text{dc}} + \mathcal{O}(1)$ applies to reconstruction $\hat{w}_i = \mathcal{Z}_{\mathcal{M}_i}$ rather than to trained $\mathbf{w}$. The pipeline imposes no restriction on which trained network may be compressed: every full-precision vector remains prunable and quantizable, and choice of cluster count k is made after training.

3.3 Hyper-Compression

Hyper-Compression (Fan et al., 2026) reframes model compression as a parameter-representation problem. Rather than pruning, quantizing, or distilling a network, it stores a small set of scalar generator parameters and regenerates weight blocks of size s through a fixed deterministic map. A

high-dimensional weight vector is thus replaced by a compact two-part code: a shared generator function $\mathcal{Z}$ plus an index map $\mathcal{M}$.

3.3.1 Ergodic Regeneration

The generator is grounded in ergodic theory. If transformation T is ergodic and measure-preserving, and the support of the invariant measure has full measure, then for any target point and any $\epsilon > 0$ there exists a starting point in any set of positive measure whose orbit under T comes within ϵ of that target. This allows representing a weight block of size s with a single integer: the number of steps taken along the trajectory before passing close to the target. Hyper-Compression realizes T by irrational rotation:

$$\tau(\theta \cdot \mathbf{a}) = \theta \cdot \mathbf{a} - \lfloor \theta \cdot \mathbf{a} \rfloor, \quad (3.7)$$

where $\mathbf{a} \in \mathbb{R}^s$ has irrationally independent components. Because components are irrationally independent, trajectory $\{\tau(\theta \cdot \mathbf{a}) : \theta = 1, 2, \dots\}$ becomes dense in the unit hypercube, so any target weight block of size s , once rescaled into that cube, is within ϵ of some trajectory point.

Instead of storing discrete vector entries in a lookup table, functional compression models parameters as points along a continuous deterministic path. Searching for the nearest trajectory point for each parameter block yields an index map that regenerates network weights without an explicit codebook.

Definition 3.2 (Hyperfunction regeneration). Let $\mathbf{w}^{(g)} \in \mathbb{R}^s$ be a block of s weights, recentered on its local centroid and scaled. Let search budget $k \in \mathbb{N}$ bound candidate trajectory steps. Hyper-Compression defines a generator function $\mathcal{Z} : [k] \rightarrow \mathbb{R}^s$ given by $\mathcal{Z}(\theta) = \tau(\theta \cdot \mathbf{a})$ with $\mathbf{a} \in \mathbb{R}^s$, and selects index map entry

$$\mathcal{M}_g = \arg \min_{\theta \in [k]} \|\mathbf{w}^{(g)} - \mathcal{Z}(\theta)\|_2, \quad (3.8)$$

storing only index map $\mathcal{M} = (\mathcal{M}_1, \dots, \mathcal{M}_m) \in [k]^m$.

As [Definition 3.2](#) formalizes, decompression recomputes generator map $\mathcal{Z}(\mathcal{M}_g) = \tau(\mathcal{M}_g \cdot \mathbf{a})$ from stored integer index $\mathcal{M}_g \leq k$. Because search budget is bounded by k , each block of s full-precision weights is replaced by an index entry requiring $\lceil \log_2 k \rceil$ bits in index map $\mathcal{M}$. Search budget k can assume different values across layers. While $\mathcal{Z}$ represents a discrete stored lookup table in Soft Weight-Sharing and Deep Compression, in Hyper-Compression $\mathcal{Z}$ is not a stored codebook matrix, but a continuous trajectory generator function $\mathcal{Z} : [k] \rightarrow \mathbb{R}^s$.

In practice, flattened weight vector $\mathbf{w}$ is split into $m = \lceil n/s \rceil$ blocks of size s , set to $s = 2$ in published experiments. Because raw weights rarely fill the unit hypercube uniformly, each block is recentered on its local centroid and scaled into a small box before applying τ ; outliers are grouped by distance from the centroid, receiving distinct scale factors, and direction $\mathbf{a}$ is optimized rather

than fixed to a generic irrational vector. Finding index map entry $\mathcal{M}_g$ is a nearest-neighbor search over precomputed trajectory points $\mathcal{Z}(\theta)$, accelerated by k - d trees and matrix operations.

The method is retraining-free and composition-free. No generator network is trained and no calibration data is required: reconstruction error is typically on the order of 10^{-4} to 10^{-3} , small enough that no fine-tuning is needed. Unlike a hypernetwork, regeneration uses no learned composition, only a single fixed ergodic map. At inference time, weights of each layer are regenerated just before use, overlapping with forward pass execution of the previous layer so decompression cost is largely hidden. Empirically, it achieves a $2.60\times$ reduction on LLaMA2-7B at average accuracy loss below 1% across zero-shot benchmarks, and stacks on top of pruning or quantization.

Hyper-Compression leverages structural redundancy similarly to Mosaic-of-Motifs (Chapter 4), but exhibits four key differences:

1. *Block regeneration*: The method operates on fixed-size windows of block size s regenerated from index map $\mathcal{M}$, rather than on scalar values. Furthermore, each block is an independently regenerated trajectory point without cross-position sharing.
2. *Function versus codebook*: Parameter regularity resides in the generating function. Weights are outputs of a deterministic hyperfunction, so structure resides in generator function $\mathcal{Z}$ (the trajectory map τ of Equation 3.7) rather than in a stored codebook matrix or shared value table.
3. *One-shot search without retraining*: The compressed model is an index map $\mathcal{M} \in [k]^m$ together with shared direction $\mathbf{a} \in \mathbb{R}^s$ and fixed generator $\mathcal{Z}$, requiring no fine-tuning or composition step. The representation is static and must be decompressed before training can resume.
4. *Lossy approximation trading against search budget*: Storing each block of size s using $\lceil \log_2 k \rceil$ bits in index map $\mathcal{M}$ lowers $\mathcal{K}(\mathbf{w})$ via description length in Definition 3.2. Approximation error grows with block size s and shrinks as search budget k increases, trading off compression ratio against fidelity.

Chapter 4

Mosaic-of-Motifs Characterization

In [Chapter 3](#), we reviewed post-training weight-sharing paradigms, including Soft Weight-Sharing ([Ullrich et al., 2017](#)), Deep Compression ([Han et al., 2016](#)), and Hyper-Compression ([Fan et al., 2026](#)). These methods enforce structural redundancies after model convergence, modifying learned weight configurations within flat loss regions where parameter gradients approach zero. However, post-training regularizers cannot alter the optimization trajectory itself and offer no explicit theoretical bounds on the algorithmic complexity of intermediate parameter representations during training.

To our knowledge, only the Mosaic-of-Motifs (MoMos) parameterization ([Bakhtiarifard et al., 2026a](#)) explicitly bounds algorithmic complexity for neural network objects. [Bakhtiarifard et al. \(2026a\)](#) formalizes parameter redundancy directly through an explicit motif codebook and a mosaic index map, analogous to QAT [Definition 2.11](#). Applied from the first training epoch, MoMos projects the parameter vector onto a constrained hypothesis class after each optimization step, guaranteeing a strictly bounded algorithmic description length throughout training while preserving gradient search dynamics.

This chapter is organized as follows:

1. [Section 4.1](#) formalizes the one-dimensional MoMos parameterization, derives its description-length upper bound and analyzes hypothesis space cardinality.
2. [Section 4.2](#) extends the formulation to two-dimensional weight tensors, preserving structural geometry across neural layers.
3. [Section 4.3](#) presents empirical evaluations on CIFAR-10, critiques the standard capacity metric, introduces the compression ratio and analyzes motif clustering dynamics over long training runs.
4. [Section 4.4](#) synthesizes key insights and discusses open questions addressed in [Chapter 5](#).

4.1 Mosaic-of-Motifs Characterization

To bound algorithmic complexity throughout neural network optimization, [Bakhtiarifard et al. \(2026b\)](#) model parameter redundancy via discrete structural block sharing. Consider an unconstrained neural network parameterized by a flattened, concatenated weight vector $\mathbf{w} \in \mathbb{R}^n$, where each scalar is stored using q bits. Under unconstrained storage, the trivial upper bound $\mathcal{K}(\mathbf{w}) \leq nq + \mathcal{O}(1)$ applies. MoMos replaces this unstructured vector with a lower-complexity decomposition comprising a shared codebook of parameter blocks (motifs) and an index map (mosaic). Each unique motif in the codebook preserves the trivial bound, while the mosaic reduces the structural complexity of the network. The formal definition of the parameterization is stated below.

Definition 4.1 (MoMos Parameterization). Let $s \in \mathbb{N}$ be a block size and $m = \lceil n/s \rceil$ the number of blocks. Fix an injective slicing operator $\psi : \mathbb{R}^n \rightarrow \mathbb{R}^{s \times m}$ that partitions $\mathbf{w}$ into m blocks, zero-padding trailing coordinates. The MoMos parameterization consists of a motif matrix $\mathcal{Z} = (\mathbf{z}_1, \dots, \mathbf{z}_k) \in \mathbb{R}^{s \times k}$ and a mosaic index map $\mathcal{M} \in [k]^m$. The reconstructed weight vector is:

$$\mathbf{w} = \psi^{-1}(\mathcal{Z}[\mathcal{M}]) = \psi^{-1}(\mathbf{z}_{\mathcal{M}_1}, \dots, \mathbf{z}_{\mathcal{M}_m}) \in \mathbb{R}^n. \quad (4.1)$$

Restricting parameter optimization to configurations expressible via a motif matrix $\mathcal{Z}$ and mosaic map $\mathcal{M}$ defines the constrained search space of the model.

Definition 4.2 (MoMos Hypothesis Class). For $s, k \in \mathbb{N}$ with $s, k \geq 1$, the (s, k) -MoMos hypothesis class is defined as:

$$\mathcal{H}_{s,k} = \{\psi^{-1}(\mathcal{Z}[\mathcal{M}]) : \mathcal{Z} \in \mathbb{R}^{s \times k}, \mathcal{M} \in [k]^m\} \subseteq \mathbb{R}^n. \quad (4.2)$$

When the number of motifs equals the number of blocks ($k = m$), every block is assigned a unique motif, yielding $\mathcal{H}_{s,m} = \mathbb{R}^n$ without parameter compression ($msq = nq$, while the mosaic indexing overhead exceeds the trivial representation). Parameter compression is achieved when $k \ll m$, forcing multiple parameter blocks to share identical motif vectors.

4.1.1 Complexity and Hypothesis Space Cardinality

Reconstructing the full weight vector $\mathbf{w}$ requires storing the motif matrix $\mathcal{Z}$, the mosaic index map $\mathcal{M}$, and the fixed slicing operator ψ . Storing each scalar at q -bit precision yields the description-length upper bound:

$$\mathcal{K}(\mathbf{w}) \leq ksq + m \lceil \log_2 k \rceil + \mathcal{O}(1). \quad (4.3)$$

In [Equation 4.3](#), ksq bits represent the motif matrix $\mathcal{Z}$, holding k motifs of s scalars at q bits. The term $m \lceil \log_2 k \rceil$ reflects the mosaic map $\mathcal{M}$, storing one $\lceil \log_2 k \rceil$ -bit address for each of the

m blocks, while $\mathcal{O}(1)$ accounts for the static implementation cost of ψ .

Theorem 5 (Cardinality of the Hypothesis Class). Quantizing scalar weights to q bits yields $N = 2^{sq}$ distinct candidate block patterns. The exact cardinality of the hypothesis class $\mathcal{H}_{s,k}$ is:

$$|\mathcal{H}_{s,k}| = \sum_{u=1}^k \binom{N}{u} u! \left\{ \begin{matrix} m \\ u \end{matrix} \right\} \quad (4.4)$$

$$= \sum_{u=1}^k (N)_u \left\{ \begin{matrix} m \\ u \end{matrix} \right\} \quad (4.5)$$

where $\left\{ \begin{matrix} m \\ u \end{matrix} \right\}$ denotes the Stirling number of the second kind, counting the partitions of m blocks into u non-empty subsets, and $u!$ assigns chosen motifs to subsets; or equivalently $(N)_u = N(N-1) \cdot (N-u+1)$.

Intuitively, $\binom{N}{u}$ is the number of ways to select u distinct motifs from all possible quantized block patterns, regardless of ordering. Swapping two codebook motifs and their corresponding indices in the mosaic map yields an equivalent representation of the reconstructed weight vector. The product $u! \left\{ \begin{matrix} m \\ u \end{matrix} \right\}$ counts the number of ways each motif can be assigned to non-empty partitions of the m blocks. Summing from $u = 1$ to k accumulates configurations across all active codebook sizes $u \leq k$, accounting for motifs that remain unused by the mosaic map.

In the compression regime $k \ll m$, the summation in [Theorem 5](#) is dominated by the $u = k$ term, yielding the approximation $|\mathcal{H}_{s,k}| \approx \binom{N}{k} k^m$. This cardinality counts distinct weight vectors rather than distinct network functions. In deep ReLU networks, multiple weight vectors parameterize functionally identical predictors due to network symmetries, such as positive-rescaling and node permutation symmetries. Consequently, $|\mathcal{H}_{s,k}|$ serves as an upper bound on the number of realizable functions.

The expressive capacity of $\mathcal{H}_{s,k}$ depends heavily on the block size s . From the asymptotic approximation $k^m = k^{n/s}$, decreasing s increases the exponent, exponentially expanding the hypothesis space. Small block sizes (such as $s = 2$ or $s = 4$) retain high expressive capacity even when $k \ll m$, explaining their superior empirical performance.

MoMos disrupts traditional weight-space symmetries in deep ReLU networks. Scaling a row of weight matrix $\mathbf{W}_l$ by $\kappa > 0$ while scaling column $\mathbf{W}_{l+1}$ by κ^{-1} preserves the input-output mapping. MoMos breaks this continuous scale invariance by requiring rows and columns across layers to be constructed from shared discrete motifs in $\mathcal{Z}$. This forces weights toward standard canonical magnitudes, reducing parameter redundancy and improving generalization ([Lim et al., 2024](#)).

4.1.2 Algorithmic Implementation

MoMos is integrated into neural network training as a projection step executed after every optimization epoch. Let $\mathcal{D}$ denote the dataset, $\mathcal{L}$ the loss function, and η the learning rate. Optimization alternates between standard gradient updates on unconstrained parameters and structural projections onto $\mathcal{H}_{s,k}$.

During each epoch, weights are updated via backpropagation using standard optimizers (such as SGD or AdamW). Following the update, $\mathbf{w}^{(t)}$ is partitioned into m blocks of size s via ψ . A codebook of $k - 1$ motifs is randomly sampled from the m parameter blocks, and a zero motif is appended to form $\mathcal{Z} \in \mathbb{R}^{s \times k}$. Each block is then assigned to its nearest motif in $\mathcal{Z}$ under the L_2 norm, updating mosaic map $\mathcal{M}$. Finally, the weight vector is reconstructed as $\mathbf{w}^{(t)} = \psi^{-1}(\mathcal{Z}[\mathcal{M}])$.

Unlike vector quantization frameworks where codebook entries are optimized via gradient descent or k -means clustering, MoMos draws motifs randomly from current parameter blocks each epoch. Optimization thus learns the index map $\mathcal{M}$, while codebook entries $\mathcal{Z}$ are refreshed dynamically across epochs. The complete training procedure is detailed in [Algorithm 2](#).

Algorithm 2 MoMos Training Loop

Require: Dataset $\mathcal{D}$, block size s , codebook size k , epochs T

Ensure: Optimized MoMos parameters $(\mathcal{Z}, \mathcal{M})$

```

1: Initialize weight vector  $\mathbf{w}^{(0)} \in \mathbb{R}^n$ 
2: for  $t = 1$  to  $T$  do
3:                                      $\triangleright$  Step 1: Unconstrained gradient update
4:   Update  $\mathbf{w}^{(t)}$  by training on  $\mathcal{D}$  via back-propagation
5:                                      $\triangleright$  Step 2: MoMos structural projection
6:    $\mathbf{b}_1, \dots, \mathbf{b}_m \leftarrow \psi(\mathbf{w}^{(t)})$ 
7:    $\mathbf{z}_1 \leftarrow \mathbf{0}_s$ 
8:    $(\mathbf{z}_i)_{i=2}^k \sim \text{Unif}(\{\mathbf{b}_1, \dots, \mathbf{b}_m\})$ 
9:    $\mathcal{Z} \leftarrow [\mathbf{z}_1, \dots, \mathbf{z}_k]$ 
10:  for  $i = 1$  to  $m$  do
11:     $\mathcal{M}_i \leftarrow \arg \min_{j \in [k]} \|\mathbf{b}_i - \mathbf{z}_j\|_2$ 
12:  end for
13:   $\mathbf{w}^{(t)} \leftarrow \psi^{-1}(\mathcal{Z}[\mathcal{M}])$                                       $\triangleright$  Reconstruct constrained parameters
14: end for
15: return  $(\mathcal{Z}, \mathcal{M})$ 

```

4.2 2D Mosaic-of-Motifs

The one-dimensional formulation in [Definition 4.1](#) flattens parameter tensors into a linear sequence, disregarding the underlying spatial geometry of neural network layers. In multilayer perceptrons, rows of a weight matrix correspond to individual output neurons, while columns correspond to input features. In convolutional networks, parameters are organized spatially across kernel di-

mensions. One-dimensional slicing groups spatially unrelated parameters, forcing motifs to span arbitrary layer boundaries.

To preserve spatial structure across parameter tensors, we extend the slicing operator ψ to multi-dimensional block tiling.

Definition 4.3 (MoMos 2D Parameterization). Let $r, c \in \mathbb{N}$ and set block size $s = rc$. For each parameter tensor $\mathbf{w}_i \in \mathbb{R}^{d_{\text{out}}^{(i)} \times d_{\text{in}}^{(i)}}$, pad $\mathbf{w}_i$ to shape $r \lceil d_{\text{out}}^{(i)} / r \rceil \times c \lceil d_{\text{in}}^{(i)} / c \rceil$ with trailing zeros. Partition the padded tensor into non-overlapping $r \times c$ blocks and flatten each block to length s . Concatenating these vectors across all l layers defines slicing operator $\psi_{r,c} : \mathbb{R}^n \rightarrow \mathbb{R}^{s \times m}$, where total block count m is:

$$m := \sum_{i=1}^l \left\lceil d_{\text{out}}^{(i)} / r \right\rceil \cdot \left\lceil d_{\text{in}}^{(i)} / c \right\rceil. \quad (4.6)$$

The MoMos 2D parameterization is given by $(\mathcal{Z}, \mathcal{M})$ with $\mathcal{Z} \in \mathbb{R}^{s \times k}$ and $\mathcal{M} \in [k]^m$, reconstructed as $\mathbf{w} = \psi_{r,c}^{-1}(\mathcal{Z}[\mathcal{M}])$.

In [Definition 4.3](#), r, c denote rows (output neurons) and columns (input features) in each tile, respectively. One-dimensional MoMos ([Definition 4.1](#)) is a special case with $r = 1$ and $c = s$.

4.2.1 Complexity of 2D MoMos

Because the slicing operator $\psi_{r,c}$ has a fixed description, the complexity bound follows the accounting of [Equation 4.3](#):

$$\mathcal{K}(\mathbf{w}) \leq krcq + m \lceil \log_2 k \rceil + \mathcal{O}(1). \quad (4.7)$$

The motif matrix requires $krcq$ bits for k blocks of size $s = rc$. Reconstructing layer shapes does not require storing full architecture shape metadata. For each layer, row padding is an integer strictly smaller than r ($\lceil \log_2 r \rceil$ bits) and column padding is an integer strictly smaller than c ($\lceil \log_2 c \rceil$ bits). Because tile dimensions r and c are small constants, padding offsets consume a constant cost $\mathcal{O}(1)$. The mosaic $\mathcal{M}$ encodes parameters following the original layer layout, enabling a reshape operation to restore original parameter dimensions after motif lookup.

In empirical settings, block sizes remain small (e.g., $s \in \{2, 4\}$), and layer dimensions in standard architectures are even or divisible by tile dimensions r and c . As a result, 2D tiling introduces negligible zero-padding overhead, keeping total block count m virtually identical to the 1D baseline $\lceil n/s \rceil$. Target network architectures have known, fixed shapes, eliminating shape metadata overhead in practical implementations. Because 1D MoMos ([Definition 4.1](#)) corresponds to the special case with shape $(1, s)$, we refer to MoMos 2D simply as MoMos throughout the remainder of this work.

4.3 Empirical Results

We empirically evaluate MoMos and MoMos 2D on CIFAR-10 classification using a 5-layer multi-layer perceptron (MLP) architecture with $n \approx 10^6$ parameters. Models are trained for 100 epochs across 5-fold cross-validation with fixed dataset splits and distinct initialization seeds per fold. We report mean test accuracy and standard deviation across runs.

4.3.1 Capacity Metric and Theoretical Critique

Bakhtiarifard et al. (2026a) evaluates MoMos parameterizations using model capacity $\text{cap} := k/m$, representing the fraction of unique motifs. Assuming $s \mid n$, capacity $\text{cap} = 1$ corresponds to $k = m$ (uncompressed baseline), while $\text{cap} \ll 1$ enforces high motif sharing.

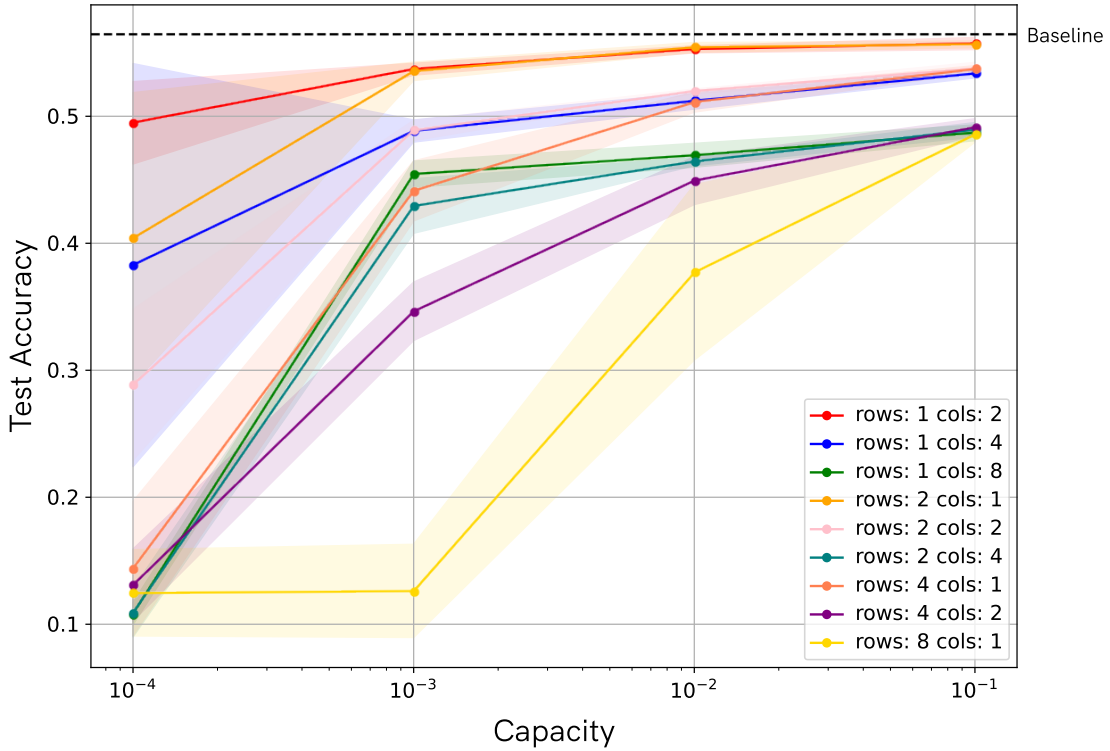


Figure 4.1: CIFAR-10 test accuracy of MoMos and MoMos 2D configurations plotted against capacity $\text{cap} = k/m$ across blocks of shape $r, c \in \{1, 2, 4, 8\}$.

As shown in Figure 4.1, smaller block sizes ($s = 2, 4$) achieve superior test accuracy across moderate capacity levels (10^{-1} to 10^{-3}). Below $\text{cap} = 10^{-3}$, test accuracy drops sharply across all configurations.

The choice of the term "capacity" for $\text{cap} = k/m$ is conceptually misleading. In statistical learning theory, capacity already designates fundamental notions such as Vapnik–Chervonenkis (VC) dimension or Cover’s separating capacity (analyzed in Chapter 6), which quantify the combinatorial number of patterns a model family can shatter. In contrast, $\text{cap} = k/m$ is merely a

structural allocation fraction: the ratio of unique motif columns to total block positions. Because these two usages refer to distinct ideas, overlapping the terminology creates ambiguity.

Beyond terminology, cap is a theoretically flawed metric for comparing compression efficiency across parameter scales n . Expressing Equation 4.3 in terms of $\text{cap} = k/m$ yields:

$$\mathcal{K}(\mathbf{w}) \leq ks q + m \lceil \log_2 k \rceil = n \cdot \text{cap} \cdot q + \frac{n}{s} \left\lceil \log_2 \left(\frac{n}{s} \cdot \text{cap} \right) \right\rceil. \quad (4.8)$$

As parameter count $n \rightarrow \infty$ at a fixed capacity fraction cap , the mosaic indexing term $\frac{n}{s} \log_2 n$ grows as $\mathcal{O}(n \log n)$, whereas the uncompressed description length nq grows as $\mathcal{O}(n)$. Consequently, for any fixed fraction cap , there exists a threshold scale n where the MoMos bound exceeds unconstrained storage ($nq \leq \mathcal{K}(\mathbf{w})$). This ill-conditioning demonstrates that cap fails to isolate true description complexity.

4.3.2 Compression Ratio

To resolve the conditioning failure of capacity, we evaluate compression using the compression ratio $r_\phi(\mathbf{w})$, defined as:

$$r_\phi(\mathbf{w}) := \frac{nq}{\hat{\mathcal{K}}_\phi(\mathbf{w})}, \quad (4.9)$$

where $\hat{\mathcal{K}}_\phi(\mathbf{w})$ evaluates description length under parameterization ϕ .

Evaluating performance against $r_{\text{MoMos}}(\mathbf{w})$ (Figure 4.2) reveals key regimes hidden by capacity metrics:

1. *Mosaic compression*: At fixed capacity cap , larger block sizes s reduce total block count $m = n/s$, shrinking mosaic address map $\mathcal{M}$ and increasing overall compression ratio $r_{\text{MoMos}}(\mathbf{w})$.
2. *Optimal Trade-Off Regimes*: Parameterizations (1,8) and (2,4) at capacity $\text{cap} = 10^{-3}$ achieve higher compression ratios and equal or superior accuracy compared to block size $s = 4$ at capacity $\text{cap} = 10^{-4}$. Similarly, block size $s = 4$ at $\text{cap} = 10^{-3}$ matches $s = 2$ while delivering substantially higher compression ratios.

This behavior connects directly to the VC dimension and separating capacity analysis in Chapter 6. As proved in Theorem 8, increasing block size s does not degrade the worst-case VC dimension, which remains preserved at $\text{VCdim} = n$ across the union over mosaics whenever $k \geq \min(m, 2^s)$. However, the block size directly determines the continuous capacity of each individual mosaic subspace $\mathcal{H}_{\mathcal{M}}$, which possesses ks degrees of freedom and separating capacity $2ks$ (Equation 6.17). When the motif count k is severely reduced (such as $\text{cap} \leq 10^{-4}$), a new regime arises where the deficit in continuous parameter capacity (ks) degrades predictive performance more severely than the combinatorial constraint on shattering. In this severe motif-reduction regime, larger block sizes maintain a larger continuous parameter volume ks , preserving

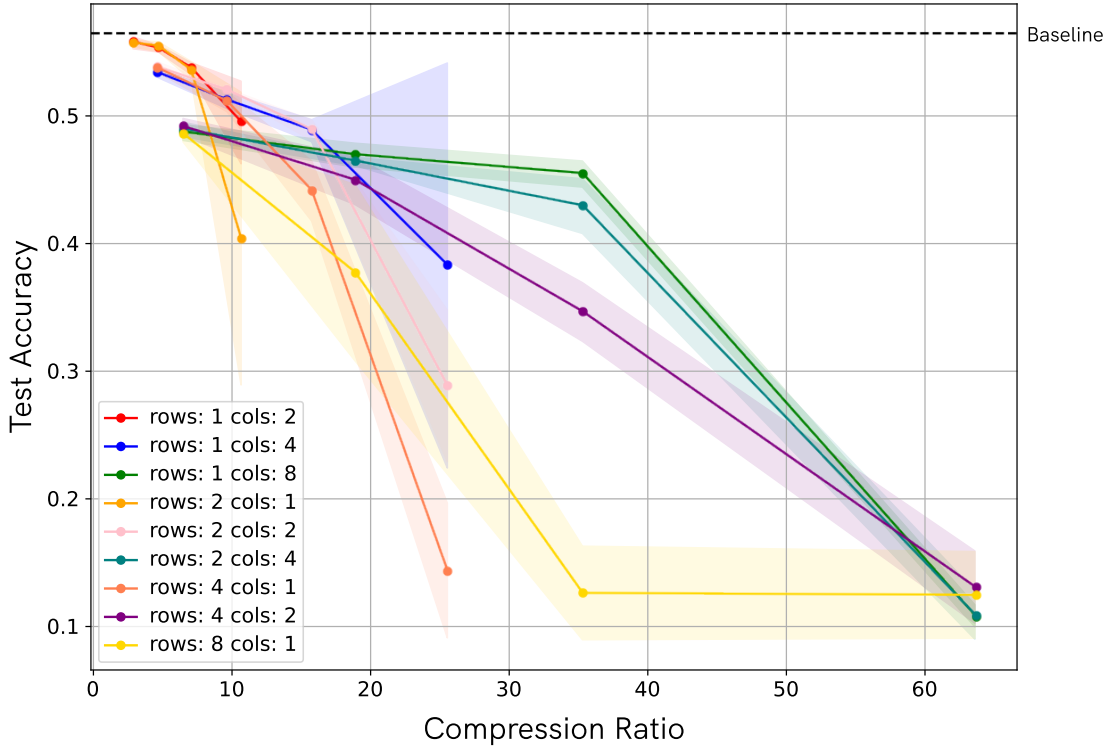


Figure 4.2: CIFAR-10 test accuracy plotted against compression ratio $r_\phi(\mathbf{w})$. At fixed capacity cap, larger block sizes achieve significantly higher compression ratios due to reduced mosaic overhead.

higher accuracy and higher compression ratios simultaneously because the ability to shatter target patterns is not yet lost while continuous capacity ks is substantially higher.

Excessively restricting motif count k triggers catastrophic capacity collapse. This threshold represents a critical trade-off limit between codebook size and mosaic address complexity, characterized theoretically for linear models in [Chapter 6](#).

[Figure 4.3](#) demonstrates that the description length of weight vector $\mathbf{w}$ under the MoMos parameterization is dominated by the mosaic map across nearly all configurations.

4.3.3 Training Dynamics and Algorithmic Complexity

[Figure 4.4](#) illustrates training dynamics over 100 epochs at capacity cap = 10^{-3} . In [Figure 4.4\(a\)](#), configurations (1, 8) and (2, 4) exhibit similar accuracy trajectories, with (2, 4) degrading slightly earlier. Without early stopping, all configurations exhibit mild performance decay near epoch 100 as parameter updates continue in constrained space.

Algorithmic complexity was tracked using $\hat{\mathcal{K}}_{\text{BDM}}(\mathbf{w})$ ([Definition 2.10](#)) prior to the development of $\hat{\mathcal{K}}_{\text{QuBD}}$ ([Bakhtiarifard et al., 2026b](#)). In [Figure 4.4\(b\)](#), block size $s = 4$ ((2, 2)) registers similar $\hat{\mathcal{K}}_{\text{BDM}}$ complexity to block size $s = 8$ ((1, 8) and (2, 4)). This behavior arises because $\hat{\mathcal{K}}_{\text{BDM}}$ is restricted to small local patterns, making algorithmic complexity estimation increasingly imprecise as block size grows.

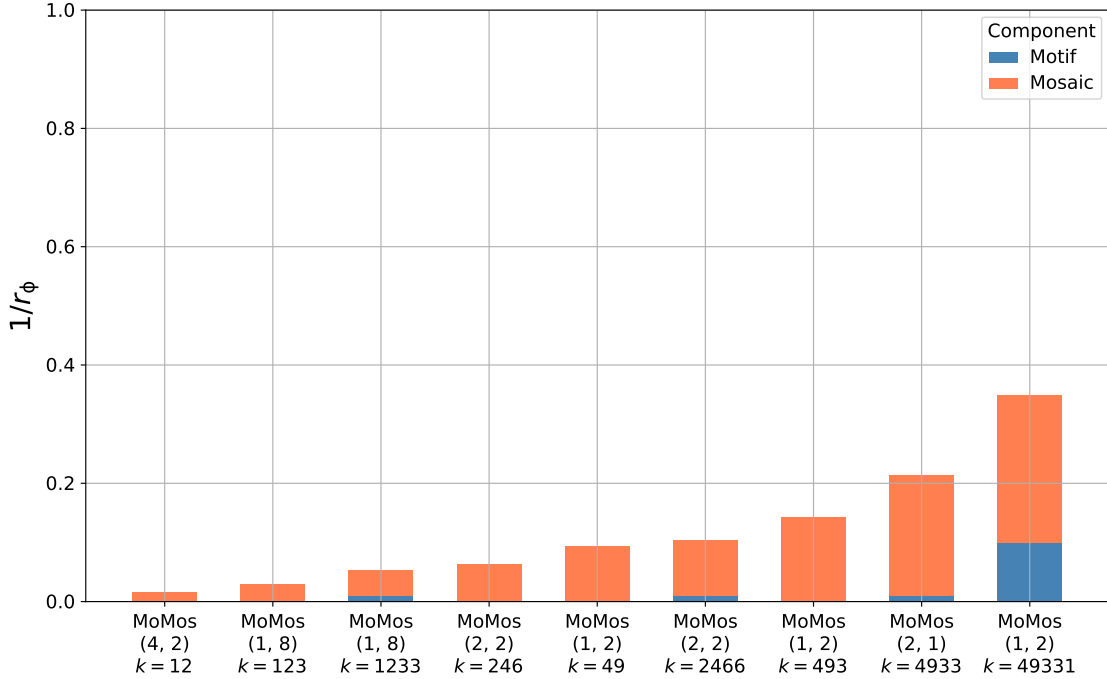


Figure 4.3: Description-length breakdown of motif matrix $\mathcal{Z}$ and mosaic map $\mathcal{M}$, normalized as $1/r_{\text{MoMos}}$ on the y-axis, showing mosaic versus motif storage across Pareto-optimal configurations of the experiment.

4.3.4 Long-Run Convergence and Soft Regularization Dynamics

To investigate long-term convergence under continuous MoMos projections, we trained MLPs for 400 epochs using shapes (1, 2) and (2, 1) at capacity $\text{cap} = 10^{-3}$ ($k = 493$ motifs).

As shown in Figure 4.5, motif usage becomes heavily skewed over extended training. For shape (1, 2), a single dominant motif reaches frequency 8,000 at epoch 380. For shape (2, 1), peak frequency reaches 7,000 at epoch 280 before redistributing toward 5,000 near epoch 380. This extreme frequency imbalance indicates that entropy coding (such as Huffman coding (Huffman, 1952)) can yield substantial additional post-training lossless compression without accuracy loss.

Figure 4.6 visualizes two-dimensional motif coordinates in $\mathbb{R}^2$ throughout training. At epoch 1, motifs align according to block geometry: shape (1, 2) forms a linear trajectory, whereas (2, 1) exhibits non-linear clusters. By epoch 200, (1, 2) forms dense linear segments with discrete gaps, while (2, 1) concentrates into distinct localized clusters. By epoch 380, motifs within each cluster merge onto overlapping coordinate locations.

The linear trajectory observed for shape (1, 2) arises from spatial feature correlations in CIFAR-10, where adjacent input pixels carry similar visual information. Because the first layer requires high capacity to extract local spatial features, subsequent layers are driven to represent correlated features across consecutive hidden units. Conversely, shape (2, 1) groups weights across different output neurons for the same input feature, thereby bypassing horizontal spatial correlations.

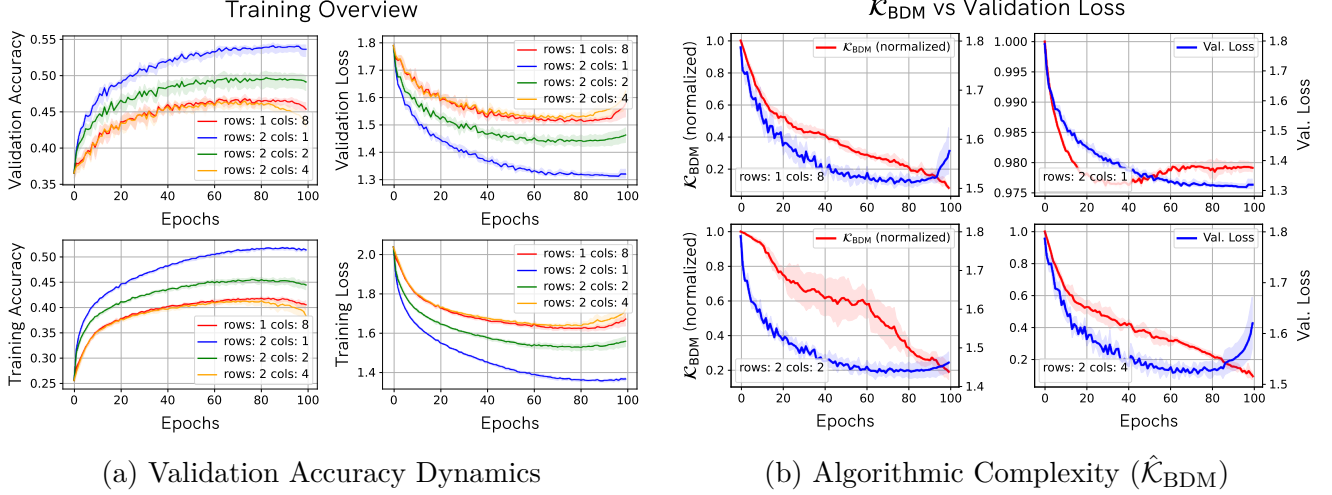


Figure 4.4: Training dynamics over 100 epochs for representative MoMos 2D configurations at capacity $\text{cap} = 10^{-3}$. (a) Validation accuracy shows mild late-stage degradation. (b) $\hat{\mathcal{K}}_{\text{BDM}}$ complexity tracking demonstrates similar complexity for block size $s = 4$ compared to $s = 8$.

Predictive performance and task accuracy are fully preserved throughout this clustering process. This phenomenon reveals two distinct regularization mechanisms operating in MoMos:

1. *Hard Regularization*: The static structural constraint defined by the (s, k) -MoMos hypothesis class $\mathcal{H}_{s,k}$ (Definition 4.2), which sets an explicit upper bound on codebook size k and block size s .
2. *Soft Regularization*: An implicit dynamic regularization induced by epoch-wise projections and random motif resampling.

While the hypothesis class $\mathcal{H}_{s,k}$ remains broad and flexible per se, the implicit soft regularization virtually shrinks the effective hypothesis class far more aggressively than the hard structural constraint alone. To survive epoch-wise resampling across training, motifs must cluster tightly around frequently assigned parameter configurations. Over extended training, the structural regularization of MoMos dominates the gradient updates, which shift the resulting motif clusters only marginally. Because motifs become heavily duplicated in coordinate space, a large fraction of redundant motifs can be merged post-training without degrading predictive accuracy.

4.4 Discussion and Future Directions

MoMos establishes a tractable framework for bounding Kolmogorov complexity directly during neural network training. By projecting parameters onto structured motif codebooks and mosaic index maps, MoMos maintains low description length without requiring post-training quantization pipelines.

Our analysis highlights three fundamental insights:

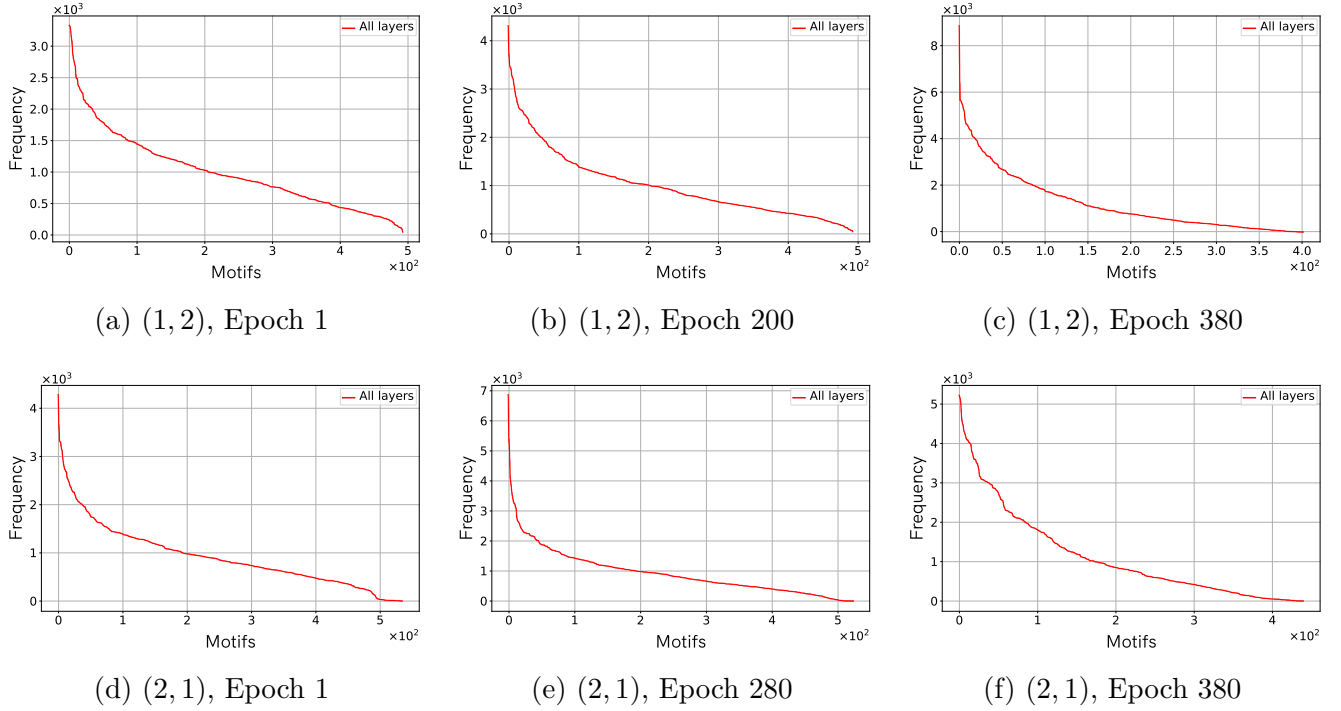
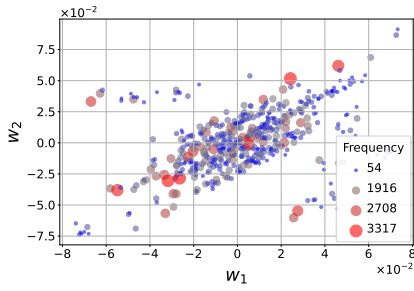


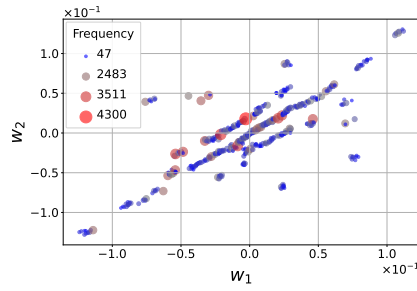
Figure 4.5: Motif assignment frequencies over 400 epochs for MoMos 2D shapes (1, 2) (top row) and (2, 1) (bottom row). Highly skewed distributions emerge, with dominant motifs assigned over 8,000 times.

1. *Capacity Conditioning*: Standard capacity $\text{cap} = k/m$ is ill-conditioned as $n \rightarrow \infty$, necessitating absolute description metrics such as compression ratio $r_{\text{MoMos}}(\mathbf{w})$.
2. *Symmetry Disruption*: Epoch-wise motif sharing breaks continuous ReLU scaling symmetries, driving parameters toward standardized magnitudes.
3. *Soft Regularization and Motif Clustering*: Epoch-wise projections induce a soft regularization that dominates late-stage gradient updates. This dynamic forces motifs to cluster and duplicate, virtually shrinking the hypothesis class far beyond the hard bound $\mathcal{H}_{s,k}$ while fully preserving model accuracy, enabling extensive post-training motif shrinkage.

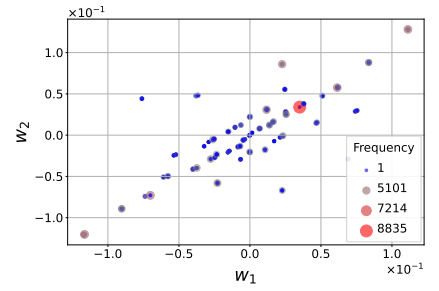
As shown in Figure 4.3, the mosaic map remains the primary storage bottleneck. While increasing block size improves the compression ratio, it can degrade predictive accuracy. In Chapter 5, we address this limitation by introducing V-Fold MoMos, a two-level hierarchical codebook parameterization that concentrates parameters into fewer clusters and enhances motif reuse.



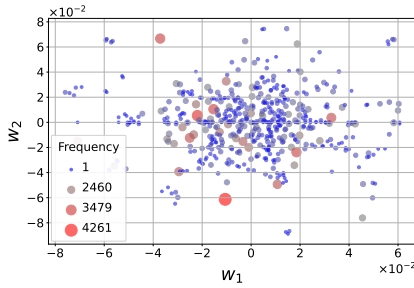
(a) (1, 2), Epoch 1



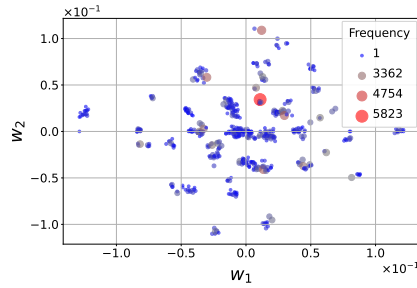
(b) (1, 2), Epoch 200



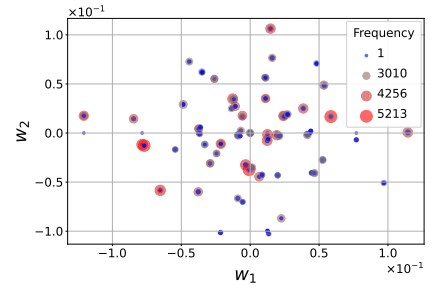
(c) (1, 2), Epoch 380



(d) (2, 1), Epoch 1



(e) (2, 1), Epoch 200



(f) (2, 1), Epoch 380

Figure 4.6: Scatter plots of motif coordinates in $\mathbb{R}^2$ across training epochs for shapes (1, 2) (top row) and (2, 1) (bottom row). Motifs progressively cluster into dense, overlapping coordinate regions by epoch 380.

Chapter 5

The V-Fold MoMos Parameterization

Standard Mosaic-of-Motifs ([Definition 4.1](#)) constrains neural network weight vectors by mapping parameter blocks of size s to a shared codebook of k learned motifs. As documented in [Chapter 4](#), the mosaic index map $\mathcal{M}$ forms the dominant storage consumer in high-dimensional models. While increasing the block size s reduces the number of blocks $m = \lceil n/s \rceil$, larger block geometries restrict hypothesis space expressivity. V-Fold MoMos addresses this trade-off through a two-level codebook hierarchy: block positions are partitioned into v folds, restricting each position to a localized codebook of $k' \leq k$ motifs and reducing mosaic storage from $\lceil \log_2 k \rceil$ to $\lceil \log_2 k' \rceil$ bits per entry.

Analogously to the baseline parameterization introduced in [Chapter 4](#), the compression ratio of a V-Fold MoMos parameterization is defined as:

$$r_{\text{V-Fold}}(\mathbf{w}) = \frac{nq}{\hat{\mathcal{K}}_{\text{V-FoldMoMos}}(\mathbf{w})}, \quad (5.1)$$

quantifying the description length of the V-Fold MoMos parameterization relative to the uncompressed nq -bit baseline. This hierarchical constraint alters training dynamics by concentrating block assignments onto a small set of frequently co-occurring motifs.

This chapter is structured as follows:

1. [Section 5.1](#) establishes the theoretical motivation and analyzes why recursive mosaic compression fails to yield genuine savings.
2. [Section 5.2](#) formalizes the V-Fold MoMos framework, deriving hypothesis space cardinality, description bounds, projection algorithms, and a worked reconstruction example.
3. [Section 5.4](#) analyzes capacity scaling behavior and structural regularization dynamics across network scales.
4. [Section 5.5](#) presents comprehensive empirical evaluations on CIFAR-10 across ResNet-20 and MLP architectures, including Pareto-optimal frontiers, 5-fold cross-validation verification,

storage allocations, algorithmic complexity tracking ($\hat{\mathcal{K}}_{\text{BDM}}, \hat{\mathcal{K}}_{\text{QuBD}}$), and long-run codebook collapse dynamics.

5. **Section 5.6** synthesizes the key insights and connects the parameterization to statistical generalization.

In the next section, we establish the theoretical foundation for localized codebooks by examining the limitations of recursive mosaic compression.

5.1 Theoretical Motivation and Recursive Limitations

As established in **Section 4.1.1** and **Section 4.4**, the description length of standard MoMos is dominated by the mosaic index map: $\mathcal{K}(\mathcal{M}) \leq m \lceil \log_2 k \rceil$. Shrinking this term by inflating the block size s reduces the block count $m = \lceil n/s \rceil$, but exponentially decreases the cardinality of the hypothesis class $|\mathcal{H}_{s,k}| \approx \binom{m}{k} k^{n/s}$, degrading empirical network capacity.

A naive approach to mosaic compression applies MoMos recursively to the flattened mosaic vector $\mathcal{M} \in [k]^m$. Given a weight vector $\mathbf{w} \in \mathbb{R}^n$, primary MoMos yields a motif matrix $\mathcal{Z} \in \mathbb{R}^{s \times k}$ and a mosaic $\mathcal{M} \in [k]^m$. Applying MoMos to $\mathcal{M}$ with a secondary block size s' and k' secondary motifs partitions $\mathcal{M}$ into $m' = m/s'$ secondary blocks. Because each index in $\mathcal{M}$ lies in $[k]$, the domain of secondary blocks is $[k]^{s'}$, requiring a secondary mosaic $\mathcal{M}' \in [k^{s'}]^{m'}$. To maintain a lossless representation, a secondary codebook $\mathcal{Z}' \in [k^{s'}]^{k'}$ must store up to $k^{s'}$ candidate blocks. Unless the number of unique secondary blocks in $\mathcal{M}$ is strictly smaller than m' , no structural redundancy is exploited; the secondary mosaic $\mathcal{M}'$ merely reshapes $\mathcal{M}$ into s' -element tuples without reducing description length. In addition, indexing among $k^{s'}$ motifs requires $\lceil \log_2 k^{s'} \rceil = \lceil s' \log_2 k \rceil \leq s'(\lceil \log_2 k \rceil + 1)$ bits per index, canceling the reduction in block count $m' = m/s'$.

Achieving genuine mosaic compression without enlarging the primary codebook $\mathcal{Z}$ requires restricting the active motif choices at each block position. By partitioning $\mathcal{Z}$ into v localized folds $\mathcal{I}_1, \dots, \mathcal{I}_v$, each holding $k' \leq k$ motifs, the mosaic index storage per block drops to $\lceil \log_2 k' \rceil$ bits. A deterministic partition mapping $\sigma: [m] \rightarrow [v]$ assigns each block position $i \in [m]$ to a specific fold $\mathcal{I}_{\sigma(i)}$, creating a two-level codebook hierarchy.

In the next section, we formalize this two-level codebook hierarchy, derive its description-length bounds, and detail its algorithmic implementation.

5.2 The V-Fold MoMos Framework

To eliminate the mosaic index bottleneck while maintaining a shared global motif codebook $\mathcal{Z}$, V-Fold MoMos couples global motif learning with position-specific fold vectors.

5.2.1 Formal Definition and Hypothesis Class

As shown in [Definition 4.1](#), standard MoMos maps each block directly into $\mathcal{Z}$. V-Fold MoMos introduces an intermediate indexing stage.

Definition 5.1 (V-Fold MoMos Parameterization). Let $\psi: \mathbb{R}^n \rightarrow \mathbb{R}^{s \times m}$ be a slicing operator, $k, k', v \in \mathbb{N}$ with $k' \leq k$ and $v \leq m$, and fix a partition mapping $\sigma: [m] \rightarrow [v]$. A V-Fold MoMos parameterization consists of a global motif matrix $\mathcal{Z} \in \mathbb{R}^{s \times k}$, a tuple of v fold vectors $\mathcal{I} = (\mathcal{I}_1, \dots, \mathcal{I}_v)$ with $\mathcal{I}_j \in [k]^{k'}$ for each $j \in [v]$, and a local mosaic $\mathcal{M} \in [k']^m$. The reconstructed parameter vector is:

$$\mathbf{w} = \psi^{-1} \left(\mathcal{Z} [\mathcal{I}_{\sigma(1)}[\mathcal{M}_1], \dots, \mathcal{I}_{\sigma(m)}[\mathcal{M}_m]] \right) \in \mathbb{R}^n. \quad (5.2)$$

Restricting parameter optimization to configurations expressible through the hierarchical fold decomposition defines the constrained search space.

Definition 5.2 (V-Fold MoMos Hypothesis Class). For $s, k, v, k' \in \mathbb{N}$ with $k' \leq k$ and $v \leq m$, the (s, k, v, k') -V-Fold MoMos hypothesis class is:

$$\mathcal{H}_{s,k,v,k'} = \left\{ \psi^{-1} \left(\mathcal{Z} [\mathcal{I}_{\sigma(1)}[\mathcal{M}_1], \dots, \mathcal{I}_{\sigma(m)}[\mathcal{M}_m]] \right) : \mathcal{Z} \in \mathbb{R}^{s \times k}, \mathcal{I} \in \left([k]^{k'} \right)^v, \mathcal{M} \in [k']^m \right\} \subseteq \mathcal{H}_{s,k}. \quad (5.3)$$

For each block position $i \in [m]$, the global motif index $\mathcal{I}_{\sigma(i)}[\mathcal{M}_i] \in [k]$ is evaluated in two stages: $\sigma(i)$ selects the fold assigned to position i , and $\mathcal{M}_i \in [k']$ indexes into that fold. Setting $k' = k$ with each $\mathcal{I}_j$ as the identity mapping recovers standard MoMos ($\mathcal{H}_{s,k,v,k} = \mathcal{H}_{s,k}$).

5.2.2 Hypothesis Space Cardinality

Quantizing scalar weights to q bits yields $N = 2^{sq}$ distinct candidate block patterns. Counting configurations $(\mathcal{Z}, \mathcal{I}, \mathcal{M})$ directly overcounts solutions, because distinct configurations frequently reconstruct identical weight vectors. To compute the exact cardinality $|\mathcal{H}_{s,k,v,k'}|$, we follow the structured combinatorial framework of [Leboeuf et al. \(2020\)](#):

1. **Define the target set:** Characterize exactly which reconstructed weight vectors $\hat{\mathbf{w}}$ the class contains.
2. **Decompose into disjoint sub-problems:** Split solutions by the number u of unique motifs that actually appear across the network.
3. **Count the assembly combinations:** Holding the motif set fixed, count admissible block assignments fold by fold.
4. **Recurse and invert:** Apply binomial inversion to ensure that every one of the u selected motifs is utilized by at least one block.

Applying the first step to $\mathcal{H}_{s,k,v,k'}$ gives the following realizability characterization.

Lemma 1 (Realizability). $\hat{\mathbf{w}} \in \mathcal{H}_{s,k,v,k'}$ if and only if (i) for every fold $j \in [v]$, the blocks of $\hat{\mathbf{w}}$ at fold- j positions take at most k' distinct values, and (ii) at most k distinct block values appear in $\hat{\mathbf{w}}$ overall.

Proof. Necessity: fold- j positions read motifs indexed by $\mathcal{I}_j$ with $|\mathcal{I}_j| = k'$; every block value is an entry of $\mathcal{Z}$. Sufficiency: let U be the set of block values of $\hat{\mathbf{w}}$, $|U| \leq k$. Build $\mathcal{Z}$ containing U and arbitrary padding; set $\mathcal{I}_j$ to the indices of the values used at fold j , padded to size k' ; let $\mathcal{M}$ read off the blocks. This reconstructs $\hat{\mathbf{w}}$. Unused or duplicate motifs in $\mathcal{Z}$ and $\mathcal{I}$ are harmless, so no constraint such as $|\mathcal{Z}| = |\bigcup_i \mathcal{I}_i|$ is needed on the class itself. $\square$

A solution is an assignment of one motif to each of the m blocks, subject to the two conditions of [Lemma 1](#): each fold's blocks use at most k' distinct motifs, and at most k distinct motifs appear in the whole network. The cardinality is computed in three steps:

1. Split solutions by the number u of unique motifs that actually appear among the blocks, ensuring every solution is counted in exactly one term.
2. Holding a set of motifs fixed, count the ways blocks can be filled fold by fold. Each fold, governing $m' = m/v$ blocks (assuming σ splits m blocks into v equal folds as in [Section 5.3](#)), is a plain MoMos instance; holding the motif set fixed decouples the folds and their counts multiply.
3. Correct for the requirement that every one of the u motifs is used by at least one block. This global condition ties the folds together, as a motif can serve blocks across multiple folds.

For the first step, a solution is determined by the grouping of blocks that share a motif together with an injective choice of which u of the $N = 2^{sq}$ possible motifs the groups carry:

$$|\mathcal{H}_{s,k,v,k'}| = \sum_{u=1}^k (N)_u \Pi_u, \quad (5.4)$$

where $(N)_u = N(N-1) \cdots (N-u+1)$ and Π_u is the number of admissible groupings with exactly u groups: at most k' groups meet each fold's blocks, by [Lemma 1](#). (This matches the growth-function decomposition $\tau(m) \leq \sum_u (n)_u \pi^u(m)$ of [Leboeuf et al. \(2020\)](#), holding with equality because [Lemma 1](#) identifies admissible groupings exactly.)

For the second step, work with u *distinguishable* motifs; distinguishability enables folds to coherently share motifs, and is removed at the end by dividing by $u!$. Fix a subset of $r \leq u$ motifs without requiring all of them to be used. Each fold independently fills its m' blocks from the r available motifs, using at most $\min(r, k', m')$ distinct ones. Each fold is therefore a plain MoMos

instance, contributing

$$A(r) = \sum_{t=1}^{\min(r, k', m')} (r)_t \left\{ \begin{matrix} m' \\ t \end{matrix} \right\}, \quad (5.5)$$

where t is the number of motifs the fold actually uses, and $\left\{ \begin{matrix} m' \\ t \end{matrix} \right\}$ is the Stirling number of the second kind, counting partitions of m' blocks into t nonempty subsets; $(r)_t$ assigns distinct motifs to subsets. The v folds contribute the product $A(r)^v$. Here r is a variable of the derivation rather than a model parameter.

For the third step, the product $A(u)^v$ also counts assignments leaving some of the u motifs unused. Such assignments use fewer than u motifs and belong to smaller terms of (Equation 5.4), so they must be excluded. Let $G(u)$ be the number of assignments where every one of the u motifs is used by at least one block. Classifying assignments counted by $A(u)^v$ by the subset of motifs actually used (of size $u' \leq u$):

$$A(u)^v = \sum_{u'=1}^u \binom{u}{u'} G(u') \quad \implies \quad G(u) = A(u)^v - \sum_{u'=1}^{u-1} \binom{u}{u'} G(u'). \quad (5.6)$$

Binomial inversion yields the closed form

$$G(u) = \sum_{w=0}^u (-1)^w \binom{u}{w} A(u-w)^v; \quad (5.7)$$

the alternating signs perform the required exclusion, since an assignment leaving exactly $d \geq 1$ motifs unused receives total weight $\sum_{w=0}^d (-1)^w \binom{d}{w} = 0$. Removing distinguishability, $\Pi_u = G(u)/u!$, and using $(N)_u/u! = \binom{N}{u}$, the exact cardinality is:

$$|\mathcal{H}_{s,k,v,k'}| = \sum_{u=1}^k \binom{N}{u} G(u), \quad N = 2^{sq}. \quad (5.8)$$

Three consistency checks verify (Equation 5.8):

1. When $k' \geq \min(m', k)$, the fold cap is vacuous, $A(r) = r^{m'}$, (Equation 5.7) reduces to the classical surjection count $G(u) = u! \left\{ \begin{matrix} m' \\ u \end{matrix} \right\}$, and (Equation 5.8) recovers the standard MoMos cardinality $\sum_u (N)_u \left\{ \begin{matrix} m' \\ u \end{matrix} \right\}$ (Theorem Theorem 5).
2. When $v = 1$, (Equation 5.8) reduces to standard MoMos with codebook size $\min(k, k')$.
3. When the zero motif $\mathbf{z}_1 \equiv 0$ is pinned, the $u = k$ term must contain the zero motif, replacing $\binom{N}{k}$ with $\binom{N-1}{k-1}$.

All three special cases, as well as (Equation 5.8), were verified against exhaustive enumeration of all $(\mathcal{Z}, \mathcal{I}, \mathcal{M})$ for small parameter settings.

The index u formalizes the number of motifs actually used, satisfying $u \leq \min(k, vk', m)$. Per-fold counting in isolation is not independent across folds: a shared motif merges block groups across folds, which the common global motif set handles exactly.

5.2.3 Description-Length Upper Bound

As derived in [Section 4.1.1](#), the description length of standard MoMos is bounded by $\mathcal{K}(\mathbf{w}) \leq ksq + m \lceil \log_2 k \rceil + \mathcal{O}(1)$. A V-Fold MoMos model is specified by the global motif codebook $\mathcal{Z}$, the fold vectors $\mathcal{I}$, the local mosaic $\mathcal{M}$, and the constant-size reconstruction routine $\mathcal{O}(1)$. Storing each scalar at q -bit precision yields the Kolmogorov complexity bound $\mathcal{K}(\mathbf{w})$:

$$\mathcal{K}(\mathbf{w}) \leq ksq + vk' \lceil \log_2 k \rceil + m \lceil \log_2 k' \rceil + \mathcal{O}(1). \quad (5.9)$$

Compared to standard MoMos ([Definition 4.1](#)), V-Fold MoMos reduces the mosaic term from $m \lceil \log_2 k \rceil$ to $m \lceil \log_2 k' \rceil$ by restricting block choices, at the expense of an overhead $vk' \lceil \log_2 k \rceil$ for the fold vectors $\mathcal{I}$. V-Fold MoMos achieves a strictly smaller description-length bound than standard MoMos when:

$$vk' \lceil \log_2 k \rceil \leq m \lceil \log_2(k/k') \rceil. \quad (5.10)$$

In the boundary regime where $k' = \lceil m/v \rceil$, each fold holds up to $m' = \lceil m/v \rceil$ motifs, matching the number of blocks assigned to its partition. Because these m' blocks contain at most m' distinct motifs, each fold can include all motifs utilized by its group, recovering the full MoMos hypothesis space ($\mathcal{H}_{s,k,v,k'} = \mathcal{H}_{s,k}$). However, setting $k' = \lceil m/v \rceil$ introduces redundant overhead for $\mathcal{I}$, making the description-length bound strictly larger than standard MoMos. Genuine compression requires a strict sub-codebook restriction, $k' < \lceil m/v \rceil$, where $\mathcal{H}_{s,k,v,k'} \subsetneq \mathcal{H}_{s,k}$.

5.3 Algorithmic Implementation

V-Fold MoMos is implemented as a secondary projection step following a primary MoMos 2D projection ([Definition 4.3](#)). As shown in [Section 4.1.2](#), the primary projection constructs the global motif matrix $\mathcal{Z} \in \mathbb{R}^{s \times k}$ and initial mosaic assignments $\mathcal{M}$. The secondary projection refines $\mathcal{M}$ into fold-restricted form while keeping $\mathcal{Z}$ fixed.

Extending MoMos to a two-level hierarchy requires formalizing the spatial grouping of blocks into composites. In two-dimensional MoMos ([Definition 4.3](#)), the slicing operator $\psi_{r,c}$ partitions a 2D weight matrix into blocks of shape (r, c) . In V-Fold MoMos, a higher-level slicing operator $\psi_{\hat{r}, \hat{c}}$ groups these parameter blocks into composites of shape $(\hat{r}, \hat{c})$, each containing $v = \hat{r} \hat{c}$ blocks. The relative 2D coordinate of a block within its composite determines its position index $\sigma(i) \in [v]$, mapping each block to its assigned fold.

Fold Construction. For each composite position $i \in [v]$, motif selection counts are aggregated across all composites. Fold $\mathcal{I}_i$ retains $k' = \lceil k \cdot c' \rceil$ motifs via frequency-weighted sampling without replacement based on per-position counts $\mathbf{c}_i$, where $c' \in (0, 1]$ denotes the secondary capacity factor. The zero motif is explicitly anchored at index 1 of every fold $\mathcal{I}_i$, producing fold vectors $\mathcal{I} = (\mathcal{I}_1, \dots, \mathcal{I}_v)$ with $\mathcal{I}_i \in [k]^{k'}$.

Block Reassignment. Following fold construction, every block $\mathbf{b}_i$ is reassigned to the nearest motif within its assigned fold under squared L_2 distance:

$$\mathcal{M}_i = \arg \min_{j \in [k']} \left\| \mathbf{b}_i - \mathbf{z}_{\mathcal{I}_{\sigma(i)}[j]} \right\|_2^2. \quad (5.11)$$

The weight vector is reconstructed as $\mathbf{w} = \psi_{r,c}^{-1} \left(\mathcal{Z} [\mathcal{I}_{\sigma(1)}[\mathcal{M}_1], \dots, \mathcal{I}_{\sigma(m)}[\mathcal{M}_m]] \right)$, projecting parameters onto $\mathcal{H}_{s,k,v,k'}$. **Algorithm 3** summarizes the complete training loop.

Algorithm 3 V-Fold MoMos Training Loop

Require: Dataset $\mathcal{D}$, block shape (r, c) with $s = rc$, motif count k , composite shape $(\hat{r}, \hat{c})$ with $v = \hat{r}\hat{c}$, secondary capacity c' , epochs T

Ensure: Optimized V-Fold parameters $(\mathcal{Z}, \mathcal{I}, \mathcal{M})$

```

1: Initialize weights  $\mathbf{w}^{(0)} \in \mathbb{R}^n$ 
2: for  $t = 1$  to  $T$  do
3:                                      $\triangleright$  Step 1: Back-propagation
4:   Update  $\mathbf{w}^{(t)}$  by training on  $\mathcal{D}$  via back-propagation
5:                                      $\triangleright$  Step 2: MoMos 2D projection
6:    $\mathcal{Z}, \mathcal{M} \leftarrow$  MoMos 2D projection of  $\psi_{r,c}(\mathbf{w}^{(t)})$ 
7:                                      $\triangleright$  Step 3: V-Fold projection
8:   Group blocks into composites of shape  $(\hat{r}, \hat{c})$ 
9:    $\sigma(i) \leftarrow$  position of block  $i$  inside its composite
10:  for  $i = 1$  to  $v$  do
11:     $\mathbf{c}_i \leftarrow$  counts of motifs at position  $i$  across all composites
12:     $\mathcal{I}_i[1] \leftarrow 0$ 
13:     $(\mathcal{I}_i[j])_{j=2}^{k'} \sim \text{Unif}_{\mathbf{c}_i}([k])$  where  $k' = \lceil kc' \rceil$ 
14:  end for
15:  for  $i = 1$  to  $m$  do
16:     $\mathcal{M}_i \leftarrow \arg \min_{j \in [k']} \left\| \mathbf{b}_i - \mathbf{z}_{\mathcal{I}_{\sigma(i)}[j]} \right\|_2^2$ 
17:  end for
18:   $\mathbf{w}^{(t)} \leftarrow \psi_{r,c}^{-1} \left( \mathcal{Z} [\mathcal{I}_{\sigma(1)}[\mathcal{M}_1], \dots, \mathcal{I}_{\sigma(m)}[\mathcal{M}_m]] \right)$ 
19:                                      $\triangleright$  Step 4: Evaluation
20:  Compute validation accuracy and loss
21: end for
22: return  $(\mathcal{Z}, \mathcal{I}, \mathcal{M})$ 

```

5.3.1 Worked Reconstruction Example

To illustrate the reconstruction pipeline and description-length accounting, we evaluate a single 4×8 weight matrix $\mathbf{w}$ ($n = 32$) with $q = 32$ -bit precision. The matrix exhibits structural repetitions so that all MoMos representations are exact:

$$\mathbf{w} = \begin{bmatrix} 1 & 2 & 5 & 6 & 3 & 4 & 7 & 8 \\ 9 & 10 & 13 & 14 & 11 & 12 & 15 & 16 \\ 3 & 4 & 7 & 8 & 1 & 2 & 5 & 6 \\ 11 & 12 & 15 & 16 & 9 & 10 & 13 & 14 \end{bmatrix}. \quad (5.12)$$

Parameters are grouped into $m = 16$ blocks of size $s = 2$, drawn from a global codebook of $k = 8$ motifs.

V-Fold MoMos to MoMos. The global motif matrix $\mathcal{Z} \in \mathbb{R}^{2 \times 8}$ contains horizontal pairs as columns:

$$\mathcal{Z} = \begin{bmatrix} 1 & 5 & 3 & 7 & 9 & 13 & 11 & 15 \\ 2 & 6 & 4 & 8 & 10 & 14 & 12 & 16 \end{bmatrix}. \quad (5.13)$$

Blocks are grouped into 2×2 composites ($v = 4$). Each composite position is assigned a fold $\mathcal{I}_j \in [8]^2$ containing $k' = 2$ motif indices:

$$\mathcal{I}_1 = \boxed{1, 3}, \quad \mathcal{I}_2 = \boxed{2, 4}, \quad \mathcal{I}_3 = \boxed{5, 7}, \quad \mathcal{I}_4 = \boxed{6, 8}, \quad (5.14)$$

where colors indicate the position within the composite. The local mosaic $\mathcal{M} \in [2]^{16}$ stores 1-bit indices pointing into these folds:

$$\mathcal{M} = \begin{bmatrix} \boxed{1} & \boxed{1} & \boxed{2} & \boxed{2} \\ \boxed{1} & \boxed{1} & \boxed{2} & \boxed{2} \\ \boxed{2} & \boxed{2} & \boxed{1} & \boxed{1} \\ \boxed{2} & \boxed{2} & \boxed{1} & \boxed{1} \end{bmatrix} \in [2]^{16}. \quad (5.15)$$

Mapping each local entry $\mathcal{M}_i$ through its assigned fold, $\mathcal{M}_i^{\text{global}} = \mathcal{I}_{\sigma(i)}[\mathcal{M}_i]$, recovers the standard MoMos mosaic:

$$\begin{bmatrix} \boxed{1} & \boxed{1} & \boxed{2} & \boxed{2} \\ \boxed{1} & \boxed{1} & \boxed{2} & \boxed{2} \\ \boxed{2} & \boxed{2} & \boxed{1} & \boxed{1} \\ \boxed{2} & \boxed{2} & \boxed{1} & \boxed{1} \end{bmatrix} \xrightarrow{\mathcal{I}_{\sigma(i)}[\cdot]} \begin{bmatrix} \boxed{1} & \boxed{2} & \boxed{3} & \boxed{4} \\ \boxed{5} & \boxed{6} & \boxed{7} & \boxed{8} \\ \boxed{3} & \boxed{4} & \boxed{1} & \boxed{2} \\ \boxed{7} & \boxed{8} & \boxed{5} & \boxed{6} \end{bmatrix} = \mathcal{M}^{\text{global}}. \quad (5.16)$$

MoMos to Weight Vector. Reconstructing the weight matrix $\mathbf{w}$ from $\mathcal{M}^{\text{global}}$ and $\mathcal{Z}$, we color the global mosaic by motif:

$$\mathcal{M}^{\text{global}} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 3 & 4 & 1 & 2 \\ 7 & 8 & 5 & 6 \end{bmatrix}. \quad (5.17)$$

Replacing each index in $\mathcal{M}^{\text{global}}$ with the corresponding column of $\mathcal{Z}$:

$$\mathcal{Z} = \begin{bmatrix} 1 & 5 & 3 & 7 & 9 & 13 & 11 & 15 \\ 2 & 6 & 4 & 8 & 10 & 14 & 12 & 16 \end{bmatrix}, \quad (5.18)$$

reconstructs the final weight matrix $\mathbf{w}$:

$$\mathbf{w} = \begin{bmatrix} 1 & 2 & 5 & 6 & 3 & 4 & 7 & 8 \\ 9 & 10 & 13 & 14 & 11 & 12 & 15 & 16 \\ 3 & 4 & 7 & 8 & 1 & 2 & 5 & 6 \\ 11 & 12 & 15 & 16 & 9 & 10 & 13 & 14 \end{bmatrix}. \quad (5.19)$$

Description-Length Comparison. Using the bounds derived in Equation 5.9 and Equation 4.3, and omitting common constant routine terms $\mathcal{O}(1)$:

1. *Unconstrained Baseline*: Storing 32 parameters at 32-bit precision requires $nq = 32 \cdot 32 = 1024$ bits.
2. *MoMos* ($s = 2, k = 8$): Motif matrix costs $ksq = 8 \cdot 2 \cdot 32 = 512$ bits; mosaic storage costs $m \lceil \log_2 k \rceil = 16 \cdot 3 = 48$ bits; total is 560 bits.
3. *V-Fold MoMos* ($1 \times 2, v = 4, k' = 2$): Motif matrix costs 512 bits; fold vectors cost $vk' \lceil \log_2 k \rceil = 4 \cdot 2 \cdot 3 = 24$ bits; local mosaic costs $m \lceil \log_2 k' \rceil = 16 \cdot 1 = 16$ bits; total is 552 bits.

V-Fold MoMos achieves superior compression because $vk' \lceil \log_2 k \rceil = 24 \leq m \lceil \log_2(k/k') \rceil = 32$. 5.1 summarizes bit allocations and compression ratios across parameterizations.

In the next section, we analyze capacity scaling properties and examine how the two-level fold hierarchy introduces structural regularization across different network scales.

5.4 Capacity Scaling and Structural Regularization

As shown in Section 4.3.1, capacity $\text{cap} = k/m$ fails to provide a scale-invariant metric across parameter counts n . However, for a fixed architecture and fixed block size $s = 2$, fixing the

Representation	$\mathcal{Z}$	$\mathcal{M}$	$\mathcal{I}$	Total	RAS
Full precision	—	—	—	1024	$1.0\times$
MoMos ($s=2, k=8$)	512	48	—	560	$1.8\times$
MoMos 2D ($2\times 1, k=8$)	512	48	—	560	$1.8\times$
V-Fold ($1\times 2, v=4, k'=2$)	512	16	24	552	$1.9\times$

Table 5.1: Description length in bits ($q = 32$) for the 4×8 matrix example across parameterizations, split into codebook $\mathcal{Z}$, mosaic $\mathcal{M}$, and folds $\mathcal{I}$, alongside compression ratio RAS.

primary capacity to $\text{cap} = 0.1\%$ provides a controlled baseline. Secondary fold codebook sizes in V-Fold MoMos scale via the secondary capacity factor $c' \in (0, 1]$, setting $k' = \lceil k \cdot c' \rceil$.

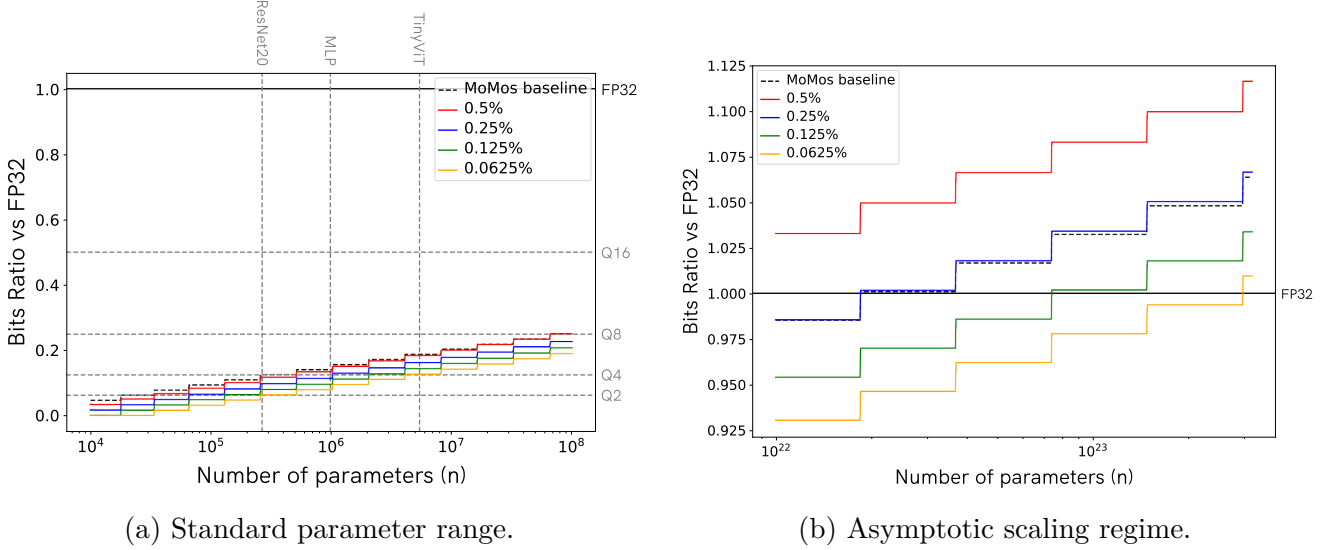


Figure 5.1: Relative description-length ratio of MoMos and V-Fold MoMos compared to the full-precision 32-bit baseline across parameter counts: (a) standard parameter range, and (b) over-parameterized regime where linear capacity scaling causes bit ratios relative to full precision to exceed 1.0.

Figure 5.1 (a) illustrates relative description-length curves computed from analytical complexity bounds ($q = 32$). The V-Fold MoMos models use $v = 128$ folds with fold capacities $c' \in \{1/2, 1/4, 1/8, 1/16\}$.

Figure 5.1 (b) shows that parameterizing motif counts linearly with parameter count ($k = n \cdot \text{cap}/s$) causes description-length bottlenecks in over-parameterized regimes. Hypothesis space cardinality grows super-exponentially, $|\mathcal{H}_{s,k}| \propto k^m = (m \cdot \text{cap})^m$, with $m = n/s$. If k scales linearly with n , total description length eventually exceeds the full-precision baseline nq . Guaranteeing genuine asymptotic compression as $n \rightarrow \infty$ requires k to scale sublinearly with n .

V-Fold MoMos imposes regularization across two distinct structural levels: the local motif codebook ($\mathcal{Z} \in \mathbb{R}^{s \times k}$) and the composite tiling layout. While standard MoMos permits k^v composite arrangements over v block positions, V-Fold restricts position j to fold $\mathcal{I}_j$ of size k' , reducing

allowable composites to $(k')^v$. This represents an exponential structural reduction by $(k/k')^v$ across the network, acting as a broader regularizer while preserving local motif flexibility.

Parameterizing V-Fold MoMos with $k' = \lceil c'm' \rceil$ alters the asymptotic compression behavior. In this formulation, all three terms of the bound grow as functions of n , and standard MoMos eventually achieves stronger theoretical compression than unconstrained V-Fold MoMos. In realistic parameter ranges with fixed sublinear codebook sizes, V-Fold MoMos consistently provides a strict description-length advantage.

In the next section, we evaluate V-Fold MoMos empirically on standard neural network benchmarks, assessing Pareto efficiency, cross-validation stability, and training dynamics.

5.5 Empirical Evaluation

Empirical performance is evaluated using multi-objective optimization on a 5-layer multi-layer perceptron (MLP) architecture (Bakhtiarifard et al., 2026a) and ResNet-20 (He et al., 2015) on CIFAR-10 classification:

1. *Baseline Reproduction*: Reproducing the baseline MLP pipeline demonstrates that training for 100 epochs matches the predictive accuracy reported for 200 epochs in Bakhtiarifard et al. (2026a).
2. *Pareto Front Identification*: Hyperparameter search via Optuna identifies the Pareto optimal frontier across four training paradigms: unconstrained FP32, QAT, standard MoMos, and V-Fold MoMos.
3. *Cross-Validation Protocol*: Datasets are partitioned into 5-fold cross-validation splits (4 training folds, 1 validation fold). Each candidate configuration on the Pareto frontier is evaluated across 5 distinct random initialization seeds over 200 epochs.
4. *Multi-Objective Genetic Optimization*: Pareto frontiers are extracted using the NSGA-II multi-objective genetic algorithm (Deb et al., 2002).
5. *Training Dynamics Metrics*: Evaluation along the Pareto frontier tracks parameter sparsity, binarized Block Decomposition Method complexity ($\hat{\mathcal{K}}_{\text{BDM}}$) (Zenil et al., 2018), Quantized Block Decomposition ($\hat{\mathcal{K}}_{\text{QuBD}}$) (Bakhtiarifard et al., 2026b), weight entropy, parameter L_2 norm, and LZMA lossless compression ratios.

5.5.1 Pareto Analysis and Cross-Validation Verification

Both MoMos and V-Fold MoMos exhibit strong potential for further compression when coupled with entropy coding, such as Huffman encoding (Huffman, 1952), because empirical motif distributions across mosaics and folds are highly non-uniform.

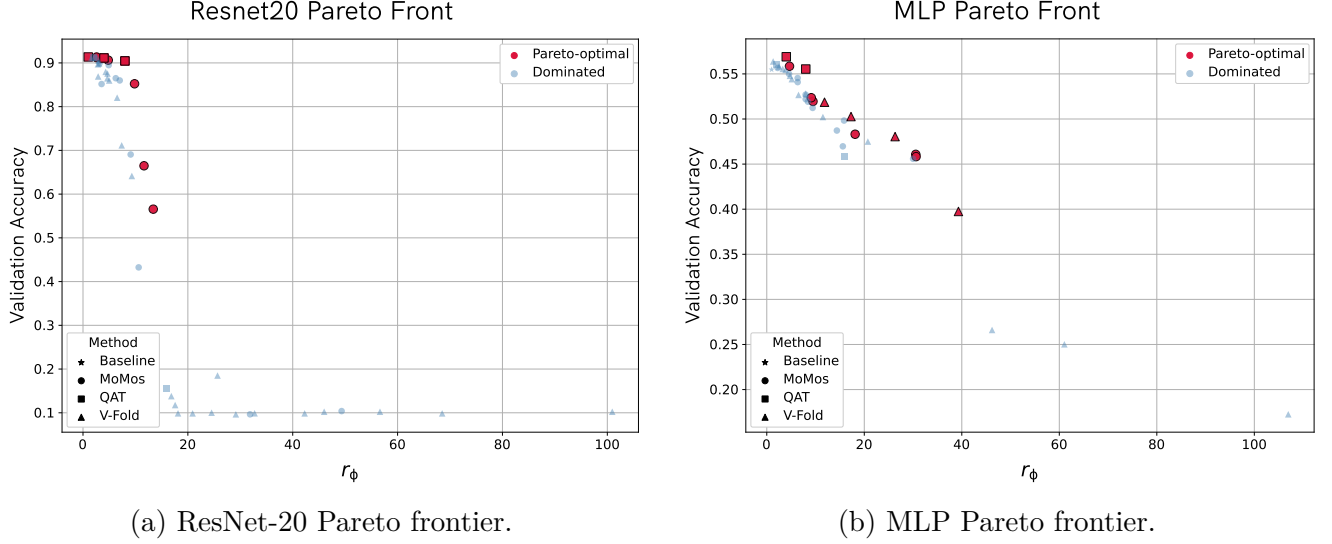


Figure 5.2: Pareto-optimal frontiers identified via NSGA-II multi-objective optimization across compression ratio r_ϕ and validation accuracy for unconstrained baseline, QAT, standard MoMos, and V-Fold MoMos on (a) ResNet-20 and (b) MLP architectures.

Figure 5.2 reveals clear differences in compression behavior between convolutional and fully connected architectures. On ResNet-20 (5.2a), standard MoMos competes directly with QAT along the Pareto frontier, whereas V-Fold MoMos configurations remain strictly dominated. Convolutional architectures possess strong architectural inductive biases through spatial weight sharing and residual skip connections. In relatively compact networks such as ResNet-20, hierarchical fold constraints impose indexing overhead while restricting the small number of convolutional filter motifs, over-regularizing the parameter space. V-Fold MoMos is therefore most beneficial for large-scale, over-parameterized networks where broader structural regularization prevents overfitting without exhausting parameter capacity.

On the MLP architecture (5.2b), V-Fold MoMos configurations interpolate smoothly between standard MoMos parameterizations along the Pareto frontier. This demonstrates that V-Fold MoMos provides finer-grained control over compression trade-offs, enabling practitioners to target precise intermediate bit budgets, even though its uncompressed Pareto envelope closely matches that of standard MoMos.

Table 5.2 reports the performance and complexity metrics for all Pareto-optimal configurations. On ResNet-20, QAT4 and MoMos 2D (2, 2) reach $r_\phi \approx 8.00$ and $r_\phi \approx 9.83$ with test accuracies of 91.36% and 83.08%, respectively. On the MLP architecture, V-Fold MoMos configurations achieve high compression ratios ($r_\phi \in [11.84, 39.31]$), maintaining test accuracy within 41.12% to 53.17% while substantially reducing normalized algorithmic complexity (r_{QuBD} decreases from 0.869 down to 0.365, and r_{BDM} reaches 0.098).

Method	Configuration	$r_\phi \uparrow$	Test Acc. (%) $\uparrow$	$r_{\text{BDM}} \downarrow$	$r_{\text{QuBD}} \downarrow$
ResNet-20 (CIFAR-10)					
Baseline	FP32 ($q = 32$)	1.00	91.95 ± 0.10	0.996	0.967
MoMos 2D	$(1, 1)$, $k = 2253$	2.61	92.05 ± 0.18	0.991	0.932
QAT	$q = 8$	4.00	91.96 ± 0.17	0.992	0.958
QAT	$q = 4$	8.00	91.36 ± 0.31	0.976	0.891
MoMos 2D	$(2, 2)$, $k = 1063$	9.83	83.08 ± 2.46	0.125	0.924
MoMos 2D	$(2, 2)$, $k = 526$	11.64	75.36 ± 2.41	0.118	0.880
MoMos 2D	$(1, 4)$, $k = 291$	13.40	48.52 ± 5.50	0.313	0.889
MLP (CIFAR-10)					
QAT	$q = 8$	4.00	58.03 ± 0.49	0.371	0.869
QAT	$q = 4$	8.00	57.51 ± 0.37	0.370	0.479
MoMos 2D	$(2, 2)$, $k = 2891$	9.48	54.38 ± 0.23	0.875	0.794
V-Fold	$(2, 2)$, $k = 771$, $v = 380$	11.84	53.17 ± 0.37	0.509	0.740
V-Fold	$(2, 2)$, $k = 517$, $v = 79$	17.28	51.41 ± 0.40	0.298	0.621
MoMos 2D	$(2, 2)$, $k = 122$	18.12	50.39 ± 0.42	0.404	0.751
V-Fold	$(1, 4)$, $k = 209$, $v = 32$	26.32	49.26 ± 0.11	0.098	0.448
MoMos 2D	$(2, 4)$, $k = 182$	30.56	47.92 ± 0.54	0.445	0.352
MoMos 2D	$(2, 4)$, $k = 173$	30.63	47.61 ± 0.42	0.391	0.336
V-Fold	$(1, 4)$, $k = 234$, $v = 64$	39.31	41.12 ± 1.18	0.132	0.365

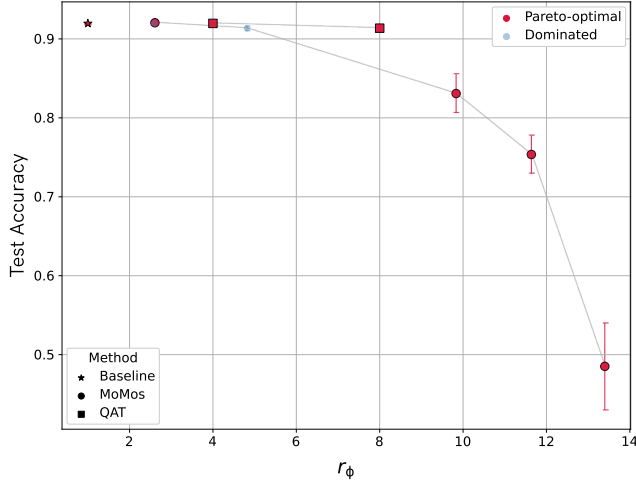
Table 5.2: Cross-validated Pareto-optimal configurations for ResNet-20 and MLP on CIFAR-10 (5 folds, 200 epochs). Test accuracy values report the empirical mean $\pm$ standard deviation across the 5 cross-validation folds. r_ϕ denotes the description-length compression ratio, while $r_{\text{BDM}} = \hat{\mathcal{K}}_{\text{BDM}} / \max(\hat{\mathcal{K}}_{\text{BDM}})$ and $r_{\text{QuBD}} = \hat{\mathcal{K}}_{\text{QuBD}} / \max(\hat{\mathcal{K}}_{\text{QuBD}})$ report normalized BDM and QuBD complexity metrics.

5.5.2 Storage Allocation and Codebook Breakdown

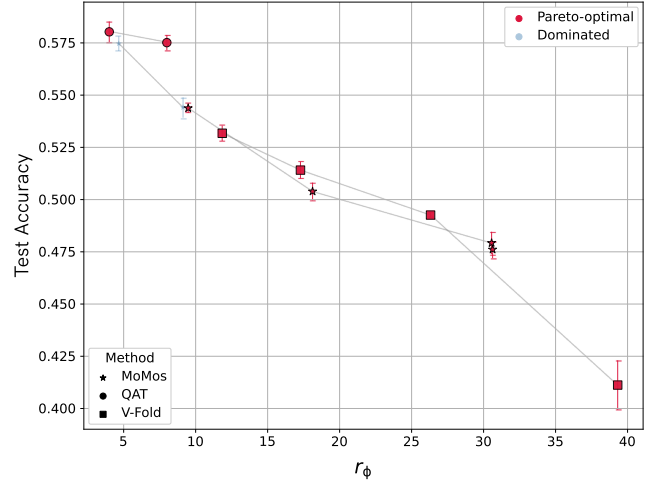
To verify the preliminary Pareto frontiers, configurations on the frontier were retrained for 200 epochs using 5-fold cross-validation. While theoretical compression ratios r_ϕ are deterministic functions of parameter counts and codebook dimensions, validation accuracy exhibits stochastic variation across initialization seeds and dataset splits.

Figure 5.3 (a–b) confirms that the initial Pareto frontiers remain stable under full cross-validation. Cross-validation variance is tightly bounded across well-performing configurations. Highly constrained or suboptimal parameterizations display larger performance variance during early training, which diminishes consistently as validation accuracy converges. Because both MoMos and V-Fold MoMos employ frequency-weighted stochastic sampling during projection, this empirical stability indicates that the two-level codebook hierarchy reliably identifies representative motif sub-codebooks.

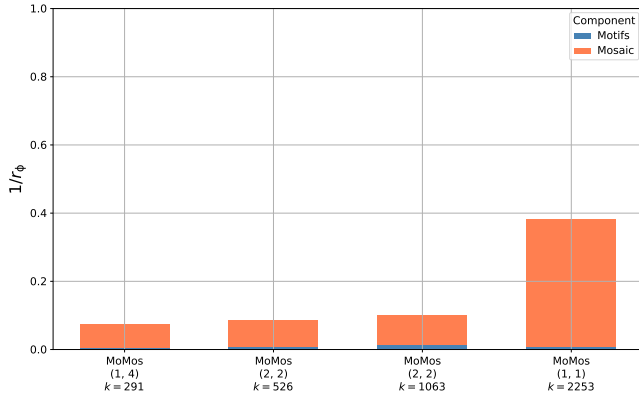
Figure 5.3 (c–d) details the storage distribution $1/r_\phi$ across model components. Across both architectures, the mosaic index map $\mathcal{M}$ remains the dominant storage consumer. In V-Fold



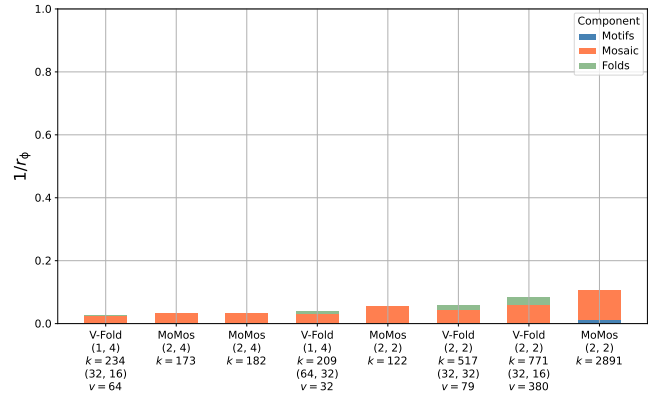
(a) ResNet-20 validation accuracy vs. r_ϕ .



(b) MLP validation accuracy vs. r_ϕ .



(c) ResNet-20 component description length ($1/r_\phi$).



(d) MLP component description length ($1/r_\phi$).

Figure 5.3: Cross-validation verification across 5 folds (200 epochs). Top row: validation accuracy against compression ratio r_ϕ for (a) ResNet-20 and (b) MLP. Bottom row: fractional description length decomposition ($1/r_\phi$) across motifs, mosaic, and fold codebooks for (c) ResNet-20 and (d) MLP.

MoMos configurations, fold vectors $\mathcal{I}$ account for a sizable fraction of total description length while reducing mosaic bit-depth from $\lceil \log_2 k \rceil$ to $\lceil \log_2 k' \rceil$. Applying entropy coding to both $\mathcal{M}$ and $\mathcal{I}$ presents immediate opportunities to reduce mosaic and fold algorithmic complexity.

5.5.3 Training Dynamics and Algorithmic Complexity Trajectories

Figure 5.4 tracks optimization dynamics over 200 epochs for representative Pareto configurations. MoMos parameterizations on ResNet-20 (5.4a) exhibit pronounced loss oscillations and slower convergence relative to the baseline, reflecting the friction between discrete projection steps and convolutional filter structure. Conversely, optimization on the MLP architecture (5.4b) oscillates less, and V-Fold MoMos accentuates the gradual late-stage plateau in training and validation accuracy observed in standard MoMos.

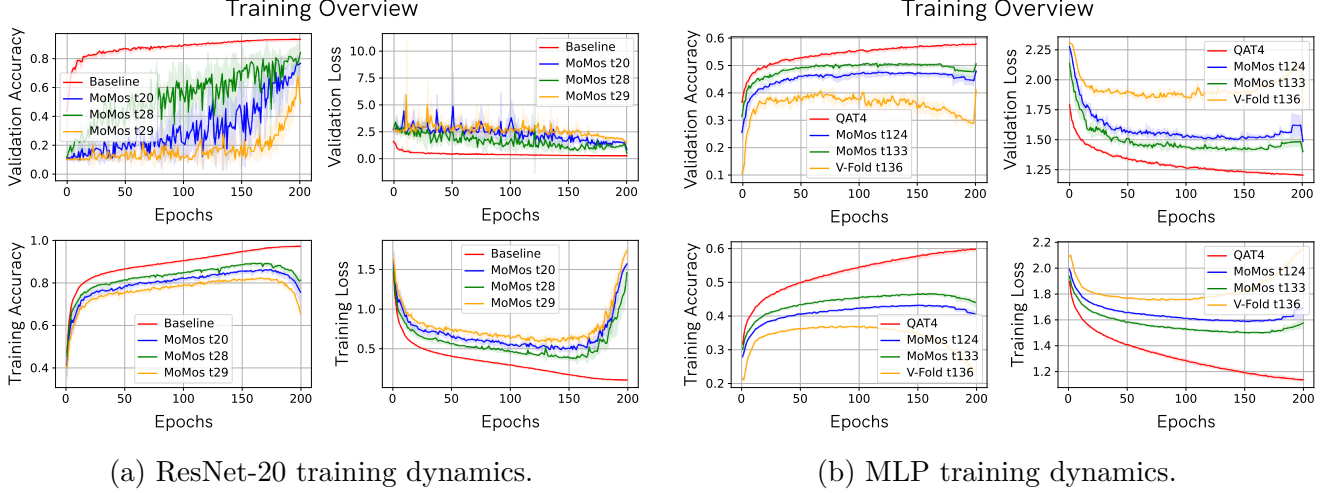


Figure 5.4: Training and validation trajectories over 200 epochs across 5-fold cross-validation for representative Pareto configurations on (a) ResNet-20 (unconstrained baseline and MoMos (2, 2) variants with $k \in \{291, 526, 1063\}$) and (b) MLP (QAT4, MoMos (2, 4) with $k = 173$, MoMos (2, 2) with $k = 122$, and V-Fold MoMos (1, 4) with $k = 234, v = 64$).

Figure 5.5 traces the evolution of algorithmic complexity estimators ($\hat{\mathcal{K}}_{\text{BDM}}$, $\hat{\mathcal{K}}_{\text{QuBD}}$) and normalized weight entropy against validation loss. For the unconstrained baseline on ResNet-20, parameter entropy correlates closely with validation loss reduction, confirming that unregularized gradient descent compresses representations as learning progresses.

Following the framework of Bakhtiarifard et al. (2026b), Figure 5.5 shows that the ResNet-20 baseline exhibits increased validation loss beyond epoch 100, consistent with overfitting. This observation explains why training for 100 epochs matches the generalization performance of 200 epochs. MoMos and V-Fold MoMos parameterizations mitigate this overfitting behavior, as both $\hat{\mathcal{K}}_{\text{BDM}}$ and $\hat{\mathcal{K}}_{\text{QuBD}}$ decrease monotonically throughout training for ResNet-20 and MLP models.

$\hat{\mathcal{K}}_{\text{QuBD}}$ captures a pronounced reduction in complexity during late-stage MLP training. While normalized parameter entropy reflects a similar downward trend, it does not account for the multi-scale structural dependencies tracked by block decomposition methods. During early training, algorithmic complexity decreases rapidly across all models (unconstrained, QAT, MoMos, and V-Fold MoMos), indicating that early optimization moves parameters toward structurally simpler regions of the parameter space.

As optimization converges and gradient magnitudes diminish, stochastic batch sampling and momentum noise begin to dominate parameter updates in unconstrained models. In contrast, discrete projections under MoMos and V-Fold MoMos exert stronger structural pressure as gradient steps shrink. This dynamic guides parameters toward algorithmically simpler configurations, inducing the subtle late-stage accuracy plateau while enforcing substantially lower description length.

5.5.4 Long-Run Convergence and Motifs Clustering

As an exploratory study of long-run optimization dynamics rather than terminal accuracy benchmarking, a 400-epoch training run was conducted on CIFAR-10 with the MLP architecture. MoMos baseline configurations with shapes $(1, 2)$ and $(2, 1)$ reached final test accuracies of 57.00% and 56.14%, respectively. V-Fold MoMos $((2, 1)$, capacity 0.1%, composite shape $(4, 32)$ yielding $v = 128$ folds, secondary capacity factor $c' = 1/4$) achieved a test accuracy of 55.24%, reflecting the higher compression level and codebook restriction.

Figure 5.6 analyzes the storage distribution between standard MoMos and V-Fold MoMos across linear and logarithmic scales.

Figure 5.7 illustrates the 400-epoch training and validation trajectories. The structural regularization imposed by V-Fold MoMos becomes apparent by epoch 100, where its validation trajectory settles below the less constrained MoMos $(1, 2)$ and $(2, 1)$ configurations while remaining above the larger $(2, 2)$ block geometry.

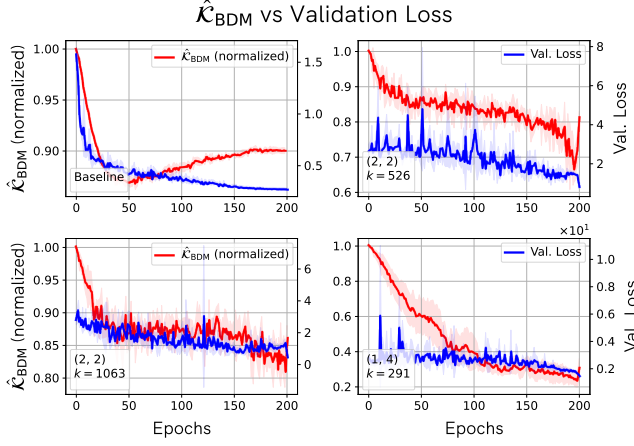
Figure 5.8 tracks the evolution of motif assignment frequencies across training. During the first 200 epochs, motif frequencies evolve similarly in MoMos and V-Fold MoMos. Over the final 200 epochs, the active motif count in V-Fold MoMos decreases from several hundred to approximately 60, while the peak assignment frequency increases by an order of magnitude, alongside a long tail of rarely selected motifs.

Figure 5.9 displays the spatial distribution of motif coordinates in $\mathbb{R}^2$ across training. At initialization, motifs are distributed uniformly in both parameterizations. By epoch 200, both MoMos and V-Fold MoMos form spatial clusters, with V-Fold MoMos exhibiting fewer, more separated clusters. By epoch 380, these clusters merge into a small number of dense coordinate locations. As reflected in the frequency scale, V-Fold MoMos converges toward a parameter configuration with substantially lower description length.

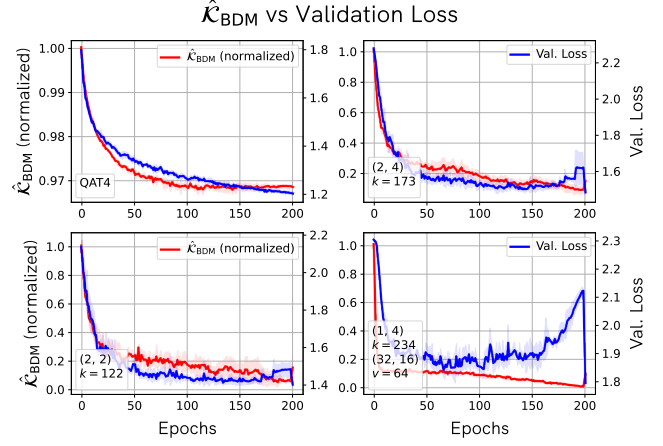
5.6 Chapter Summary

In summary, standard MoMos clusters parameter blocks into a shared codebook, often collapsing effective codebook size below the nominal bound $\mathcal{H}_{s,k}$. V-Fold MoMos introduces multi-scale clustering dynamics by coupling global motif learning with localized fold restrictions.

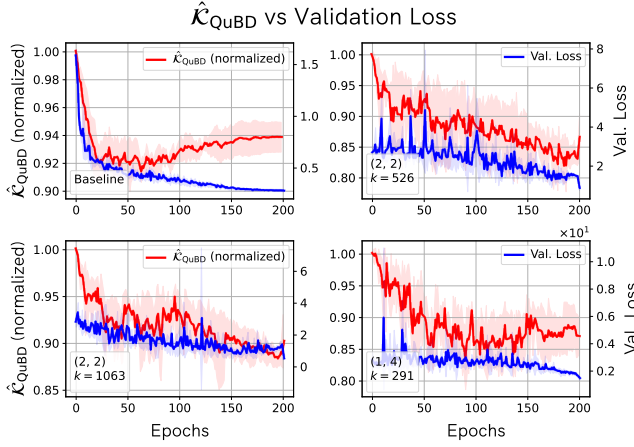
In Chapter 6, we connect Kolmogorov complexity bounds with generalization guarantees by analyzing the expressivity and VC dimension of MoMos and V-Fold MoMos hypothesis classes.



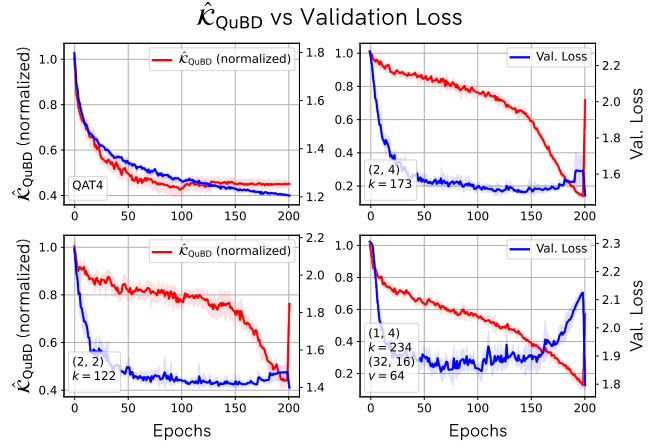
(a) ResNet-20 $\hat{K}_{\text{BDM}}$ vs. validation loss.



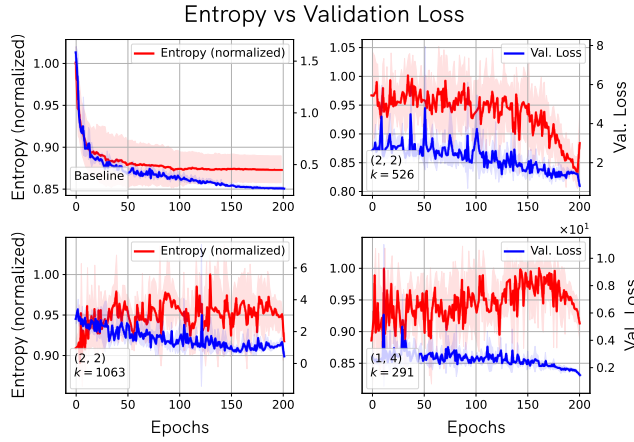
(b) MLP $\hat{K}_{\text{BDM}}$ vs. validation loss.



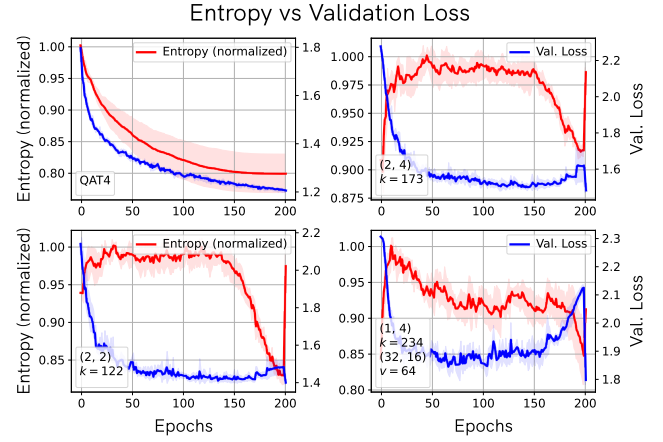
(c) ResNet-20 $\hat{K}_{\text{QuBD}}$ vs. validation loss.



(d) MLP $\hat{K}_{\text{QuBD}}$ vs. validation loss.

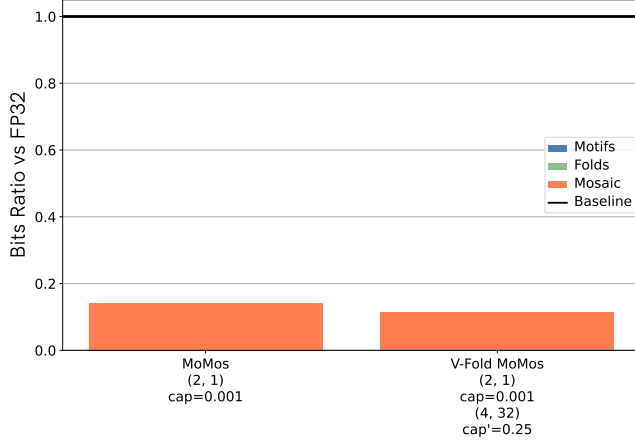


(e) ResNet-20 normalized entropy vs. validation loss.

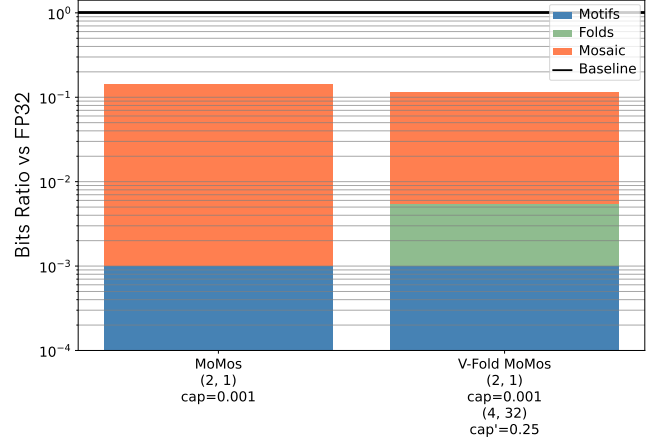


(f) MLP normalized entropy vs. validation loss.

Figure 5.5: Algorithmic complexity and entropy dynamics across 200 training epochs against validation loss for ResNet-20 (left column) and MLP (right column): (a–b) normalized Block Decomposition Method ($\hat{K}_{\text{BDM}}$), (c–d) Quantized Block Decomposition ($\hat{K}_{\text{QuBD}}$), and (e–f) normalized parameter entropy.



(a) Linear scale breakdown.



(b) Logarithmic scale breakdown.

Figure 5.6: Component description-length breakdown for MoMos (2, 1) (cap = 0.1%) and V-Fold MoMos (2, 1) (cap = 0.1%, $v = 128$, $c' = 0.25$) on (a) linear scale and (b) logarithmic scale, illustrating the relative storage contribution of motifs, folds, and mosaic maps.

Training Overview

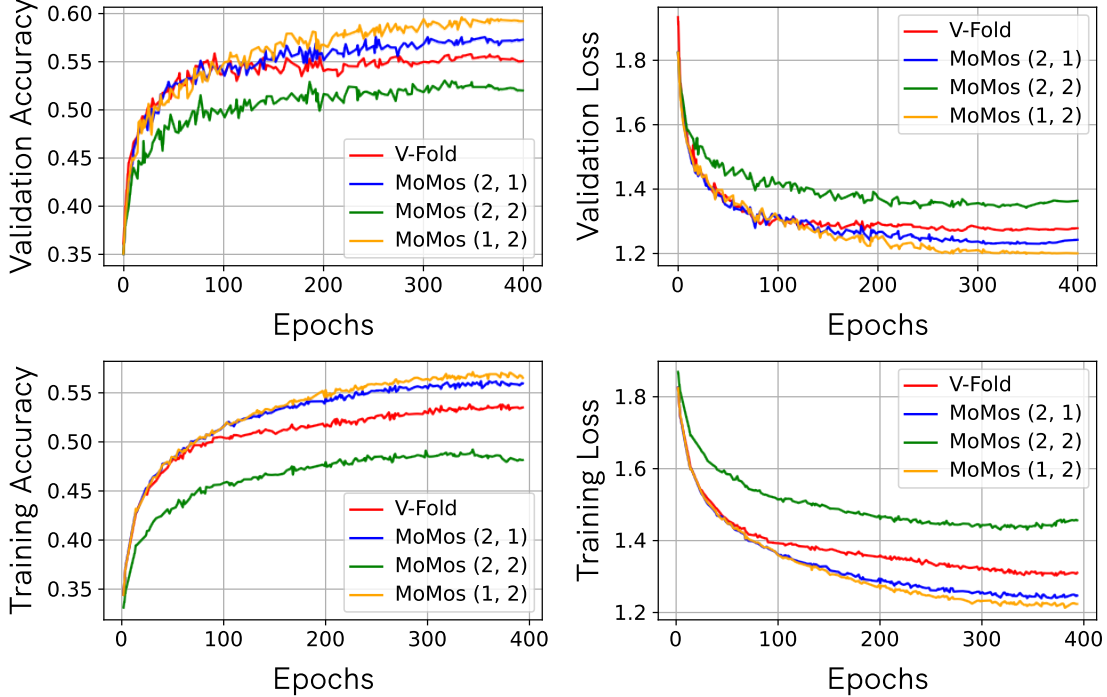
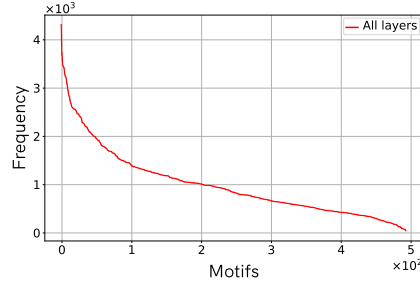


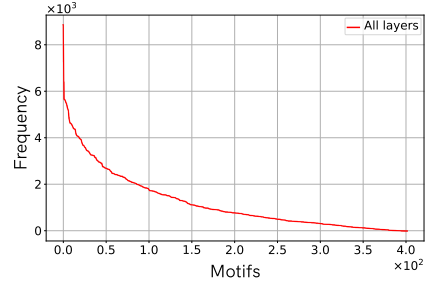
Figure 5.7: Training dynamics over 400 epochs on CIFAR-10 comparing MoMos configurations (1, 2), (2, 1), (2, 2) and V-Fold MoMos (2, 1) ($v = 128$, $c' = 1/4$). Top row: validation accuracy and validation loss. Bottom row: training accuracy and training loss.



(a) (1, 2), Epoch 1



(b) (1, 2), Epoch 200



(c) (1, 2), Epoch 380



(d) (2, 1), Epoch 1

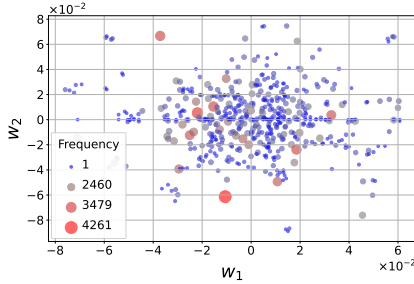


(e) (2, 1), Epoch 200

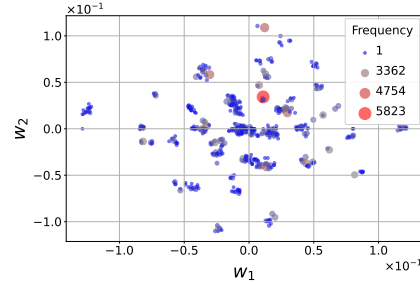


(f) (2, 1), Epoch 380

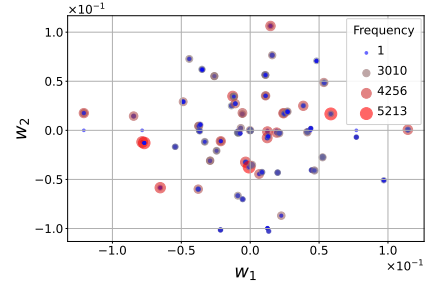
Figure 5.8: Motif assignment frequencies over 400 epochs for MoMos shape (1, 2) (top row) and V-Fold MoMos shape (2, 1) (bottom row). Highly skewed distributions emerge, with dominant motifs assigned over 3.5×10^4 times.



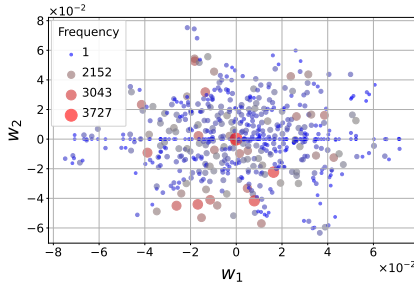
(a) (2, 1), Epoch 1



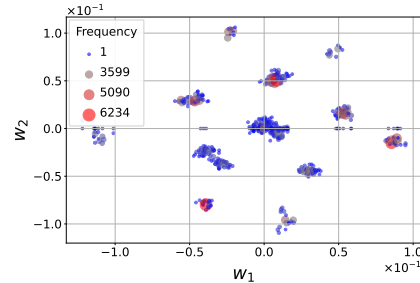
(b) (2, 1), Epoch 200



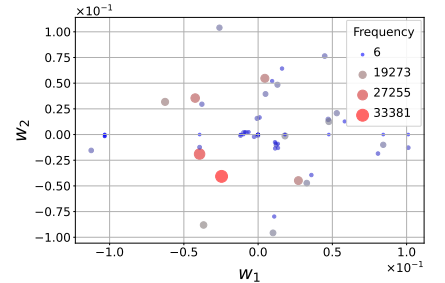
(c) (2, 1), Epoch 380



(d) (2, 1), Epoch 1



(e) (2, 1), Epoch 200



(f) (2, 1), Epoch 380

Figure 5.9: Scatter plots of motif coordinates in $\mathbb{R}^2$ across training epochs for MoMos (2, 1) (top row) and V-Fold MoMos (2, 1) (bottom row). Motifs progressively cluster into dense, overlapping coordinate regions by epoch 380.

Chapter 6

Toward Generalization Guarantees for MoMos

In the preceding chapters, we examined algorithmic complexity and developed the Mosaic-of-Motifs (Definition 4.1) and V-Fold MoMos (Definition 5.1) parameterizations to bound the description length of neural network parameter vectors. These methods constrain parameter representations during optimization, guaranteeing an explicit upper bound on algorithmic complexity throughout training. A central open theoretical question is how such algorithmic compression translates into generalization guarantees on unseen data.

This chapter investigates this connection across four sections:

1. Section 6.1 formalizes expected and empirical risk, defines the generalization gap, and establishes the countable-hypothesis PAC bound, showing that lower Kolmogorov complexity provably implies a tighter generalization gap (Wilson, 2025).
2. Section 6.2 analyzes the combinatorial and geometric expressivity of hypothesis spaces, defining the VC dimension, connecting it to hypothesis space cardinality, proving the exact VC dimension of linear models, and connecting the result to Cover’s function-counting theorem and separating capacity (Cover, 1965).
3. Section 6.3 analyzes the VC dimension of MoMos-parameterized linear models, proving the standard-basis shattering condition, establishing the union bound over mosaics, deriving codebook scaling trade-offs, and illustrating the preservation of VC dimension with a concrete plane classifier example.
4. Section 6.4 synthesizes the theoretical findings and connects these guarantees to the thesis conclusion.

6.1 Generalization and Algorithmic Complexity Bounds

Let $\mathcal{X} \subseteq \mathbb{R}^d$ denote the input space, $\mathcal{Y}$ the target label space, and $\mathcal{D}$ an unknown data-generating distribution over $\mathcal{X} \times \mathcal{Y}$. A learning algorithm receives a training sample $S = (z_1, \dots, z_p) \sim \mathcal{D}^p$, where each example $z_i = (x_i, y_i)$ is drawn independently and identically distributed (i.i.d.). Given a hypothesis class $\mathcal{H} \subseteq \mathcal{Y}^{\mathcal{X}}$, candidate hypotheses are evaluated through a loss function $\ell: \mathcal{Y} \times \mathcal{Y} \rightarrow [a, a + \Delta]$ with bounded range $\Delta > 0$. For binary classification with labels $\mathcal{Y} = \{\pm 1\}$, the standard zero-one loss $\ell(\hat{y}, y) = \mathbf{1}(\hat{y} \neq y)$ has range $\Delta = 1$.

To evaluate model performance on unseen data, we distinguish between the expected loss across the data-generating distribution and the average loss evaluated on training instances, also called the empirical loss (or empirical risk).

Definition 6.1 (Expected and Empirical Risk). Let $\mathcal{D}$ be a probability distribution over $\mathcal{X} \times \mathcal{Y}$, $S = ((x_i, y_i))_{i=1}^p \sim \mathcal{D}^p$, and $\ell: \mathcal{Y} \times \mathcal{Y} \rightarrow [a, a + \Delta]$ a loss function. For a hypothesis $h \in \mathcal{H}$, the expected risk is

$$R(h) = \mathbb{E}_{(x,y) \sim \mathcal{D}} [\ell(h(x), y)], \quad (6.1)$$

and the empirical risk (or empirical loss) evaluated on sample S is

$$\hat{R}_S(h) = \frac{1}{p} \sum_{i=1}^p \ell(h(x_i), y_i). \quad (6.2)$$

The generalization gap of hypothesis h is $\text{gen}(h) = R(h) - \hat{R}_S(h)$.

Classical uniform convergence bounds based on VC dimension or Rademacher complexity bound $\sup_{h \in \mathcal{H}} (R(h) - \hat{R}_S(h))$ uniformly over the entire hypothesis class $\mathcal{H}$, penalizing the total capacity of $\mathcal{H}$. In overparameterized models capable of interpolating arbitrary labels, such bounds become vacuous.

We report and adapt the countable-hypothesis generalization framework presented by [Shalev-Shwartz and Ben-David \(2014\)](#) and [Wilson \(2025\)](#). This bound shows that a hypothesis with lower algorithmic complexity is provably guaranteed a tighter generalization gap. When hypotheses are discrete or quantized to finite bit precision q , the hypothesis space $\mathcal{H}$ is countable. In this setting, we assign a prior probability distribution $P(h)$ over $\mathcal{H}$ before observing training data and apply Hoeffding's inequality with a non-uniform union bound.

To establish uniform generalization guarantees across a countable hypothesis space weighted by a prior, we bound the deviation between expected and empirical risk simultaneously for all hypotheses.

Theorem 6 (Countable Hypothesis Generalization Bound). Let $\mathcal{H}$ be a countable hypothesis class and let P be a probability distribution over $\mathcal{H}$ chosen independently of the training sample $S \sim \mathcal{D}^p$. For any loss function with bounded range $\Delta > 0$ and any confidence parameter $\delta \in (0, 1)$, with

probability at least $1 - \delta$ over the random draw of $S \sim \mathcal{D}^p$, every $h \in \mathcal{H}$ simultaneously satisfies

$$R(h) \leq \hat{R}_S(h) + \Delta \sqrt{\frac{\ln(1/P(h)) + \ln(1/\delta)}{2p}}. \quad (6.3)$$

Proof. Fix a single hypothesis $h \in \mathcal{H}$. The individual losses $\ell(h(x_i), y_i)$ are independent random variables taking values in $[a, a + \Delta]$ with expected value $\mathbb{E}[\ell(h(x_i), y_i)] = R(h)$. By Hoeffding's inequality, for any $\epsilon > 0$,

$$\mathbb{P} \left[R(h) - \hat{R}_S(h) > \epsilon \right] \leq \exp \left(-\frac{2p\epsilon^2}{\Delta^2} \right). \quad (6.4)$$

For each $h \in \mathcal{H}$, assign a target failure probability $\delta(h) = \delta P(h)$. Setting the right-hand side equal to $\delta(h)$ and solving for ϵ yields

$$\epsilon(h) = \Delta \sqrt{\frac{\ln(1/P(h)) + \ln(1/\delta)}{2p}}. \quad (6.5)$$

Applying the union bound over all hypotheses in the countable class $\mathcal{H}$:

$$\mathbb{P} \left[\exists h \in \mathcal{H} : R(h) - \hat{R}_S(h) > \epsilon(h) \right] \leq \sum_{h \in \mathcal{H}} \delta(h) = \delta \sum_{h \in \mathcal{H}} P(h) = \delta. \quad (6.6)$$

Taking the complement of this event establishes that $R(h) - \hat{R}_S(h) \leq \epsilon(h)$ holds simultaneously for all $h \in \mathcal{H}$ with probability at least $1 - \delta$. $\square$

Following [Wilson \(2025\)](#), we specialize [Theorem 6](#) by grounding the prior distribution $P(h)$ in algorithmic information theory. Drawing on universal algorithmic probability ([Definition 2.7](#)) and Kraft's inequality ([Li and Vitányi, 2008](#)), any prefix-free Kolmogorov complexity $\mathcal{K}(h)$ satisfies $\sum_{h \in \mathcal{H}} 2^{-\mathcal{K}(h)} \leq 1$. Defining the Solomonoff algorithmic prior $P(h) = 2^{-\mathcal{K}(h)}/Z$, where the normalization factor satisfies $Z = \sum_{h' \in \mathcal{H}} 2^{-\mathcal{K}(h')} \leq 1$, the negative log-prior evaluates to

$$\ln \left(\frac{1}{P(h)} \right) = \mathcal{K}(h) \ln 2 + \ln Z \leq \mathcal{K}(h) \ln 2. \quad (6.7)$$

Substituting this bound into [Theorem 6](#) yields the algorithmic generalization bound:

$$R(h) \leq \hat{R}_S(h) + \Delta \sqrt{\frac{\mathcal{K}(h) \ln 2 + \ln(1/\delta)}{2p}}. \quad (6.8)$$

[Equation 6.8](#) establishes that expected risk is strictly bounded by empirical risk plus model compressibility. Lower Kolmogorov complexity $\mathcal{K}(h)$ directly tightens the generalization gap bound.

Empirically, [Figure 5.5](#) shows that the approximation of algorithmic complexity decreases during training, while [5.4a](#) shows that the difference between training accuracy and validation accuracy decreases mildly, aligning with the theoretical prediction of [Equation 6.8](#).

6.2 Vapnik–Chervonenkis Dimension and Linear Models

While algorithmic information theory bounds generalization through individual hypothesis compressibility, classical learning theory evaluates the capacity of the entire hypothesis space $\mathcal{H}$. Introduced in statistical learning theory ([Shalev-Shwartz and Ben-David, 2014](#)), the Vapnik–Chervonenkis (VC) dimension provides a distribution-free measure of classification capacity by characterizing the combinatorial shattering ability of $\mathcal{H}$.

This section analyzes this combinatorial capacity for linear classification models. We review the relationship between VC dimension and hypothesis space cardinality, establish the exact VC dimension for homogeneous and affine linear models, and connect the result to Cover’s function-counting theorem and separating capacity ([Cover, 1965](#)).

6.2.1 VC Dimension and Hypothesis Space Cardinality

Following [Shalev-Shwartz and Ben-David \(2014\)](#), we formalize the capacity of a binary classifier family independently of data distributions by measuring the maximal sample size on which all possible binary labelings can be realized.

Definition 6.2 (Shattering and VC Dimension). Let $\mathcal{H} \subseteq \{\pm 1\}^{\mathcal{X}}$ be a hypothesis class of binary classifiers. A finite set of points $X = \{x_1, \dots, x_p\} \subseteq \mathcal{X}$ is shattered by $\mathcal{H}$ if for every binary labeling $\varepsilon \in \{\pm 1\}^p$, there exists $h \in \mathcal{H}$ such that $h(x_i) = \varepsilon_i$ for all $i \in [p]$. The Vapnik–Chervonenkis (VC) dimension of $\mathcal{H}$ is

$$\text{VCdim}(\mathcal{H}) = \sup \{|X| : X \subseteq \mathcal{X} \text{ is shattered by } \mathcal{H}\}. \quad (6.9)$$

If sets of arbitrarily large cardinality can be shattered, $\text{VCdim}(\mathcal{H}) = \infty$.

For a finite hypothesis class $\mathcal{H}$, shattering a set of size $d = \text{VCdim}(\mathcal{H})$ requires $\mathcal{H}$ to realize all 2^d distinct dichotomies on those d points. Because the number of distinct dichotomies realized on any sample cannot exceed the total number of hypotheses $|\mathcal{H}|$, we obtain the direct cardinality relationship:

$$2^d \leq |\mathcal{H}| \implies \text{VCdim}(\mathcal{H}) \leq \log_2 |\mathcal{H}|. \quad (6.10)$$

The growth function $\Pi_{\mathcal{H}}(p) = \max_{x_1, \dots, x_p} |\{(h(x_1), \dots, h(x_p)) : h \in \mathcal{H}\}|$ tracks the maximum number of dichotomies realized on p points. In the framework of partitioning machines ([Leboeuf et al., 2020](#)), a hypothesis class shatters p points if and only if its 2-partitioning function achieves the maximum Stirling number of the second kind, $\pi_{\mathcal{H}}^2(p) = \left\{ \begin{smallmatrix} p \\ 2 \end{smallmatrix} \right\} = 2^{p-1} - 1$. If geometric or structural

constraints restrict $\pi_{\mathcal{H}}^2(p) < \binom{p}{2}$, the class fails to achieve full partitioning capacity, bounding its VC dimension strictly below p .

6.2.2 VC Dimension of Linear Models

Linear threshold classifiers compute binary labels via the sign of an inner product between a weight vector and an input vector.

Definition 6.3 (Linear Classifier). For a parameter vector $\mathbf{w} \in \mathbb{R}^n$, the homogeneous linear classifier is $h_{\mathbf{w}}(x) = \text{sign}(\langle \mathbf{w}, x \rangle)$, with hypothesis class $\mathcal{H}_{\text{lin}} = \{x \mapsto \text{sign}(\langle \mathbf{w}, x \rangle) : \mathbf{w} \in \mathbb{R}^n\}$. An affine linear classifier on $\mathbb{R}^\ell$ is $x \mapsto \text{sign}(\langle \mathbf{w}, x \rangle + b)$ with $\mathbf{w} \in \mathbb{R}^\ell$ and $b \in \mathbb{R}$, equivalent to $\mathcal{H}_{\text{lin}}$ on $\mathbb{R}^{\ell+1}$ under the coordinate lift $x \mapsto (x, 1)$.

To determine the capacity of linear classifiers, we prove their exact VC dimension.

Theorem 7 (VC Dimension of Linear Models). The VC dimension of the homogeneous linear hypothesis class $\mathcal{H}_{\text{lin}}$ in $\mathbb{R}^n$ is $\text{VCdim}(\mathcal{H}_{\text{lin}}) = n$. Consequently, affine linear classifiers in $\mathbb{R}^\ell$ have VC dimension $\ell + 1$.

Proof. We establish the equality by proving matching lower and upper bounds.

(Lower bound: $\text{VCdim}(\mathcal{H}_{\text{lin}}) \geq n$). Consider the n standard basis vectors $e_1, \dots, e_n \in \mathbb{R}^n$. Given any arbitrary target labeling $\varepsilon = (\varepsilon_1, \dots, \varepsilon_n) \in \{\pm 1\}^n$, define the weight vector $\mathbf{w} = \sum_{j=1}^n \varepsilon_j e_j$. For each $i \in [n]$, evaluating the classifier yields $\langle \mathbf{w}, e_i \rangle = \varepsilon_i$, and therefore $\text{sign}(\langle \mathbf{w}, e_i \rangle) = \text{sign}(\varepsilon_i) = \varepsilon_i$. Since all 2^n labelings are realized on $\{e_1, \dots, e_n\}$, the set is shattered, showing $\text{VCdim}(\mathcal{H}_{\text{lin}}) \geq n$.

(Upper bound: $\text{VCdim}(\mathcal{H}_{\text{lin}}) \leq n$). Let $x_1, \dots, x_{n+1} \in \mathbb{R}^n$ be an arbitrary set of $n + 1$ points. Because any $n + 1$ vectors in $\mathbb{R}^n$ are linearly dependent, there exist real coefficients $\alpha_1, \dots, \alpha_{n+1} \in \mathbb{R}$, not all zero, such that

$$\sum_{i=1}^{n+1} \alpha_i x_i = 0. \quad (6.11)$$

Partition the index set $[n + 1]$ into positive and non-positive index subsets: $I = \{i : \alpha_i > 0\}$ and $J = \{j : \alpha_j \leq 0\}$. Since $(-\alpha_i)$ is an equally valid dependence, replace α by $-\alpha$ if necessary so that I is non-empty (possible because the coefficients are not all zero). Rearranging the linear dependence gives

$$\sum_{i \in I} \alpha_i x_i = \sum_{j \in J} (-\alpha_j) x_j, \quad (6.12)$$

where $\alpha_i > 0$ for all $i \in I$ and $-\alpha_j \geq 0$ for all $j \in J$.

Suppose for contradiction that $\mathcal{H}_{\text{lin}}$ shatters $\{x_1, \dots, x_{n+1}\}$. Then there exists a weight vector $\mathbf{w} \in \mathbb{R}^n$ realizing the labeling $\varepsilon_i = +1$ for $i \in I$ and $\varepsilon_j = -1$ for $j \in J$. This requires $\langle \mathbf{w}, x_i \rangle > 0$

for all $i \in I$ and $\langle \mathbf{w}, x_j \rangle < 0$ for all $j \in J$. Taking the inner product with $\mathbf{w}$ on both sides yields

$$\sum_{i \in I} \alpha_i \langle \mathbf{w}, x_i \rangle = \sum_{j \in J} (-\alpha_j) \langle \mathbf{w}, x_j \rangle. \quad (6.13)$$

The left-hand side is strictly positive because $I \neq \emptyset$, $\alpha_i > 0$, and $\langle \mathbf{w}, x_i \rangle > 0$ for every $i \in I$. The right-hand side is non-positive because $-\alpha_j \geq 0$ and $\langle \mathbf{w}, x_j \rangle < 0$ for every $j \in J$. This contradiction proves that no linear classifier can separate I from J . Thus no set of $n+1$ points in $\mathbb{R}^n$ can be shattered, establishing $\text{VCdim}(\mathcal{H}_{\text{lin}}) \leq n$. By the coordinate lift of [Definition 6.3](#), the upper bound $\text{VCdim}(\mathcal{H}_{\text{lin}}) \leq n$ restricts immediately to the affine domain, giving $\text{VCdim} \leq \ell + 1$. For the lower bound, the $\ell + 1$ points $0, e_1, \dots, e_\ell \in \mathbb{R}^\ell$ are shattered: given a target labeling $\varepsilon_0, \varepsilon_1, \dots, \varepsilon_\ell \in \{\pm 1\}$, taking $b = \varepsilon_0$ and $w_i = \varepsilon_0$ if $\varepsilon_i = \varepsilon_0$, else $w_i = -3\varepsilon_0$, realizes $\text{sign}(b) = \varepsilon_0$ and $\text{sign}(w_i + b) = \varepsilon_i$ for each $i \in [\ell]$. Hence affine linear classifiers in $\mathbb{R}^\ell$ have VC dimension exactly $\ell + 1$. $\square$

[Cover \(1965\)](#) provides an exact combinatorial enumeration of realizable dichotomies for linear models. For N points in general position in $\mathbb{R}^n$, the number of homogeneously linearly separable dichotomies is

$$C(N, n) = 2 \sum_{k=0}^{n-1} \binom{N-1}{k}. \quad (6.14)$$

When $N \leq n$, the binomial sum spans the complete binomial expansion 2^{N-1} , yielding $C(N, n) = 2^N$ (exact shattering). When $N > n$, $C(N, n) < 2^N$, and the fraction of realizable dichotomies strictly decreases.

[Cover \(1965\)](#) defines the separating capacity of a family of decision surfaces with n degrees of freedom as $N = 2n$ pattern vectors. Under random binary labelings assigned by fair coin tosses, the probability of linear separability $P(N, n) = 2^{-(N-1)} \sum_{k=0}^{n-1} \binom{N-1}{k}$ undergoes a sharp phase transition at $N = 2n$, where $P(2n, n) = 1/2$. As $n \rightarrow \infty$, $P(N, n) \rightarrow 1$ for $N < 2n(1 - \epsilon)$ and $P(N, n) \rightarrow 0$ for $N > 2n(1 + \epsilon)$. The separating capacity is therefore twice the VC dimension, defining the sample size threshold where linear memorization breaks down.

6.3 MoMos-Parameterized Linear Models

We now analyze the expressivity of linear models constrained by the Mosaic-of-Motifs parameterization ([Definition 4.1](#)).

Definition 6.4 (MoMos-Parameterized Linear Model). Let $s \in \mathbb{N}$ be the block size, $m = n/s$ the block count with $s \mid n$, and $k \in \mathbb{N}$ the motif count. The MoMos linear hypothesis class is

$$\mathcal{H}_{s,k} = \{x \mapsto \text{sign}(\langle \hat{\mathbf{w}}, x \rangle) : \hat{\mathbf{w}} = \psi^{-1}(\mathcal{Z}[\mathcal{M}]), \mathcal{Z} \in \mathbb{R}^{s \times k}, \mathcal{M} \in [k]^m\}. \quad (6.15)$$

Next, we determine the exact condition under which $\mathcal{H}_{s,k}$ shatters the n standard basis vectors, preserving the VC dimension of the unconstrained linear model.

Theorem 8 (Standard Basis Shattering for MoMos). The n standard-basis inputs $e_1, \dots, e_n \in \mathbb{R}^n$ are shattered by $\mathcal{H}_{s,k}$ if and only if $\min(m, 2^s) \leq k$.

Proof. ($\Leftarrow$) Fix any binary labeling $\varepsilon \in \{\pm 1\}^n$. Partition ε into m consecutive blocks of size s , $\varepsilon = (\varepsilon_1, \dots, \varepsilon_m)$ with each $\varepsilon_b \in \{\pm 1\}^s$. For any standard basis vector e_i belonging to block b , evaluating the classifier yields $\langle \hat{\mathbf{w}}, e_i \rangle = \hat{\mathbf{w}}_{b,j}$, the j -th entry of the reconstructed block $\hat{\mathbf{w}}_b$. Thus, block b realizes the target signs if its reconstructed motif satisfies $\text{sign}(\hat{\mathbf{w}}_b) = \varepsilon_b$. The total number of distinct sign patterns across the m blocks is at most $\min(m, 2^s)$. Because $k \geq \min(m, 2^s)$, we can assign a distinct motif vector $\mathbf{z}_j \in \mathbb{R}^s$ to each unique required sign pattern such that $\text{sign}(\mathbf{z}_j) = \varepsilon_b$, setting $\mathcal{M}(b) = j$. The reconstructed weight vector $\hat{\mathbf{w}} = \psi^{-1}(\mathcal{Z}[\mathcal{M}])$ satisfies $\text{sign}(\langle \hat{\mathbf{w}}, e_i \rangle) = \varepsilon_i$ for all $i \in [n]$.

($\Rightarrow$) By contraposition, assume $k < m$ and $k < 2^s$. We construct a labeling ε that cannot be realized by $\mathcal{H}_{s,k}$. Because $k + 1 \leq m$ and $k + 1 \leq 2^s$, we can select a labeling $\varepsilon \in \{\pm 1\}^n$ such that the first $k + 1$ blocks $\varepsilon_1, \dots, \varepsilon_{k+1}$ exhibit $k + 1$ pairwise distinct sign patterns in $\{\pm 1\}^s$. For any valid reconstruction $\hat{\mathbf{w}}_b = \mathbf{z}_{\mathcal{M}(b)}$, matching the signs on the standard basis vectors requires $\text{sign}(\mathbf{z}_{\mathcal{M}(b)}) = \varepsilon_b$. Because the first $k + 1$ sign patterns are pairwise distinct, the assigned motifs $\mathbf{z}_{\mathcal{M}(1)}, \dots, \mathbf{z}_{\mathcal{M}(k+1)}$ must be pairwise distinct. Consequently, the mosaic index map $\mathcal{M}$ must be injective when restricted to the first $k + 1$ block indices $[k + 1]$. This defines an injective map from a set of size $k + 1$ into the motif index set $[k]$ of size k , which is impossible by the pigeonhole principle. Hence $\mathcal{H}_{s,k}$ cannot shatter $\{e_1, \dots, e_n\}$. $\square$

To bound the general VC dimension of $\mathcal{H}_{s,k}$ across arbitrary input sets, we decompose the hypothesis class into a union over all possible mosaic index assignments:

$$\mathcal{H}_{s,k} = \bigcup_{\mathcal{M} \in [k]^m} \mathcal{H}_{\mathcal{M}}, \quad (6.16)$$

where $\mathcal{H}_{\mathcal{M}} = \{x \mapsto \text{sign}(\langle \psi^{-1}(\mathcal{Z}[\mathcal{M}]), x \rangle) : \mathcal{Z} \in \mathbb{R}^{s \times k}\}$. For a fixed mosaic $\mathcal{M}$, each reconstructed block $\hat{\mathbf{w}}_b$ is drawn from the k motif columns of $\mathcal{Z}$. The reconstructed weight vector $\hat{\mathbf{w}}$ is therefore confined to a linear subspace of $\mathbb{R}^n$ of dimension at most $d^* = ks$, representing the total number of free continuous parameters in $\mathcal{Z}$.

By [Theorem 7](#), the VC dimension of each subspace class is $\text{VCdim}(\mathcal{H}_{\mathcal{M}}) \leq ks$. The total number of distinct mosaics is k^m . Applying the finite-union bound for VC classes ([Shalev-Shwartz and Ben-David, 2014](#); [Leboeuf et al., 2020](#)), the VC dimension of the union satisfies

$$\text{VCdim}(\mathcal{H}_{s,k}) \leq \mathcal{O}(ks + \ln(k^m)) = \mathcal{O}(ks + m \ln k). \quad (6.17)$$

[Equation 6.17](#) highlights the structural role of the mosaic and motif components:

1. **Mosaic Term and Cardinality:** The term $m \ln k = m \log_2 k \ln 2$ reflects the combinatorial cardinality of the mosaic assignment space, $|\{\mathcal{M}\}| = k^m$. In [Equation 6.10](#), the upper bound on VC dimension is governed by the logarithm of hypothesis space cardinality. In MoMos, the mosaic term $m \log_2 k$ tracks this cardinality bound and corresponds, up to the integer ceiling, to the mosaic description length $m \lceil \log_2 k \rceil$ in the Kolmogorov complexity bound ([Definition 4.1](#)). When parameterized on a logarithmic scale with $k = 2^\kappa$ ($\kappa \in \mathbb{N}$), this term becomes exactly $m\kappa$.
2. **Motif Degrees of Freedom:** The term ks accounts for the continuous geometric degrees of freedom provided by the motif matrix $\mathcal{Z} \in \mathbb{R}^{s \times k}$.
3. **Connection to Cover’s Separating Capacity:** In the framework of [Cover \(1965\)](#), imposing parameter redundancy restricts the reconstructed vector $\mathbf{w}$ to a lower-dimensional subspace. For a fixed mosaic $\mathcal{M}$, the model possesses at most ks degrees of freedom, yielding a baseline separating capacity of $2ks$ pattern vectors. The mosaic $\mathcal{M}$ acts as a discrete routing mechanism that selects among k^m possible subspace orientations. Cover’s Theorem 3 establishes that coordinate constraints reduce linear degrees of freedom; under MoMos, the separating capacity is determined by the continuous motif capacity $2ks$ operating in tandem with the combinatorial switching capacity $m \log_2 k$ of the mosaic.
4. **Block Size, Motif Scaling, and Description Length:** Fixing the mosaic storage budget $m \lceil \log_2 k \rceil$, the VC dimension identifies the minimal mosaic size required to preserve full expressivity while the block size s can vary. Minimizing the Kolmogorov complexity bound under this condition yields the minimal block size $s = 1$, requiring only $k = 2$ motifs to shatter the basis. Because the block size s is the exponent controlling the minimum number of motifs ($k \sim 2^s$), increasing s requires the motif count to scale exponentially (k^v when grouping v blocks) to preserve the original hypothesis space. This matches the recursive analysis in [Section 5.4](#), showing why inflating block size without expanding the codebook restricts network expressivity.

This trade-off explains the empirical Pareto frontiers in [Chapter 4 \(Figure 4.2\)](#). Under severe motif reduction (high compression ratios), models with larger block sizes ($s = 4, 8$) achieve higher test accuracy than small-block parameterizations ($s = 1, 2$) with heavily depleted codebooks. Although basis shattering remains theoretically possible, the continuous degrees of freedom ks within each active subspace become the primary performance bottleneck. Larger block sizes maintain a larger continuous parameter volume ks for a given overall description length, preserving continuous model capacity and sustaining higher predictive performance at high compression rates.

Example: Plane Linear Classifier under MoMos. To illustrate how the union over mosaics preserves the VC dimension of the unconstrained model while restricting parameter configurations, consider an affine linear classifier in $\mathbb{R}^2$ with parameter vector $(a, b, c) \in \mathbb{R}^3$ defining the line $ax + by + c = 0$. The unconstrained model has $n = 3$ and $\text{VCdim} = 3$.

Applying MoMos with block size $s = 1$ and $k = 2$ motifs $z_1, z_2 \in \mathbb{R}$, the parameter vector is divided into $m = 3$ blocks of size 1. Identifying mosaic assignments $\mathcal{M} \in [2]^3$ up to relabeling of the two motifs yields three non-degenerate tied families of lines:

$$\mathcal{H}_{\text{MoMos}(s=1, k=2)} = \{ax + ay + b = 0\} \cup \{ax + by + a = 0\} \cup \{ax + by + b = 0\}. \quad (6.18)$$

Individually, each family has $ks = 2$ degrees of freedom (Equation 6.17) and, as an affine threshold on a single coordinate, cannot shatter three points. However, their union shatters the three non-collinear points $P_1 = (0, 1)$, $P_2 = (4, 2)$, and $P_3 = (1, 0)$.

Grouping coefficients, the three families define the decision functions:

$$F_1(x, y; a, b) = \text{sign}(a(x + y) + b), \quad (6.19)$$

$$F_2(x, y; a, b) = \text{sign}(a(x + 1) + by), \quad (6.20)$$

$$F_3(x, y; a, b) = \text{sign}(ax + b(y + 1)). \quad (6.21)$$

Negating parameters $(a, b) \mapsto (-a, -b)$ directly inverts the predicted sign on every input, since $F_i(x, y; -a, -b) = -F_i(x, y; a, b)$ for each $i \in \{1, 2, 3\}$. Consequently, whenever a parameter choice (a, b) realizes a target labeling $\varepsilon \in \{\pm 1\}^3$, the negated vector $(-a, -b)$ within the same family realizes the complementary labeling $-\varepsilon$. It therefore suffices to realize four representative dichotomies (the all-positive labeling and the three singletons):

Target	Line $\ell(x, y) = 0$	$\ell(P_1)$	$\ell(P_2)$	$\ell(P_3)$
All+	$F_1 : x + y = 0$	+1	+6	+1
$\{P_1\} +$	$F_3 : -2x + y + 1 = 0$	+2	-5	-1
$\{P_2\} +$	$F_1 : 2x + 2y - 7 = 0$	-5	+5	-5
$\{P_3\} +$	$F_2 : x - 3y + 1 = 0$	-2	-1	+2

These four lines, together with their sign-flipped complements, realize all eight binary dichotomies on $\{P_1, P_2, P_3\}$ (Figure 6.1). Hence $\text{VCdim} = 3$ is preserved, showing that the union over mosaics maintains the unconstrained VC dimension while enforcing structural block sharing.

6.4 Chapter Summary

This chapter established theoretical foundations connecting algorithmic complexity bounds with statistical generalization:

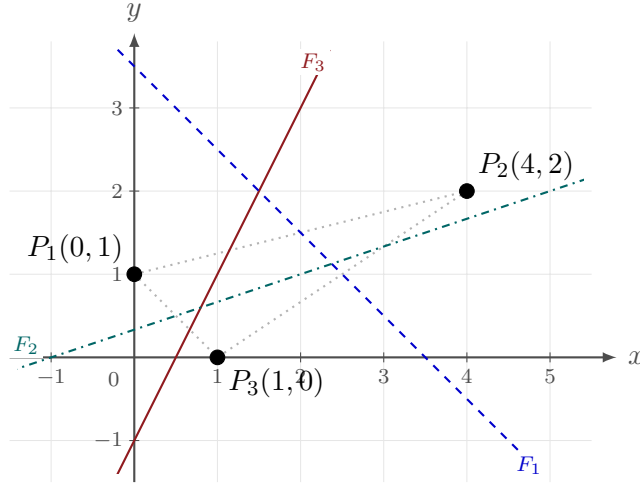


Figure 6.1: Separation of three non-collinear points $P_1 = (0, 1)$, $P_2 = (4, 2)$, and $P_3 = (1, 0)$ in $\mathbb{R}^2$ by three lines drawn from the tied MoMos families F_1 , F_2 , and F_3 : $F_3 : -2x + y + 1 = 0$ isolates P_1 , $F_1 : 2x + 2y - 7 = 0$ isolates P_2 , and $F_2 : x - 3y + 1 = 0$ isolates P_3 . Each line assigns a positive sign to its target point and negative signs to the remaining pair, realizing all singleton dichotomies and confirming that $\text{VCdim} = 3$ is preserved under MoMos parameter tying.

1. **Countable PAC Generalization Bounds:** Derived from Kraft’s inequality and the Solomonoff algorithmic prior, [Equation 6.8](#) guarantees that minimizing the Kolmogorov complexity $\mathcal{K}(\mathbf{w})$ of quantized neural network parameterizations directly tightens the expected generalization gap.
2. **Combinatorial Capacity Preservation:** [Theorem 8](#) and [Equation 6.17](#) prove that MoMos preserves the unconstrained VC dimension $d_{\text{VC}} = n$ across linear models whenever $k \geq \min(m, 2^s)$, showing that parameter tying does not degrade worst-case combinatorial expressivity.
3. **Block Size and Motif Scaling:** Minimizing the Kolmogorov complexity bound under a fixed mosaic storage budget yields $s = 1$ with only $k = 2$ motifs, since larger block sizes require exponentially more motifs to preserve the hypothesis space ([Section 6.3](#)).

Chapter 7

Conclusion and Future Directions

This thesis investigated the intersection of algorithmic information theory (AIT) and deep neural network simplification, establishing theoretical and empirical frameworks to bound the Kolmogorov complexity of neural network parameters during training. Traditional network compression paradigms, such as Soft Weight-Sharing (Ullrich et al., 2017), Deep Compression (Han et al., 2016), and Hyper-Compression (Fan et al., 2026), operate primarily as post-training interventions. In contrast, this work focused on in-training structural constraints that guarantee explicit upper bounds on description length throughout gradient optimization.

7.1 Summary of Contributions

The core contributions of this work are structured across five chapters:

1. **Foundations of Algorithmic Information Theory (Chapter 2)**: We reviewed the Universal Turing Machine, formalized Kolmogorov complexity $\mathcal{K}(x)$, and analyzed computable approximations including the Coding Theorem Method ($\hat{\mathcal{K}}_{\text{CTM}}$), the Block Decomposition Method ($\hat{\mathcal{K}}_{\text{BDM}}$), and the Quantized Block Decomposition Method ($\hat{\mathcal{K}}_{\text{QuBD}}$). We identified $\hat{\mathcal{K}}_{\text{QuBD}}$ as an effective proxy for tracking algorithmic complexity across multi-bit continuous weight configurations.
2. **Weight-Sharing Paradigms (Chapter 3)**: We categorized existing post-training weight-sharing methods, demonstrating how discrete value clustering, Gaussian mixture priors, and functional generators reduce description length after standard convergence.
3. **Mosaic-of-Motifs Characterization (Chapter 4)**: We formalized the one-dimensional and two-dimensional Mosaic-of-Motifs (MoMos) parameterizations (Bakhtiarifard et al., 2026a). We derived explicit description-length upper bounds $\hat{\mathcal{K}}_{\text{MoMos}}(\mathbf{w})$ and evaluated capacity scaling, symmetry breaking, and implicit soft regularization across CNN and MLP architectures.

4. **V-Fold MoMos Framework (Chapter 5):** We addressed the storage bottleneck of standard MoMos by introducing V-Fold MoMos. By partitioning parameter tensors into v localized folds, V-Fold MoMos restricts each position to a sub-codebook of $k' \leq k$ motifs. We proved description bounds, designed projection algorithms, and demonstrated improved Pareto-optimal trade-offs between compression ratio and predictive accuracy on CIFAR-10.
5. **Generalization and Expressivity Bounds (Chapter 6):** We established the connection between Kolmogorov complexity and generalization through countable-hypothesis PAC bounds (Wilson, 2025). We then analyzed the combinatorial expressivity of MoMos-parameterized linear models, proving that MoMos preserves the full unconstrained VC dimension $d_{VC} = n$ across canonical standard-basis configurations.

7.2 Key Insights

Our theoretical analysis and empirical evaluations yield four principal insights regarding algorithmic complexity and parameter tying in neural networks:

The Mosaic as the Primary Compression Bottleneck In standard MoMos, the parameter vector is decomposed into a shared motif codebook $\mathcal{Z} \in \mathbb{R}^{s \times k}$ and an index mosaic $\mathcal{M} \in [k]^m$. As the parameter count n grows while block size s remains fixed, the description length of the mosaic $m \lceil \log_2 k \rceil$ dominates the codebook storage ksq . The mosaic is essential for expressivity and preserves the full VC dimension of linear models; at the same time, it constitutes the asymptotic storage bottleneck. V-Fold MoMos directly alleviates this limitation by subdividing block assignments into localized codebooks, reducing mosaic storage from $\lceil \log_2 k \rceil$ to $\lceil \log_2 k' \rceil$ bits per entry.

Multi-Scale Structure and Decomposability Large neural networks exhibit intrinsic multi-scale structure and regularities. Theoretical frameworks such as Homogenized Transformers (Koubbi et al., 2026) study the activation dynamics of randomly initialized transformers using homogenization limits, demonstrating that deep networks display macroscopic, multi-scale particle dynamics across layers. In algorithmic information theory, an object is compressible when it decomposes into recurring sub-patterns. Gradient optimization under MoMos and V-Fold MoMos exploits this decomposability, guiding parameters along constrained trajectories that converge to low-complexity configurations without sacrificing predictive accuracy. Unlike post-training heuristics that search for nearby quantized weights within an ϵ -ball of an unconstrained minimum, in-training structural bounding alters the entire optimization path.

Soft Regularization and Optimization Dynamics MoMos parameterization can be formulated as a soft inductive bias (Wilson, 2025), replacing standard L_2 weight regularization with a distortion penalty: the L_2 norm of the difference between the parameter vector obtained after backpropagation and its discrete MoMos projection. Because the zero motif is always included in the codebook, this distortion is guaranteed to add no more perturbation than standard L_2 regularization. Convergence under standard gradient optimization (SGD, GD, or Adam) remains guaranteed under the conditions established in statistical learning theory (Shalev-Shwartz and Ben-David, 2014). To mitigate the computational cost of computing full projections at every optimization step, motifs and mosaics can be extracted at the start of each epoch, computing the distortion penalty with respect to that fixed mosaic across mini-batches. Over extended training epochs, this dynamic causes motifs to cluster and coalesce into redundant coordinate locations, shrinking the effective hypothesis space and enabling substantial post-training consolidation. This in-training regularization operates in fundamental contrast to post-training retraining routines such as Deep Compression (Han et al., 2016), which only apply pruning and quantization after convergence.

Discrepancy Between Combinatorial Capacity and Algorithmic Bounds While countable PAC bounds rigorously show that bounding Kolmogorov complexity $\mathcal{K}(\mathbf{w})$ tightens generalization guarantees, worst-case combinatorial measures such as the VC dimension do not contract under MoMos parameter tying. Because the hypothesis space is defined as the union over all possible discrete mosaic configurations, the model retains the ability to shatter n standard-basis points. This discrepancy confirms that classical worst-case combinatorial capacities are too coarse to reflect the generalization benefits of parameter sharing, highlighting the need for distribution-dependent complexity measures.

7.3 Practical Implications

The methods developed in this thesis offer direct practical utility for large-scale and resource-constrained deep learning:

Bandwidth-Efficient Distributed Training Training modern frontier architectures containing billions or trillions of parameters requires distributed model parallelism across accelerator clusters. Inter-device communication of gradients and weights constitutes a primary scaling bottleneck. MoMos and V-Fold MoMos naturally decouple the global continuous codebook $\mathcal{Z}$ from local discrete mosaics $\mathcal{M}$. In a distributed training regime, worker nodes can synchronize only the compact motif matrix $\mathcal{Z}$ across iterations, preventing codebook collapse while keeping the voluminous mosaic indices local. This decoupling has the potential to reduce inter-node communication traffic by several orders of magnitude.

Democratized Edge Inference via NVMe Streaming Emerging inference frameworks such as Colibri (Fornaro, 2026) and DS4 (Sanfilippo, 2026) democratize large model execution on commodity hardware by storing parameters on NVMe solid-state drives (SSDs) and streaming individual layers into RAM or GPU memory on demand. Memory bandwidth and I/O throughput remain the critical latency drivers. Under MoMos and V-Fold MoMos, small motif codebooks can reside permanently within fast on-chip SRAM or system RAM, requiring only low-bit mosaic indices to be fetched from disk. This architecture minimizes memory transfer overhead and substantially reduces energy consumption.

7.4 Limitations

Several limitations of the current study point to areas for refinement:

1. **Fixed-Length Index Accounting:** Our theoretical description bounds assume uniform, fixed-length binary encoding for mosaic indices. In practice, motif selection frequencies are highly non-uniform. Omitting entropy coding (e.g., Huffman encoding (Huffman, 1952)) leaves additional compressibility unexploited in our reported compression metrics.
2. **Worst-Case Generalization Bounds:** The VC dimension analysis in Chapter 6 demonstrates that the combinatorial capacity of linear models is preserved under MoMos, but it provides loose generalization bounds on real-world data distributions.
3. **Empirical Scope and Architectural Diversity:** Our empirical evaluation focused on multi-layer perceptrons and ResNet-20 on CIFAR-10. Validating the scalability of V-Fold MoMos on large vision transformers and autoregressive language models remains to be completed.
4. **Activation Function Specificity:** Experimental evaluations in this thesis were conducted using ReLU activations. Because the ReLU function is piecewise linear and positively homogeneous, it exhibits scale symmetries that interact distinctively with parameter tying and motif sharing. Evaluating activations without positive homogeneity is necessary to determine whether these empirical clustering dynamics generalize across arbitrary non-linearities.
5. **Absence of Vector Quantization Benchmarking:** Although MoMos shares structural principles with vector quantization (VQ), such as partitioning weight vectors into blocks mapped to a discrete codebook of learned prototypes, we did not investigate or benchmark against the prolific vector quantization literature. Classical and modern VQ paradigms, including Product Quantization, Residual Vector Quantization, and learned codebook optimization, pursue closely related compression objectives. Evaluating how MoMos compares

with or incorporates mature VQ techniques for codebook initialization, clustering, and rate-distortion trade-offs remains an unaddressed limitation.

7.5 Future Research Directions

Building on the results of this thesis, several promising directions present natural avenues for future research:

Fully Recursive and Hierarchical Codebooks V-Fold MoMos provides a two-level codebook hierarchy. A natural theoretical extension is a fully recursive MoMos formulation, where motif vectors are themselves represented as mosaics over smaller sub-motifs. Formulating a stable optimization algorithm for multi-tier recursive codebooks could enable logarithmic description scaling aligned with the scaling laws of modern neural network innovation.

Functional and Continuous Motif Parameterization Rather than storing explicit discrete coordinate arrays, motif codebooks could be generated by compact continuous coordinate networks or hyper-networks, connecting MoMos with Hyper-Compression (Fan et al., 2026). Functional generation would allow continuous interpolation across motif spaces and eliminate representation symmetries associated with arbitrary codebook indexing.

Downstream Entropy Coding and Post-Training Consolidation Integrating adaptive Huffman coding (Huffman, 1952) directly into the MoMos training pipeline would bridge the gap between nominal description bounds and empirical compressed model storage. Post-training k-means clustering can merge collapsed motifs, further compressing the model before deployment.

Distributed Codebook Synchronization Protocols Future work should empirically implement and benchmark the proposed codebook-only distributed training algorithm on multi-GPU and multi-node clusters, measuring exact communication bandwidth reductions, synchronization latencies, and final task accuracies.

Characterizing Subspaces of Optimal Solutions Comparing parameter configurations across multiple converged models raises the fundamental question of whether optimal solutions share a common low-dimensional manifold. Under the MoMos parameterization, the parameter space is explicitly factorized into a discrete mosaic index map $\mathcal{M}$ and continuous motif coordinates $\mathcal{Z}$, defining a family of linear subspaces $\mathcal{H}_{\mathcal{M}}$. This structured parameterization offers a tractable framework to identify and characterize the geometric subspace occupied by optimal solutions. In particular, unconstrained neural network solutions may be contained within or lie close to the

subspace spanned by converged MoMos representations, providing a geometric lens on neural network mode connectivity and solution equivalence.

Importance-Guided Motif Sampling and Collapse Mitigation During extended training, both MoMos and V-Fold MoMos exhibit codebook collapse, where motifs coalesce into redundant coordinate clusters and shrink the occupied hypothesis class far below the nominal bound $\mathcal{H}_{s,k}$. Actively mitigating collapse through informed sampling criteria rather than uniform or frequency-based selection is critical to: (i) apply MoMos effectively to architectures that appear less inherently compressible, such as convolutional networks, and (ii) stabilize deeper recursive codebook hierarchies without premature capacity degradation. First, weighting candidate blocks by the L_2 norm of their forward activations preserves mature parameters that have converged to stable, highly predictive features and no longer produce large gradient errors. Second, sampling blocks proportionally to the L_2 norm of their backpropagated gradients retains blocks carrying critical directional signals during active learning, while down-weighting inactive blocks whose gradients remain near zero. Finally, combining activation and gradient norms with running momentum and variance buffers from optimizers such as SGD with momentum or Adam offers a unified, dynamics-aware sampling policy to sustain codebook diversity throughout optimization.

Bibliography

- Pedram Bakhtiarifard, Pınar Tözün, Christian Igel, and Raghavendra Selvan. Climate And Resource Awareness is Imperative to Achieving Sustainable AI (and Preventing a Global AI Arms Race), February 2025. URL <http://arxiv.org/abs/2502.20016>. arXiv:2502.20016 [cs.LG].
- Pedram Bakhtiarifard, Tong Chen, Jonathan Wenshøj, Erik B. Dam, and Raghavendra Selvan. Algorithmic Simplification of Neural Networks with Mosaic-of-Motifs, May 2026a. URL <http://arxiv.org/abs/2602.14896>. arXiv:2602.14896 [cs.LG].
- Pedram Bakhtiarifard, Sophia N. Wilson, Mahmoud Afifi, Jonathan Wenshøj, and Raghavendra Selvan. Characterizing Learning in Deep Neural Networks using Tractable Algorithmic Complexity Analysis, May 2026b. URL <http://arxiv.org/abs/2605.15551>. arXiv:2605.15551 [cs.LG].
- Thomas M. Cover. Geometrical and Statistical Properties of Systems of Linear Inequalities with Applications in Pattern Recognition. *IEEE Transactions on Electronic Computers*, EC-14(3):326–334, June 1965. ISSN 0367-7508. doi: 10.1109/PGEC.1965.264137. URL <http://ieeexplore.ieee.org/document/4038449/>.
- K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, April 2002. ISSN 1941-0026. doi: 10.1109/4235.996017. URL <https://ieeexplore.ieee.org/document/996017>.
- Fenglei Fan, Juntong Fan, Dayang Wang, Jingbo Zhang, Zelin Dong, Shijun Zhang, Ge Wang, and Tieyong Zeng. Hyper-Compression: Model Compression via Hyperfunction, February 2026. URL <http://arxiv.org/abs/2409.00592>. arXiv:2409.00592 [cs.LG] version: 4.
- Vincenzo Fornaro. JustVugg/colibri, August 2026. URL <https://github.com/JustVugg/colibri>. original-date: 2026-07-01T12:27:49Z.
- Jonathan Frankle and Michael Carbin. The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks, March 2019. URL <http://arxiv.org/abs/1803.03635>. arXiv:1803.03635 [cs.LG].

- Song Han, Huizi Mao, and William J. Dally. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding, February 2016. URL <http://arxiv.org/abs/1510.00149>. arXiv:1510.00149 [cs.CV].
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep Residual Learning for Image Recognition, December 2015. URL <http://arxiv.org/abs/1512.03385>. arXiv:1512.03385 [cs.CV].
- David A. Huffman. A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the IRE*, 40(9):1098–1101, September 1952. ISSN 2162-6634. doi: 10.1109/JRPROC.1952.273898. URL <https://ieeexplore.ieee.org/document/4051119>.
- Hugo Koubbi, Borjan Geshkovski, and Philippe Rigollet. Homogenized Transformers, April 2026. URL <http://arxiv.org/abs/2604.01978>. arXiv:2604.01978 [math].
- Jean-Samuel Leboeuf, Frédéric LeBlanc, and Mario Marchand. Decision trees as partitioning machines to characterize their generalization properties, October 2020. URL <http://arxiv.org/abs/2010.07374>. arXiv:2010.07374 [cs.LG].
- Ming Li and Paul Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Texts in Computer Science. Springer, New York, NY, 2008. ISBN 978-0-387-33998-6 978-0-387-49820-1. doi: 10.1007/978-0-387-49820-1. URL <http://link.springer.com/10.1007/978-0-387-49820-1>.
- Derek Lim, Theo Moe Putterman, Robin Walters, Haggai Maron, and Stefanie Jegelka. The Empirical Impact of Neural Parameter Symmetries, or Lack Thereof, October 2024. URL <http://arxiv.org/abs/2405.20231>. arXiv:2405.20231 [cs.LG].
- J. Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, September 1978. ISSN 00051098. doi: 10.1016/0005-1098(78)90005-5. URL <https://linkinghub.elsevier.com/retrieve/pii/0005109878900055>.
- Eduardo Y. Sakabe, Felipe S. Abrahão, Alexandre Simões, Esther Colombini, Paula Costa, Ricardo Gudwin, and Hector Zenil. Binarized Neural Networks Converge Toward Algorithmic Simplicity: Empirical Support for the Learning-as-Compression Hypothesis, September 2025. URL <http://arxiv.org/abs/2505.20646>. arXiv:2505.20646 [cs].
- Salvatore Sanfilippo. antirez/ds4, August 2026. URL <https://github.com/antirez/ds4>. original-date: 2026-05-06T17:17:05Z.
- Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, 1 edition, May 2014. ISBN 978-1-107-05713-5 978-

1-107-29801-9. doi: 10.1017/CBO9781107298019. URL <https://www.cambridge.org/core/product/identifier/9781107298019/type/book>.

Michael Sipser. *Introduction to the theory of computation*. Cengage Learning, Australia Brazil Japan Korea Mexiko Singapore Spain United Kingdom United States, third edition, international edition edition, 2013. ISBN 978-1-133-18779-0 978-1-133-18781-3 978-0-357-67058-3.

Karen Ullrich, Edward Meeds, and Max Welling. Soft Weight-Sharing for Neural Network Compression, May 2017. URL <http://arxiv.org/abs/1702.04008>. arXiv:1702.04008 [stat.ML].

Guillermo Valle-Pérez, Chico Q. Camargo, and Ard A. Louis. Deep learning generalizes because the parameter-function map is biased towards simple functions, April 2019. URL <http://arxiv.org/abs/1805.08522>. arXiv:1805.08522 [stat].

Andrew Gordon Wilson. Deep Learning is Not So Mysterious or Different, July 2025. URL <http://arxiv.org/abs/2503.02113>. arXiv:2503.02113 [cs].

Hector Zenil, Santiago Hernández-Orozco, Narsis A. Kiani, Fernando Soler-Toscano, and Antonio Rueda-Toicen. A Decomposition Method for Global Evaluation of Shannon Entropy and Local Estimations of Algorithmic Complexity, June 2018. URL <http://arxiv.org/abs/1609.00110>. arXiv:1609.00110 [cs.IT].

Hector Zenil, Fernando Soler Toscano, and Nicolas Gauvrit. The Block Decomposition Method. In Hector Zenil, Fernando Soler Toscano, and Nicolas Gauvrit, editors, *Methods and Applications of Algorithmic Complexity: Beyond Statistical Lossless Compression*, pages 125–149. Springer, Berlin, Heidelberg, 2022. ISBN 978-3-662-64985-5. doi: 10.1007/978-3-662-64985-5_6. URL https://doi.org/10.1007/978-3-662-64985-5_6.